SYSTEM DESIGN INTERVIEW SERIES · VOLUME I

The System Design Interview Playbook

A growing compendium of real interview problems, solved end-to-end: rigorous definitions, back-of-the-envelope mathematics, protocol design, and production-grade Rust implementations.

IN THIS VOLUME

Providence Salumu

First Edition · September 2026 · A living document — chapters are added incrementally

The System Design Interview Playbook

Volume I · First Edition

Written and typeset by **Providence Salumu**
Composed in Typst · Body set in Noto Serif · Display set in Noto Sans · Code set in Noto Sans Mono

REVISION HISTORY

v0.1	Chapter 1 — Real-Time Collaborative Text Editor	2026-09-05
v0.2	Chapter 2 — Maps & Navigation Service (Google Maps)	2026-09-05
v0.3	Chapter 3 — Distributed Rate Limiter	2026-09-05
v0.4	Chapter 4 — Distributed Logging & Metrics Platform	2026-09-05
v0.5	Chapter 5 — Hierarchical Comment System (Reddit)	2026-09-05
v0.6	Chapter 6 — Top-K Leaderboard (Gaming)	2026-09-05
v0.7	Chapter 7 — Recommendation Engine (YouTube / TikTok)	2026-09-05

Contents

FRONT MATTER

Preface

This book exists because system design interviews reward a very specific skill that ordinary engineering work rarely exercises in full: *taking an ambiguous, enormous problem and shrinking it, in real time and out loud, into a coherent, defensible architecture.* Knowing how Netflix or Google Docs works is not enough; you must be able to **derive** such a design from first principles while an interviewer watches you think.

> **DEFINITION — System design interview**
>
> An interview format, common for senior engineering roles, in which the candidate is asked to design a large-scale software system (for example, "design YouTube" or "design a URL shortener") within 45–60 minutes. There is no single correct answer. The interviewer evaluates how you clarify ambiguity, structure the problem, justify trade-offs, and reason about scale, failure, and consistency.

Each chapter of this book is a complete, self-contained walkthrough of one such problem. Every chapter follows the same battle-tested structure, which mirrors the natural arc of a strong interview performance:

THE CHAPTER FRAMEWORK

1. **Problem statement** — the prompt exactly as an interviewer would pose it.
2. **Clarifying questions & scope** — shrinking an ambiguous prompt into a concrete target.
3. **Functional requirements** — what the system must *do*.
4. **Non-functional requirements** — how *well* it must do it (scale, latency, availability).
5. **Back-of-the-envelope estimation** — arithmetic that sizes the problem before designing.
6. **The core challenge** — the one or two genuinely hard ideas this problem tests.
7. **API & protocol design** — the contract between clients and servers.
8. **Data model & storage** — what we persist, where, and why.
9. **High-level architecture** — the boxes and the arrows, with every box justified.
10. **Deep dives** — the mechanisms, including complete Rust reference implementations.
11. **Trade-offs & alternatives** — what we gave up, and what we would do differently.
12. **Wrap-up** — likely follow-up questions, common pitfalls, and an interview checklist.

Conventions used in this book

Three conventions deserve explicit mention.

Terms are defined before they are used. System design is full of overloaded vocabulary — "consistency", "snapshot", "version vector" — and interviews are lost when a candidate uses a term they cannot define. Whenever a new concept appears, it is introduced in a **Definition** box before the surrounding text relies on it.

All code is written in Rust. Rust's explicit ownership and type system make concurrency logic — the hardest part of most designs — precise and auditable. The listings are not toys: they are faithful, compilable sketches of the real algorithms, with tests.

Numbers are always justified. Every estimate states its assumptions. A number without assumptions is decoration; a number with assumptions is engineering.

How this book grows

This is a living document. New chapters are appended over time, each tackling one new problem. The framework above is fixed, so you always know where to find the requirements, the math, the architecture, and the code — regardless of the problem.

Providence Salumu — September 2026

CHAPTER 1

Designing a Real-Time Collaborative Text Editor

PROBLEM SOURCE

This chapter solves the problem posed in the talk **"12: Design Google Docs / Real Time Text Editor"** from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*, 2024, 47 min). The talk walks the problem in mock-interview form — both a high-level design (HLD) and a low-level design (LLD) of the concurrency machinery. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol specifications, and Rust reference implementations.

1.1 The Problem Statement

The interviewer looks up and says:

> *"Design a real-time collaborative text editor — something like Google Docs. Multiple people should be able to edit the same document at the same time and see each other's changes as they happen."*

This is one of the richest prompts in the system design canon. It looks like a CRUD application — until you realize that its heart is a distributed-state synchronization problem with no trivially correct answer. The prompt deliberately leaves almost everything open: scale, feature scope, consistency guarantees, offline behavior. How you close those gaps is most of the grade.

DEFINITION — High-level design (HLD) / low-level design (LLD)

HLD is the architecture of the whole system: the major services, data stores, and network paths, and the justification for each. *LLD* is the internal mechanics of the interesting components: data structures, algorithms, and protocols. A strong interview performance does HLD first, then drills into LLD for the one or two components that carry the real difficulty — in this problem, the concurrency-control core.

1.2 Scope & Clarifying Questions

Never design the prompt you were handed; design the prompt you **negotiated**. The series this chapter draws on runs its interviews as a dialogue, and that is exactly how a real interview should feel. A strong opening exchange looks like this:

Speaker	Dialogue
Candidate	"Are we building plain text editing, or rich text — formatting, embedded images, tables?"

Speaker	Dialogue
Interviewer	"Plain text is fine. Assume a document is an ordered sequence of characters."
Candidate	"How many collaborators can edit one document simultaneously?"
Interviewer	"Up to 50 concurrent editors per document."
Candidate	"And the total scale — how many users and documents are we designing for?"
Interviewer	"10 million daily active users. Documents themselves can be up to a few hundred kilobytes of text."
Candidate	"Do users need to see each other's cursors and selections live — presence?"
Interviewer	"Yes, cursors and names, like Google Docs shows."
Candidate	"Is offline editing in scope, or can we assume clients are connected while editing?"
Interviewer	"Assume connected for the core design; if time remains, sketch how offline would work."
Candidate	"Do we need version history — the ability to view and restore older revisions?"
Interviewer	"Yes, keep a full history of changes."

From this exchange we can freeze the scope. Everything later in the chapter traces back to these decisions, so state them explicitly and get a nod from the interviewer before moving on.

AGREED SCOPE

Plain-text documents up to 500 KB; up to **50 concurrent editors per document**; **10M daily active users**; live cursors and presence; full version history; connected editing for the core design (offline sketched as an extension).

INTERVIEW TIP — Negotiate scope in both directions

Candidates think clarifying questions only **shrink** scope. They also **expand** it deliberately: asking "do we need presence?" signals that you know the feature exists and costs something. Offer it, let the interviewer decide, and note the cost either way.

1.3 Functional Requirements

DEFINITION — Functional requirement (FR)

A statement of what the system must **do** — an observable behavior a user can verify, such as "a user can create a document." Functional requirements define the feature set; they say nothing about how fast, how available, or how consistent the system must be. Those qualities are *non-functional requirements*, defined next.

Our scoped functional requirements:

1. **FR-1 — Document management.** Users can create, rename, open, and delete documents.
2. **FR-2 — Real-time collaborative editing.** Multiple users can edit one document concurrently; every connected editor sees every other editor's changes within a fraction of a second.
3. **FR-3 — Presence.** Each editor sees who else is in the document, with a live cursor (and selection) per collaborator, labeled with their name.

4. **FR-4 — Persistence.** A document is never lost because a client disconnects; reopening it later yields its latest committed state.
5. **FR-5 — Version history.** Users can browse the full history of a document and view or restore any earlier revision.
6. **FR-6 — Sharing & access control.** A document has an owner; the owner grants view or edit access to others via a shareable link or explicit invite.

Out of scope (say so explicitly): rich text, comments and suggestions, end-to-end encryption, and offline-first editing as a core flow.

1.4 Non-Functional Requirements

DEFINITION — Non-functional requirement (NFR)

A statement of how **well** the system must perform its functions: scale, latency, availability, durability, consistency. NFRs are where design decisions actually come from — two systems with identical functional requirements (a chat app and a payment ledger) can have opposite architectures because their NFRs differ.

Three qualities dominate this problem, and they deserve precise vocabulary:

DEFINITION — Latency

The time between a cause and its observable effect. Two latencies matter here: *edit latency* (my keystroke appears in my own document — must feel instant, under 16 ms, so it must be applied locally without a network round trip) and *propagation latency* (my keystroke appears on a collaborator's screen — our target: under 150 ms for users in the same region).

DEFINITION — Availability

The fraction of time the system is able to serve requests. Editing must remain available even when individual servers fail, so no single machine may be indispensable. We will aim for the classic "three nines" (99.9% ≈ 8.8 hours of downtime per year) for the editing path.

DEFINITION — Consistency / convergence

In a system where many actors mutate shared state concurrently, *consistency* describes what guarantees observers get about the state they see. For a collaborative editor the crucial guarantee is **convergence**: once the same set of edits has been delivered to every replica, every replica must hold the **identical document**. If two users could permanently end up looking at different text, the product is broken no matter how fast it is. How convergence is achieved — without locking the document — is the core challenge of this chapter and gets its own section (Section 1.6).

The remaining NFRs, stated as targets:

Quality	Target
Propagation latency	p50 < 150 ms within a region; p95 < 400 ms cross-region
Edit (local echo) latency	< 16 ms — edits apply locally, synchronously, before any server contact
Editors per document	Up to 50 concurrent, no degradation
System scale	10M DAU; 1M simultaneous connections at peak

Quality	Target
Durability	No committed edit is ever lost (history is the product's memory)
Availability	99.9% for editing; reading a document should survive almost anything

KEY INSIGHT — Consistency is the NFR that shapes this design

Most interview problems are dominated by throughput (design Twitter) or storage (design Dropbox). This one is dominated by **correctness under concurrency**. The moment two users type at the same time, you need a mathematical answer to "what is the document now?" — Section 1.6 is where the interview is won or lost.

1.5 Back-of-the-Envelope Estimation

DEFINITION — Back-of-the-envelope estimation

Rapid, approximate arithmetic — using round numbers and stated assumptions — that sizes a system **before** designing it. Its purpose is not precision; it is to discover which constraints bite. A design for 500 messages per second and a design for 500,000 messages per second are different systems, and you cannot know which one you owe the interviewer until you count.

DEFINITION — DAU / QPS

DAU (daily active users) counts distinct users active in a day. *QPS* (queries per second) is the request rate a system handles. Interview estimates usually convert DAU into QPS via an activity model ("each user does X, Y times a day").

Assumptions (state them, write them down, invite correction):

- 10M DAU; each user actively edits on 3 documents per day, 20 minutes per session.
- At peak, 5% of DAU are simultaneously connected: **500k concurrent connections** (we design for 1M to leave headroom).
- An active typist produces 3 keystroke-operations per second in bursts; averaged over a session (reading, thinking, pausing), 0.5 ops/sec per connected user.
- One operation (a character insert/delete plus metadata) ≈ 250 bytes on the wire.
- Average document size: 20 KB of text (500 KB worst case per our scope).

Derived numbers:

Quantity	Estimate	How
Peak edit operations	≈ 250k ops/sec	500k conns × 0.5 ops/s
Edit ingress bandwidth	≈ 60 MB/s	250k ops/s × 250 B
Operations stored per day	≈ 2.2B	25k ops/s avg × 86,400 s
Operation log growth	≈ 550 GB/day	2.2B ops × 250 B
New documents per day	≈ 3M	10M users × 0.3
Doc-open QPS (REST)	≈ 600 avg / 6k peak	10M × 5 opens/day ÷ 86,400, ×10 peak factor

Quantity	Estimate	How
Connections per WebSocket gateway	50k	commodity servers hold tens of thousands of idle-ish connections
Gateways needed	20–40	1M conns ÷ 25–50k each, plus redundancy

KEY INSIGHT — What the math tells us

Two facts jump out. First, the **system-wide** write rate (hundreds of thousands of ops/sec) is large but utterly **sharded by document**: each document sees at most 50 editors producing maybe 150 ops/sec in a frantic burst — trivial for one thread. Second, the connection tier is where the scale lives: a million mostly- idle WebSocket connections is a capacity problem, not a correctness problem. Conclusion: put the hard correctness logic **per document** (where throughput is tiny) and make the connection tier **stateless and horizontal** (where throughput is huge). This observation drives the entire architecture in Section 1.8.

1.6 The Core Challenge: Concurrent Editing

Everything else in this chapter is standard distributed-systems plumbing. This section is the intellectual center of the problem — and of the source talk.

1.6.1 Why naive approaches fail

Suppose two users, Alice and Bob, are editing the document `"GO"` — a two-character document: `G` at index 0, `O` at index 1. They type **at the same time**:

- **Alice** inserts the character `A` at index 0 (she is turning `"GO"` into `"AGO"`).
- **Bob** deletes the character at index 1 (he is turning `"GO"` into `"G"`).

What should the document be when both edits are delivered? Both users' intentions are clear, and they do not conflict in spirit: the answer is `"AG"` — Alice's `A` before the `G`, and the `O` gone. Now examine two naive strategies:

Naive strategy 1 — lock the document. Only one editor at a time; everyone else waits. This is **correct** but useless: it defeats the entire purpose of the product. Real-time collaboration means no global mutual exclusion.

DEFINITION — Mutual exclusion / locking

A concurrency-control technique in which a resource may be modified by only one actor at a time; others block until the lock is released. Locks serialize work, guaranteeing correctness at the cost of parallelism — fatal when the resource is a shared document with 50 simultaneous editors.

Naive strategy 2 — last-writer-wins. Let each edit carry a timestamp; the edit with the latest timestamp prevails, earlier conflicting edits are discarded.

DEFINITION — Last-writer-wins (LWW)

A conflict-resolution rule: when two writes to the same datum race, keep the one with the later timestamp and drop the other. Simple and fast, but it **loses data by design** — one of Alice's or Bob's edits would silently vanish. LWW is acceptable for overwritten fields (a profile picture), never for merged text.

Applying LWW to our example: if Bob's delete "wins", Alice's `A` disappears. If Alice's insert "wins", Bob's delete is lost and the `O` resurrects. Neither user did anything wrong, yet one of their intentions

was destroyed. And naive **position-based** application is worse still: if Bob's `delete index 1` is applied to Alice's already-updated `"AGO"`, it deletes the `G` instead of the `O`, producing `"AO"` — a document **neither** user intended.

1.6.2 What "correct" means

We need guarantees that can be stated precisely and proven. Three of them:

> **DEFINITION — Causality**
>
> Event A *causally precedes* event B if A happened first and could have influenced B — for example, B is an edit made by a user who had already seen A applied. Two events with no causal order either way are **concurrent**. Alice's and Bob's edits above are concurrent: neither saw the other's. Concurrency — not time — is what creates conflicts; timestamps cannot detect it, which is why Section 1.7 introduces version vectors.

> **DEFINITION — Convergence**
>
> A replica set converges if, once the **same set** of edits has been delivered to every replica, every replica holds the **identical** document — regardless of the order in which the edits arrived. Convergence is a property of the algorithm, not of luck: it must hold for every possible interleaving.

> **DEFINITION — Intention preservation**
>
> The effect of an edit, as observed by its author at the moment they made it, must survive concurrency with other edits. Alice meant "`A` before `G`" and Bob meant "`O` is gone"; the converged document `"AG"` honors both. An algorithm can converge and still be bad (converging to the empty string is trivial), so convergence alone is not enough — we want convergence **with** intention preservation.

There are exactly two families of algorithms in wide production use that deliver these properties: **Operational Transformation** and **Conflict-free Replicated Data Types**. A candidate who can define, contrast, and implement one of them owns this interview.

1.6.3 Approach A — Operational Transformation (OT)

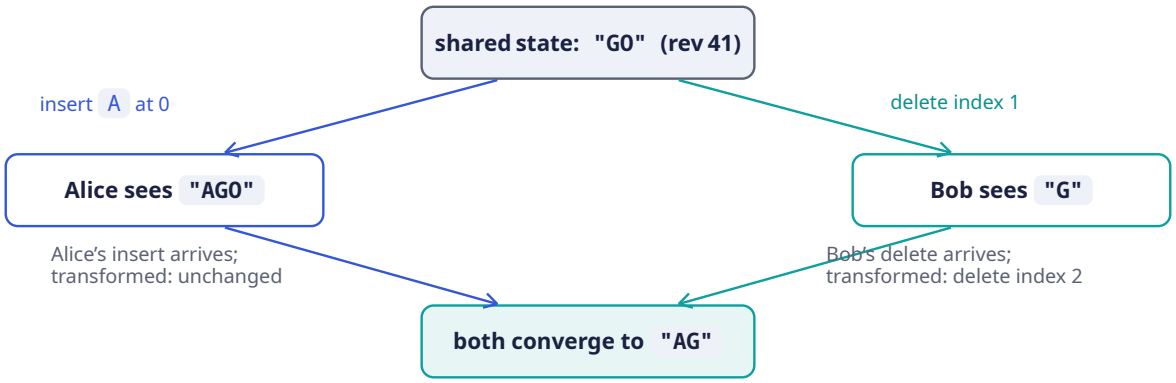
> **DEFINITION — Operational Transformation (OT)**
>
> A concurrency-control technique in which every edit is expressed as an **operation** — for plain text, `insert(position, char)` or `delete(position)` — and, when two operations are concurrent, one is **transformed** against the other: its position is adjusted so that it applies correctly to a document the other operation has already modified. OT powers Google Docs. Its lineage: the Jupiter system (1995), then Google Wave, then Docs.

The transformation rules are a small, complete matrix. To apply operation `a` after concurrent operation `b` has already been applied, transform `a` against `b`:

Pair	Rule for transforming `a` against already-applied `b`
`insert(p_a)` vs `insert(p_b)`	If `p_b < p_a`, shift `a` right by 1. If `p_b == p_a`, break the tie deterministically (e.g., by client ID) so every replica shifts the same way.
`insert(p_a)` vs `delete(p_b)`	If `p_b < p_a`, shift `a` left by 1. Otherwise unchanged.
`delete(p_a)` vs `insert(p_b)`	If `p_b <= p_a`, shift `a` right by 1. Otherwise unchanged.

Pair	Rule for transforming `a` against already-applied `b`
`delete(p_a)` vs `delete(p_b)`	If `p_b < p_a`, shift `a` left by 1. If `p_b == p_a`, both deletes target the same character: `a` becomes a no-op.

The whole of OT is this matrix plus a discipline: **every replica applies the same operations in the same total order, transforming as needed.** Our running example, drawn as the classic OT convergence diamond — both paths from the concurrent state must land on the same document:



Walking the diamond: on Alice's side, Bob's `delete(1)` is transformed against her already-applied `insert(0, A)` — since the insert sits at or before the delete position, the delete shifts right to `delete(2)`, which removes the `O` from `"AGO"` and yields `"AG"`. On Bob's side, Alice's `insert(0, A)` is transformed against his already-applied `delete(1)` — the insert position is before the delete, so it applies unchanged to `"G"`, yielding `"AG"`. Same operations, same converged document, both intentions preserved.

> **NOTE — Where the total order comes from**
>
> OT requires every replica to transform against the **same** concurrent history. The standard way to arrange that is a **single sequencer** per document: the server assigns every operation the next integer **revision** (42, 43, 44, …), and that assignment *is* the total order. Clients send operations tagged with the revision they were based on; the server transforms them across any operations committed in between. We make this precise in the deep dives (Sections 1.9 and 1.10).

1.6.4 Approach B — Conflict-free Replicated Data Types (CRDTs)

> **DEFINITION — Eventual consistency**
>
> A consistency model in which replicas that have stopped receiving new writes will **eventually** reach the same state. It promises convergence but says nothing about **when**, and nothing about what intermediate states look like.

> **DEFINITION — Conflict-free Replicated Data Type (CRDT)**
>
> A data structure whose concurrent updates are merged by rules that are commutative, associative, and idempotent — so replicas that apply the same updates in **any** order, even with duplicates, provably converge. This upgraded guarantee is called **strong eventual consistency**: convergence as soon as updates are delivered, with no central ordering and no transformation step.

For text, the canonical CRDT trick is to stop addressing characters by **integer positions** (which shift under concurrency — the very problem OT transforms away) and instead give every character a

unique, immutable, orderable identifier. In the RGA family of algorithms, an insert is recorded as "insert this new character, with a fresh globally-unique ID, immediately after character ID X." Concurrent inserts after the same character interleave deterministically by ID order; deletes merely mark a character as a tombstone (present in structure, invisible in text). Because identifiers never shift, no transformation is ever needed — merging is just set union.

1.6.5 Choosing between them

Criterion	OT	CRDT
Message size	Tiny: position + char	Larger: globally-unique IDs per character
Memory / metadata	None beyond the text	ID + tombstone per character; tombstone growth needs garbage collection
Needs central sequencer	Yes (classic form)	No — tolerates any delivery order
Algorithmic subtlety	Transformation matrix + history; easy to get subtly wrong	Merge rules; conceptually cleaner, harder to keep compact
Offline / peer-to-peer	Awkward (needs the ordering authority)	Natural fit
Used in production by	Google Docs, older Etherpad	Figma (variants), many local-first tools

Our decision: classic OT behind a per-document sequencer — the architecture the source talk describes, and the one Google Docs runs on. The sequencer is not a bottleneck (Section 1.5 showed a document peaks at 150 ops/sec) and it collapses the hard problem into "transform against a linear log," which we can implement exactly. We keep version vectors (next section) for reconnect and catch-up.

> **INTERVIEW TIP — Name the trade-off out loud**
>
> Saying "I'll use OT because Google Docs does" earns no points. Saying "I'll use OT with a per-document sequencer, accepting a single point of ordering per document — which I mitigate with failover — in exchange for small messages and a linear history that makes version history trivial" is a senior answer. Every choice in this chapter is presented as benefit-purchased-at-cost.

1.7 Version Vectors: Tracking What Each Replica Has Seen

The OT diamond handles the moment of concurrency. But a real system must also answer a bookkeeping question constantly: **which operations has this client already received?** On reconnect, on history fetch, on offline sync, the server and client must compare "what you have seen" against "what exists." Timestamps cannot express this (clock skew makes "when" unreliable across machines). We need a **logical** notion of time.

> **DEFINITION — Logical clock / Lamport timestamp**
>
> A counter, maintained by each participant, that ticks on every local event and is carried with every message; receivers merge it by taking the maximum and ticking on. Logical clocks order events by causality rather than by wall-clock time, which is exactly what distributed state needs — wall clocks on different machines cannot be trusted to agree.

DEFINITION — Version vector

A map from participant ID to a counter: `{Alice: 3, Bob: 1}` means "I have seen 3 events from Alice and 1 from Bob." Two version vectors compare pointwise:

- V_1 **dominates** V_2 (V_2 happened-before V_1) if every entry of V_1 is ≥ the corresponding entry of V_2, and at least one is strictly greater — V_1 has seen everything V_2 has, and more.
- If neither dominates, the two states are **concurrent** — each has seen something the other has not.

A version vector is a Lamport clock generalized to many writers: it captures exactly which causal history a replica has absorbed.

In our design the server already assigns a single integer revision per document (the sequencer), so the **steady-state** sync marker is just `rev = 47`. Version vectors earn their place at the edges of the design: when a client reconnects after a network blip during which a **snapshot** was taken, when replicas in different regions compare state, and in the offline extension. The comparison logic is the same everywhere: if the server's knowledge dominates the client's, send exactly the difference.

COMMON PITFALL — The snapshot / version-vector trap

The source talk's wry description — a client "never received those writes on your local copy since your version vector was more up to date than the document snapshot" — points at a real and common bug. Scenario: a snapshot of the document is written to one store, but the version metadata recording **which revisions the snapshot covers** is written to another, and the two writes are not atomic. A reconnecting client compares its version vector against **stale** snapshot metadata, concludes it is up to date, and never receives the missing operations. The cure is atomicity between a snapshot and its version metadata — Section 1.11 shows how, and defines the two-phase commit protocol the talk name- drops for exactly this purpose.

1.8 API & Protocol Design

The system has two distinct planes, and separating them is itself a design decision worth naming:

- The **control plane** — create/rename/share/open documents, fetch history. These are classic request/response operations; plain HTTPS REST is correct and simple.
- The **data plane** — the live editing session: join, send operations, receive operations, presence. This needs a persistent, bidirectional, low-latency channel per client.

DEFINITION — REST

An API style over HTTP in which resources (nouns: `/documents/{id}`) are manipulated with a fixed set of methods (verbs: `GET`, `POST`, `PATCH`, `DELETE`). Stateless by design: each request carries everything the server needs. Ideal for our control plane.

DEFINITION — WebSocket

A protocol (RFC 6455) that upgrades an HTTP connection into a long-lived, full-duplex channel: after an initial handshake, **either** side may send a message at **any** time, with a few bytes of framing overhead. This is what makes sub-150 ms propagation possible — the server can push an edit the instant it is committed, with no client polling and no new connection setup.

Why not the alternatives?

Mechanism	Behavior	Verdict
Short polling	Client re-asks "anything new?" every few seconds	Latency of seconds; wasted requests — rejected
Long polling	Server holds the request open until data exists	Near-real-time but a new HTTP request per message burst — heavy at our scale
Server-Sent Events	Server → client push over HTTP; client sends via separate requests	Workable, but asymmetric; WebSocket is cleaner for a two-way edit stream
WebSocket	One persistent, bidirectional connection	**Chosen for the data plane**

1.8.1 Control-plane endpoints

Method	Path	Purpose
`POST`	`/documents`	Create a document; returns its `doc_id`
`GET`	`/documents/{id}`	Fetch metadata + the latest committed revision number
`PATCH`	`/documents/{id}`	Rename, change settings
`DELETE`	`/documents/{id}`	Delete (owner only)
`POST`	`/documents/{id}/share`	Grant view/edit access to a user or link
`GET`	`/documents/{id}/history?from=&to=`	List revisions for the version-history UI
`GET`	`/documents/{id}/snapshot?rev=`	Fetch the document as of any revision (view/restore)

1.8.2 Data-plane protocol (WebSocket messages)

After the REST `GET`, the client opens a WebSocket to the gateway and speaks a small JSON protocol. Every message has a `type`; the important ones:

Message	Direction	Payload & semantics
`join`	client → server	`{doc_id, auth_token, last_seen_rev}` — enter the session; `last_seen_rev` enables catch-up
`joined`	server → client	`{doc_id, rev, snapshot, collaborators[]}` — current state, possibly after catch-up
`op`	client → server	`{doc_id, base_rev, op, client_seq}` — one edit, based on `base_rev`, idempotent via `client_seq`
`ack`	server → client	`{client_seq, rev}` — "your edit is committed as revision `rev`"
`op` (broadcast)	server → client	`{rev, author, op}` — a committed edit to apply (transform locally if needed)
`presence`	both	`{cursor, selection, name}` — ephemeral, never persisted
`sync`	server → client	`{rev}` / snapshot — sent when the client is too far behind to patch with ops

Example committed-operation broadcast:

```
{
  "type": "op",
  "doc_id": "d_8f31c2",
  "rev": 47,
  "author": "alice@example.com",
  "op": { "kind": "insert", "pos": 0, "ch": "A" }
}
```

Two protocol details deserve emphasis because interviewers probe them. First, `base_rev` is how the server detects concurrency: if the document is at revision 52 and your operation says `base_rev: 49`, the server transforms it across revisions 50–52 before committing (Section 1.10). Second, `client_seq` is an **idempotency key**:

> **DEFINITION — Idempotency / idempotency key**
>
> An operation is *idempotent* if performing it twice has the same effect as performing it once. Networks retry; clients reconnect and resend. A unique client-supplied key per operation (here, `(client_id, client_seq)`) lets the server recognize a duplicate delivery and answer with the original result instead of applying the edit twice — the difference between at-least-once **delivery** and effectively exactly-once **effect**.

1.9 Data Model & Storage

Four persistent entities, each with a clear job:

Entity	Contents	Store
`Document`	`doc_id`, owner, title, ACL, current `rev`	Metadata DB (relational or document store; sharded by `doc_id`)
`Operation`	`doc_id`, `rev`, author, transformed op payload, `client_seq`, timestamp	Operation log — append-only store, ordered by `rev` per document
`Snapshot`	`doc_id`, `base_rev`, full document text	Blob/object store; pointer + `base_rev` in metadata DB
`Session`	who is connected, cursors	Ephemeral presence cache — **not** persisted

The two storage ideas that interviewers reward:

> **DEFINITION — Operation log (event log)**
>
> Storing every change as an immutable, ordered, append-only record instead of overwriting state. Current state is always derivable by replaying the log. For us the log **is** the product's memory: version history (FR-5) is free, auditing is free, and recovery is replay. Append-only logs are also the fastest possible write pattern — sequential appends, no random updates.

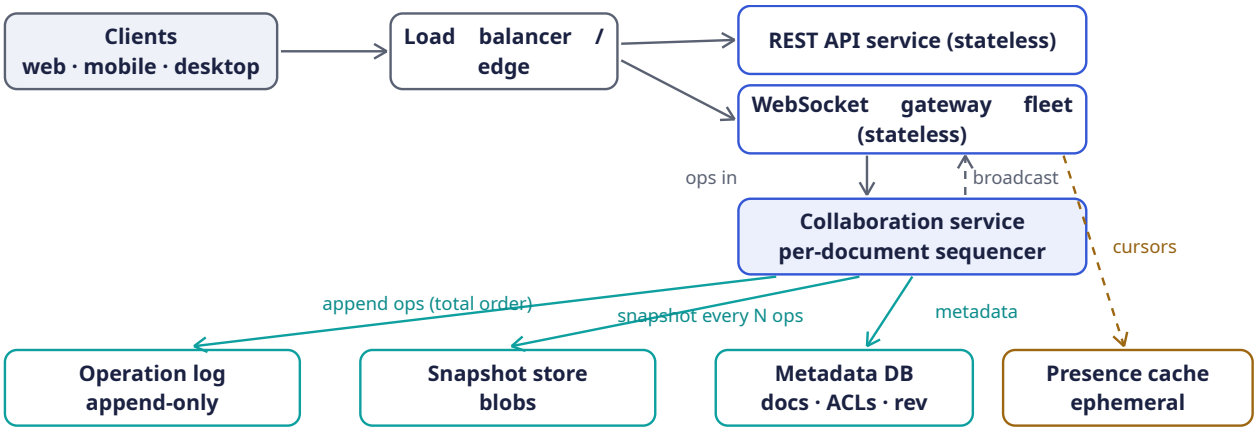
> **DEFINITION — Snapshot**
>
> A materialized copy of the full document as of a specific revision. Replaying 2 billion operations to open a document is absurd; instead store a snapshot every *N* operations (say 500) plus the handful

of operations since, and reconstruct state as `snapshot + tail of the log`. Snapshots are a read optimization over the log, never the source of truth — the log is.

> **NOTE — Why a sharded relational/document store + blob store, not one database**
>
> The metadata (small, hot, transactional) wants indexes and conditional updates; snapshots (large, immutable, cold) want cheap object storage; the operation log (append-only, ordered per key) wants a log-structured store. Choosing one engine per access pattern — and **saying why** — is the answer this section exists to elicit. In an interview, name concrete instincts: PostgreSQL-class for metadata, an S3-class blob store for snapshots, a Kafka/Cassandra-class append store for the log — while stressing the **properties** matter more than the brands.

1.10 High-Level Architecture

The estimation (Section 1.5) gave us the organizing principle: **a thin, stateless, horizontally scaled connection tier in front of a small, stateful, correctness- critical core.** Here is the whole system at a glance:



REST tier reads/writes Metadata DB directly; gateway ↔ collaboration routing is per-document (sticky)

Component responsibilities, and — more importantly — the reason each exists:

Component	Responsibility	Why it is shaped this way
WebSocket gateway	Hold 50k client connections each; terminate TLS; forward messages	Stateless: any client can reconnect to any gateway. Scale by adding boxes — this tier absorbs the 1M-connection problem
Collaboration service	Owns each active document: sequences ops, transforms, commits, broadcasts	Stateful but **sharded by document**: a document's whole session lives on one node, so its sequencer is just an in-memory counter. Throughput per doc is tiny (Section 1.5), so one node hosts thousands of docs
Operation log	Durable, ordered, append-only record per document	Source of truth; powers history (FR-5), replay, recovery

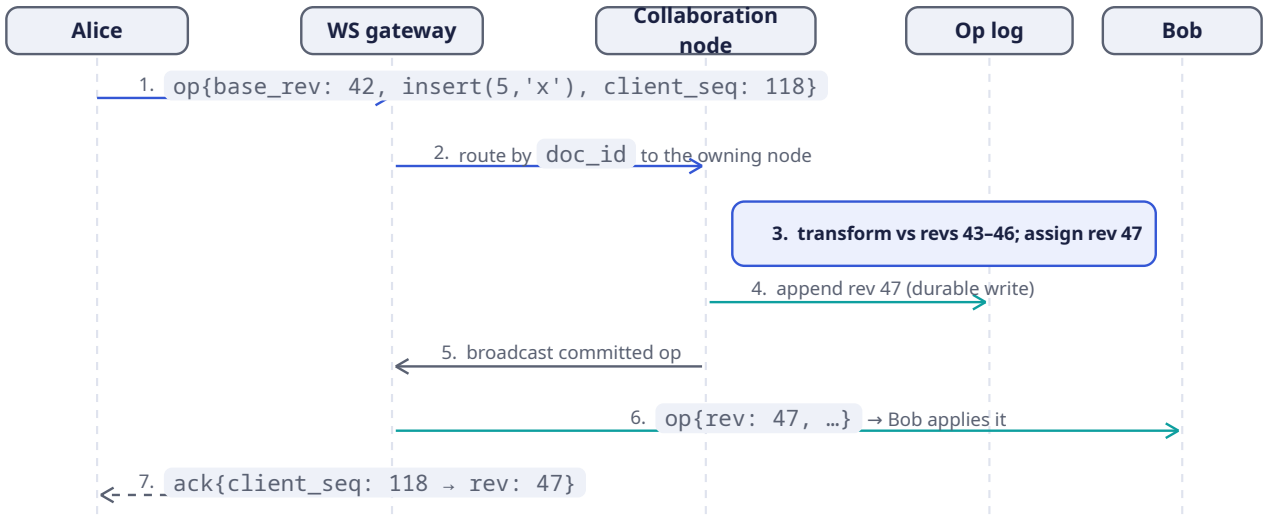
Component	Responsibility	Why it is shaped this way
Snapshot store	Immutable document snapshots every N ops	Makes open/reconnect O(1) instead of O(history)
Metadata DB	Documents, ACLs, current revision counters	Small, hot, transactional reads/writes
Presence cache	Live cursors and collaborator lists	Ephemeral by nature — losing it costs nothing; it rebuilds in seconds
REST API service	Control plane (Section 1.8)	Stateless request/response work; ordinary horizontal scaling

KEY INSIGHT — The one routing rule that makes it all work

All traffic for a given document flows through the single collaboration node that owns it. Gateways look up `doc_id → owner` (a cheap routing-table read) and forward. Because every operation on a document passes through one owner, the owner's revision counter is a **total order** — which is exactly what OT needs (Section 1.6). Ownership can migrate on failure; the invariant is that there is exactly one owner at a time.

1.11 Deep Dive: The Edit Pipeline, Step by Step

Follow one keystroke from Alice's keyboard to Bob's screen. The document `d_8f31c2` is at revision 46; Alice believes it is at revision 42 (she has not yet received four operations from other editors).



In words:

1. **Send.** Alice's keystroke is applied to her local document **immediately** (local echo — this is how edit latency stays under 16 ms) and sent as `op{base_rev: 42, …}` with her next `client_seq`.
2. **Route.** The gateway does no correctness work; it reads `doc_id`, finds the owning collaboration node, forwards.
3. **Sequence & transform.** The owner sees `base_rev: 42` but the document is at 46: Alice's operation is **concurrent** with revisions 43–46. It transforms her operation against each of them, in order (Section 1.13 implements this loop), then stamps it with the next revision, 47.

4. **Commit.** The transformed operation is appended to the operation log. Once the append is durable, the edit is **committed** — it can never be reordered or lost.
5. **Broadcast.** The committed operation goes to the gateways of every connected editor of the document.
6. **Apply.** Bob's client is not behind (its `base_rev` matches), so it applies the operation verbatim. If Bob had a **pending** unacknowledged local edit, he would first transform it against this incoming operation — the client runs the same transform matrix as the server, keeping his local echo intact.
7. **Acknowledge.** Alice learns her operation is committed as revision 47; her pending buffer clears. Her screen never flickered — the local echo and the committed result are identical by construction.

> **COMMON PITFALL — Order matters: commit before broadcast**
>
> If you broadcast before the log append is durable, a node crash between the two steps shows users edits that no longer exist — and on recovery, the "committed" history disagrees with what people saw. Durability first, **then** visibility. This is the write-path equivalent of "don't acknowledge what you can't guarantee."

1.12 Deep Dive: Snapshots, Catch-Up & the Two-Phase Lesson

Two housekeeping jobs remain: making **open/reconnect** fast, and keeping a snapshot and its metadata **atomic**. Both live at the seam between the operation log and the snapshot store.

1.12.1 Catch-up: what to send a rejoining client

When a client sends `join{last_seen_rev}`, the owner compares it to the current revision `R`:

- **Close behind** (`R - last_seen_rev ≤ 500`): send the missing operations from the log. The client transforms any pending local edits across them and is current.
- **Far behind** (offline for hours): replaying thousands of ops is slower than sending state. Send `latest snapshot + ops since the snapshot's base_rev`.
- **Offline edits** (the extension): the client presents its **version vector** (Section 1.7) instead of a scalar revision; the server computes the set difference — ops the client missed — and the client rebases its offline ops on top. Vectors, not timestamps, because they compare **causal** histories.

1.12.2 The atomicity requirement

A snapshot is useless without its metadata (`base_rev`: which revisions it covers), and dangerous with **stale** metadata — that is exactly the failure the source talk jokes about: a client whose version vector is "more up to date than the document snapshot" concludes it needs nothing and **silently misses writes**. Two writes must succeed or fail **together**: the snapshot blob, and the `{snapshot_pointer, base_rev}` record in the metadata DB.

> **DEFINITION — Two-phase commit (2PC)**
>
> A protocol for committing one atomic operation across **multiple** stores. Phase 1 (*prepare*): a coordinator asks every participant "can you commit?" Each participant does everything short of committing and votes yes/no. Phase 2 (*commit*): if all voted yes, the coordinator tells everyone to commit; if any voted no, everyone aborts. 2PC guarantees atomicity but is **blocking** (a crashed coordinator can leave participants waiting) and slow (two round trips) — so we use the minimal version of the idea, not the maximal one.

In practice, prefer the cheapest construction that delivers atomicity:

1. **One store, one transaction.** If the snapshot blob and its metadata live in the same database, a single transaction suffices — no 2PC at all.

2. **Immutable blob + conditional pointer update** (our choice). Write the snapshot to the blob store under a content-addressed name; only when that write is durable, update the metadata row with a conditional compare-and-swap (`UPDATE … WHERE base_rev = <old>`). A crash between the steps leaves an **orphan blob** — garbage-collectable, never incorrect. Atomicity without a coordinator.
3. **Full 2PC** is reserved for when two genuinely independent systems must commit together — worth naming in the interview precisely so you can argue you rarely need it.

1.13 Deep Dive: Rust Reference Implementation

The concurrency core is small enough to show in full. Three pieces: the wire protocol types, the OT transform matrix with tests, and the version vector.

1.13.1 Protocol types

The data-plane messages from Section 1.8, as Rust types (serde for JSON):

```rust
use serde::{Deserialize, Serialize};

/// One edit to a plain-text document.
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum Op {
    Insert { pos: usize, ch: char },
    Delete { pos: usize },
}

/// Data-plane messages (client ⇄ server), serialized as JSON.
#[derive(Clone, Debug, Serialize, Deserialize)]
#[serde(tag = "type", rename_all = "snake_case")]
pub enum Msg {
    Join    { doc_id: String, auth: String, last_seen_rev: u64 },
    Joined  { doc_id: String, rev: u64, snapshot: String },
    Op      { doc_id: String, base_rev: u64, client_seq: u64, op: Op },
    Ack     { client_seq: u64, rev: u64 },
    /// Server → clients: a committed operation.
    Bcast   { doc_id: String, rev: u64, author: String, op: Op },
}
```

The `#[serde(tag = "type")]` attribute makes the JSON look exactly like the protocol table: a `type` field distinguishes the variants.

1.13.2 The OT transform matrix

This is Section 1.6's table, made executable. `transform(a, b, a_wins_ties)` returns the operation `a` adjusted to apply **after** the concurrent operation `b` has been applied:

```rust
use crate::Op;

/// Transform `a` so it applies correctly *after* concurrent op `b`.
/// `a_wins_ties` breaks same-position insert/insert ties identically
/// on every replica (e.g., by client id) — determinism is what matters.
pub fn transform(a: &Op, b: &Op, a_wins_ties: bool) -> Option<Op> {
    match (a, b) {
        (Op::Insert { pos: pa, ch: ca }, Op::Insert { pos: pb, .. }) => {
            let shifted = if *pa > *pb || (*pa == *pb && !a_wins_ties) { pa + 1 } else
{ *pa };
            Some(Op::Insert { pos: shifted, ch: *ca })
        }
        (Op::Insert { pos: pa, ch: ca }, Op::Delete { pos: pb }) => {
            let shifted = if *pa > *pb { pa - 1 } else { *pa };
            Some(Op::Insert { pos: shifted, ch: *ca })
```

```
        }
        (Op::Delete { pos: pa }, Op::Insert { pos: pb, .. }) => {
            let shifted = if *pa >= *pb { pa + 1 } else { *pa };
            Some(Op::Delete { pos: shifted })
        }
        (Op::Delete { pos: pa }, Op::Delete { pos: pb }) => {
            if pa == pb { None }                        // both deleted the same char
            else { Some(Op::Delete { pos: if *pa > *pb { pa - 1 } else { *pa } }) }
        }
    }
}

/// Apply an operation to a document buffer.
pub fn apply(doc: &mut String, op: &Op) {
    match op {
        Op::Insert { pos, ch } => {
            let byte = doc.char_indices().nth(*pos).map_or(doc.len(), |(i, _)| i);
            doc.insert(byte, *ch);
        }
        Op::Delete { pos } => {
            if let Some((byte, c)) = doc.char_indices().nth(*pos) {
                doc.drain(byte..byte + c.len_utf8());
            }
        }
    }
}
```

Note the discipline the code makes explicit: character positions, not byte offsets (Rust strings are UTF-8; mixing the two is a classic production bug), and `None` for the delete/delete collision — an operation transformed into a no-op.

The convergence diamond from Section 1.6, as a test:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn go_diamond_converges() {
        let alice_op = Op::Insert { pos: 0, ch: 'A' };  // "GO" -> "AGO"
        let bob_op   = Op::Delete { pos: 1 };           // "GO" -> "G"

        // Alice's side: apply hers, transform Bob's against it, apply.
        let mut a_doc = String::from("GO");
        apply(&mut a_doc, &alice_op);
        let bob_at_alice = transform(&bob_op, &alice_op, false).unwrap();
        apply(&mut a_doc, &bob_at_alice);

        // Bob's side: apply his, transform Alice's against it, apply.
        let mut b_doc = String::from("GO");
        apply(&mut b_doc, &bob_op);
        let alice_at_bob = transform(&alice_op, &bob_op, true).unwrap();
        apply(&mut b_doc, &alice_at_bob);

        assert_eq!(a_doc, b_doc);          // convergence
        assert_eq!(a_doc, "AG");           // intention preservation
    }
}
```

1.13.3 The server-side session: transforming against history

The owner node keeps a per-document session. When an operation arrives based on an old revision, transform it across everything committed since (step 3 of the pipeline):

```
pub struct DocSession {
    pub rev: u64,
    pub doc: String,
    pub log: Vec<(u64, Op)>,        // (revision, committed op); the op log in memory
    pub since_snapshot: usize,      // ops committed since last snapshot
}

impl DocSession {
    pub fn commit(&mut self, base_rev: u64, mut op: Op) -> Result<u64, &'static str> {
        if base_rev > self.rev { return Err("base_rev from the future"); }
        // Transform across every revision the client had not seen.
        for (_, past) in self.log.iter().skip(base_rev as usize) {
            op = transform(&op, past, false).ok_or("op became a no-op")?;
        }
        self.rev += 1;
        apply(&mut self.doc, &op);
        self.log.push((self.rev, op.clone()));
        self.since_snapshot += 1;
        if self.since_snapshot >= 500 { self.snapshot(); }   // Section 1.9 policy
        Ok(self.rev)                                          // -> ack + broadcast
    }

    fn snapshot(&mut self) { /* write blob, then conditional pointer update */
        self.since_snapshot = 0;
    }
}
```

This is deliberately unglamorous — a counter and a loop. That is the payoff of the architecture: because **all** operations for a document funnel through one owner, the "distributed consensus" needed here is no consensus at all.

1.13.4 Version vector

Section 1.7's definition, implemented with its three-way comparison:

```
use std::collections::HashMap;

#[derive(Clone, Debug, Default, PartialEq, Eq)]
pub struct VersionVector(pub HashMap<String, u64>);

#[derive(Debug, PartialEq, Eq)]
pub enum Ordering { Before, After, Equal, Concurrent }

impl VersionVector {
    pub fn tick(&mut self, who: &str) { *self.0.entry(who.into()).or_insert(0) += 1; }

    /// Pointwise comparison: Before = self is causally dominated by other.
    pub fn compare(&self, other: &Self) -> Ordering {
        let mut less = false;
        let mut greater = false;
        for id in self.0.keys().chain(other.0.keys()) {
            let a = self.0.get(id).copied().unwrap_or(0);
            let b = other.0.get(id).copied().unwrap_or(0);
            less |= a < b;
            greater |= a > b;
        }
```

```
        match (less, greater) {
            (false, false) => Ordering::Equal,
            (true, false)  => Ordering::Before,
            (false, true)  => Ordering::After,
            (true, true)   => Ordering::Concurrent,
        }
    }

    /// Element-wise maximum: the merged causal history of two replicas.
    pub fn merge(&mut self, other: &Self) {
        for (id, &n) in &other.0 {
            let e = self.0.entry(id.clone()).or_insert(0);
            *e = (*e).max(n);
        }
    }
}
```

The catch-up decision from Section 1.12 is now three lines: if the client's vector is `Before` the server's, send the ops it lacks; if `Equal`, send nothing (and — per the pitfall — verify the **snapshot metadata** said the same thing); if `Concurrent`, the client has offline edits: send the diff and rebase.

1.14 Deep Dive: Presence & Live Cursors

Presence — "who is here, and where is their cursor" — looks trivial and hides one elegant detail: **cursor positions are text positions**, so they must be transformed by the same OT matrix as edits. If Alice's cursor sits at position 10 and Bob inserts three characters at position 4, Alice's cursor must render at 13. Clients transform remote cursors against every incoming operation; the server simply relays.

Design rules that keep presence cheap:

- **Never persist it.** Presence flows through the ephemeral presence cache and broadcasts; a client heartbeat every few seconds refreshes it. If presence data is lost, it rebuilds within one heartbeat — this is why the architecture draws it as a dashed, separate path.
- **Don't sequence it.** Presence messages carry no `rev` and bypass the operation log entirely; a stale cursor for 100 ms is invisible to users, while a stale **edit** is a correctness bug.
- **Throttle it.** Cursor moves fire dozens of events per second per editor; cap updates (e.g., 10/sec/client) — 50 editors × 10 updates = 500 tiny messages/sec, which the same per-document owner trivially fans out.

1.15 Scaling & Sharding

> **DEFINITION — Sharding (horizontal partitioning)**
>
> Splitting data or work across many machines by a key, so that each machine owns a disjoint slice. Sharding is the fundamental answer to "one machine is not enough": pick the right key and load grows by adding machines rather than by growing them.

Every tier shards by its natural key:

- **Gateways**: stateless, so any consistent routing works; connections balance across the fleet, and reconnects land anywhere.
- **Collaboration nodes**: sharded **by document**. A routing service maps `doc_id → owner` with a lease (a time-limited ownership grant the node must renew; on crash the lease expires and ownership

moves — this is how we keep "exactly one sequencer per document" without human intervention).
- **Operation log / snapshots / metadata**: sharded by `doc_id` as well, so all of a document's data is local to its shard.

> **KEY INSIGHT — The 50-editor cap is a load-shedding decision**
>
> Why can a document never become a "hot shard" that melts its owner? Because we capped concurrency at 50 editors — a product decision with an architectural dividend. A live-blog with a million **writers** would need a different design (tree of sequencers, or CRDTs with no central order). Naming which NFR protects you from which failure mode is exactly the reasoning interviewers listen for.

1.16 Failure Modes & Recovery

A senior answer enumerates what breaks **before** being asked. The playbook:

Failure	Handling
Client disconnects	Its edits are already durable in the log (commit-before-broadcast). On reconnect: `join` with `last_seen_rev`, catch up per Section 1.12. Presence entry expires on its own.
WebSocket gateway dies	Clients reconnect to any other gateway (they are stateless). In-flight unacked ops are resent and deduplicated by `client_seq` (idempotency).
Collaboration node dies	Lease expires; routing service assigns a new owner, which rebuilds the session state as `latest snapshot + log tail` and resumes sequencing. Clients see a reconnect, not data loss.
Duplicate message delivery	Impossible to prevent on real networks — so we make it harmless: `client_seq` idempotency on the write path, revision numbers on the read path (already have rev 47? ignore the re-broadcast).
Op-log replica loss	The log is replicated (3× is standard); a lost replica is replaced from its peers.
Network partition	A client that cannot reach the server keeps editing locally and rebases on reconnect (version-vector diff, Section 1.12) — or shows "offline, changes will sync" if we choose availability over freshness. A partitioned **server-side** minority refuses ownership transfers: correctness beats liveness there.

1.17 Trade-offs & Alternatives

The design is a bundle of purchased benefits and accepted costs. Saying the costs out loud is what distinguishes a design from a wish list:

Decision	Benefit purchased	Cost accepted
Per-document sequencer + OT	Small messages, total order, free version history, simple reasoning	Single point of **ordering** per doc (mitigated by lease failover, seconds of unavailability on crash);

Decision	Benefit purchased	Cost accepted
		OT matrix must be exactly right
CRDT instead	No sequencer; offline and peer-to-peer for free	Per-character IDs and tombstones; heavier storage and GC; harder to bolt on a clean linear history
WebSocket data plane	Sub-150 ms push; bidirectional	1M long-lived connections to manage; load balancers must tolerate them
Op log as source of truth	History, audit, replay, recovery all free	Storage growth (550 GB/day at our scale) and the snapshot machinery to keep reads fast
Single-region write ownership per doc	No cross-region consensus on the hot path	Editors far from the owner's region see higher propagation latency

NOTE — PACELC, briefly

A refinement of the CAP theorem: **if** there is a Partition, choose Availability or Consistency; **else** (normal operation), choose Latency or Consistency. Our design picks **consistency** on both branches for the edit path (single owner, total order) and **availability/latency** for presence (ephemeral, best-effort) — different subsystems may legitimately make different PACELC choices.

1.18 Observability & SLOs

DEFINITION — SLI / SLO

A *Service Level Indicator* is a measurable quantity (e.g., propagation latency at p95). A *Service Level Objective* is its target ("p95 < 400 ms"). SLOs turn "the system feels slow" into an engineering signal with an error budget.

What we instrument, at minimum: per-document ops/sec and transform rates; end-to- end propagation latency (client-side measured, tagged by region); ack latency; WebSocket connections and message rates per gateway; catch-up size distribution (reconnect storms reveal themselves here); snapshot lag (`current_rev - snapshot.base_rev`); lease-failover count. Alerts fire on SLO burn, not on individual machine failures — the failure table above exists precisely so that single failures are non-events.

1.19 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- *"Rich text?"* — Operations gain attributes (bold ranges, styles); the transform matrix grows cases, the architecture does not change.
- *"Comments / suggestions?"* — Anchored to character ranges that transform like cursors; stored in metadata DB, off the hot path.

- *"End-to-end encryption?"* — Server cannot read content, so server-side OT is impossible; this pushes you toward client-side CRDTs. A great example of a requirement that **re-architects** the core.
- *"5,000 simultaneous editors?"* — Our 50-editor cap is what licenses the single sequencer; beyond it, shard the document itself or move to CRDTs.
- *"Undo?"* — Per-user inverse operations transformed through the log — easy to describe, subtly hard to make intuitive; mention it, don't build it live.

If you remember five things:

1. The product is a distributed-state synchronizer wearing a text editor's clothes.
2. Convergence + intention preservation are the correctness bar; LWW and locks both fail it.
3. A per-document total order (sequencer) reduces "distributed consensus" to a counter and a transform loop.
4. Commit durably, then broadcast — never show a user an edit you cannot guarantee.
5. Snapshots and their version metadata commit atomically, or clients silently miss writes (Section 1.12).

1.20 Summary & Further Reading

We designed a real-time collaborative plain-text editor for 10M DAU: a stateless WebSocket gateway tier holding a million connections; a collaboration service that owns each document and imposes a total order on its operations; operational transformation to merge concurrent edits; an append-only operation log as the source of truth with periodic snapshots; version vectors for catch-up and offline diffing; and idempotent protocols that survive the network's misbehavior.

Primary source for this chapter:

- **"12: Design Google Docs/Real Time Text Editor"** — Systems Design Interview Questions With Ex-Google SWE (Jordan has no life) — the mock-interview walkthrough this chapter expands.

Foundations worth reading:

- Ellis & Gibbs, *Concurrency Control in Groupware Systems* (1989) — the original OT paper.
- Nichols et al., *Jupiter: high-latency collaboration* (1995) — OT with a central server, the direct ancestor of our design.
- Shapiro et al., *Conflict-free Replicated Data Types* (2011) — the CRDT formulation.
- Google Wave's OT whitepapers, and Figma's engineering blog on multiplayer — production war stories for both families.

1.21 Chapter 1 Glossary

A one-glance index of every term this chapter defined. Later chapters assume it.

Term	Meaning in one line
System design interview	Open-ended architecture design under a 45–60 minute clock
HLD / LLD	Whole-system architecture / internal mechanics of the hard components
Functional requirement	What the system must do
Non-functional requirement	How well it must do it: scale, latency, availability, consistency
DAU / QPS	Daily active users / queries per second
Back-of-the-envelope estimation	Approximate arithmetic that sizes the problem before designing

Term	Meaning in one line
Latency	Cause → observable effect; here: local echo < 16 ms, propagation < 150 ms
Availability	Fraction of time the system serves requests
Mutual exclusion (lock)	One writer at a time — correct, but kills real-time collaboration
Last-writer-wins	Latest timestamp overwrites — simple, but silently loses edits
Causality / concurrency	"Could A have influenced B?" If neither way: concurrent
Convergence	Same delivered edits ⇒ identical document on every replica
Intention preservation	Each author's observed edit effect survives concurrency
Operational Transformation	Adjust concurrent operations' positions so all replicas converge
CRDT	Data type whose merge rules converge without a central order
(Strong) eventual consistency	Replicas converge (immediately upon delivery, for SEC)
Logical clock / Lamport	Causality-tracking counters instead of wall-clock time
Version vector	Per-participant counters recording exactly what a replica has seen
REST	Stateless resource-oriented HTTPS API — our control plane
WebSocket	Persistent full-duplex client↔server channel — our data plane
Idempotency key	Client-supplied unique key making retried operations safe
Operation log	Immutable append-only record of all edits; the source of truth
Snapshot	Materialized document at a revision; read optimization over the log
Two-phase commit	Prepare-then-commit protocol for atomic writes across stores
Sharding	Partitioning work/data by key across machines — here, by document
Presence	Ephemeral who's-here-and-where state; never persisted
SLI / SLO	A measured indicator / its target, defining "healthy"

CHAPTER 2

Designing a Maps & Navigation Service

PROBLEM SOURCE

This chapter solves the problem posed in the talk **"13: Google Maps"** from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*, 2024, 35 min). The talk designs a mapping and navigation product: how the map itself is served, how the road network becomes a graph, how routes and ETAs are computed, and how live traffic feeds back into both. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol specifications, and Rust reference implementations.

2.1 The Problem Statement

The interviewer looks up and says:

> *"Design Google Maps. Users should be able to look at a map, search for places, and get directions from A to B — with an estimated time of arrival that accounts for current traffic."*

Where Chapter 1's problem hid its difficulty behind a familiar CRUD interface, this one hides its difficulty behind a familiar *picture*. A map feels like content — something you serve, like an image. Directions feel like a lookup — something you query, like a database. Neither is true. The map is a planet-sized rendering problem, directions are a graph algorithm running at planetary scale, and "current traffic" means the system's answers change **under you every few minutes**, driven by a firehose of GPS signals from millions of moving phones. Three genuinely hard subsystems — geospatial serving, large-scale graph search, and real-time stream processing — share one interview.

DEFINITION — ETA (estimated time of arrival)

The system's prediction of how long a journey from A to B will take, usually expressed as a duration or an arrival clock time. An ETA is a **prediction**, not a measurement: it must be computed before the journey happens, from a model of the road network plus everything currently known about conditions on it. ETA accuracy is the quality bar this whole chapter is judged against — Section 2.4 makes it a formal requirement and Section 2.18 shows how to measure it.

2.2 Scope & Clarifying Questions

Never design the prompt you were handed; design the prompt you **negotiated**. Google Maps is a dozen products wearing one icon, so the first job is to shrink it. A strong opening exchange looks like this:

Speaker	Dialogue
Candidate	"The full product is enormous — map rendering, place search, directions, turn-by-turn navigation, live traffic, Street View, satellite imagery, transit schedules, offline maps. Which parts are we designing?"
Interviewer	"Focus on five: render the map, search for places, compute directions with an ETA, show live traffic, and run a turn-by-turn navigation session. Skip Street View, satellite, transit, and offline."
Candidate	"What scale — how many users, and how many actively navigating at once?"
Interviewer	"One billion monthly users, a few hundred million daily. Assume up to 15 million people in an active navigation session at peak."
Candidate	"Latency expectations?"
Interviewer	"The map must feel instant while panning. A route request should come back within a second or two."
Candidate	"How accurate does the ETA need to be?"
Interviewer	"Within about ten percent of actual travel time on typical trips."
Candidate	"Do drivers' phones report GPS positions back to us during navigation? That would be our live traffic signal."
Interviewer	"Yes — anonymized location updates, with user consent, every few seconds while navigating."
Candidate	"Availability target? People use this while driving; a dead navigation app mid-highway is a safety problem."
Interviewer	"Four nines for the navigation path."

AGREED SCOPE

Five features: **map rendering**, **place search**, **directions with ETA**, **live traffic**, **turn-by-turn navigation**. Scale: **1B MAU**, 200M DAU, **15M concurrent navigation sessions** at peak. Latency: map feels instant, routes within 1–2 s. ETA accurate to ±10%. Navigation availability **99.99%**. Phones send anonymized GPS updates every few seconds while navigating.

INTERVIEW TIP — Ask where the data comes from

Most candidates ask about users and features; few ask **"what data do we get to see?"** That single question unlocks this entire problem: the answer (GPS updates from phones) is what makes live traffic possible at all. In a real interview, the questions that reveal **inputs** are worth as much as the questions that reveal **scale**.

2.3 Functional Requirements

Chapter 1 defined functional and non-functional requirements; recall that FRs are what the system must **do**. Our scoped functional requirements:

1. **FR-1 — Map rendering.** The user can view an interactive map of the world and pan and zoom it smoothly, from a whole-planet view down to individual streets.
2. **FR-2 — Place search.** The user can find places by name, address, or category ("coffee near me") and see them on the map.

3. **FR-3 — Directions.** The user can request a route between two points and receives a good route: distance, step-by-step instructions, and a path drawn on the map.
4. **FR-4 — ETA.** Every route comes with a travel-time estimate that reflects **current** conditions, not just speed limits.
5. **FR-5 — Turn-by-turn navigation.** The user can start a navigation session and receives timely spoken/visual instructions, a live ETA that updates en route, and an automatic new route if they stray from the planned path.
6. **FR-6 — Traffic overlay.** The map shows current congestion (green/yellow/red) on roads, fresh to within a few minutes.

Out of scope, stated explicitly: Street View and satellite imagery, public transit timetables, offline maps, business-owner tooling, and social features.

2.4 Non-Functional Requirements

> **DEFINITION — Freshness**
>
> The age of the information behind an answer, measured from when reality changed to when the system's output reflects it. A traffic overlay that shows speeds from an hour ago is **stale**; our target is that any road's displayed condition reflects measurements from the last few minutes. Freshness is the NFR that makes this problem a **streaming** system rather than a static one.

Four qualities dominate, stated as targets:

Quality	Target
Map latency	Tiles visible within 100 ms while panning (p95); panning must never stutter
Route latency	Directions within 1 s for typical trips; 2 s for cross-country
ETA accuracy	Within ±10% of realized travel time on typical trips
Traffic freshness	Displayed speeds reflect the last 2–5 minutes of reality
Availability	99.99% on the navigation path (4.4 s of allowed downtime per day); 99.9% elsewhere
Scale	1B MAU; 200M DAU; 15M concurrent navigation sessions at peak

> **KEY INSIGHT — Different subsystems, different NFRs**
>
> Notice that "the system" has no single latency or availability number. Tile serving is a massive, cacheable **read** workload where 100 ms matters and a stale tile is harmless. Routing is a **compute** workload where one second is generous but the algorithm must be exact. The traffic pipeline is a **write** workload where a few minutes of lag degrade quality, not correctness. Saying "it depends which plane you mean" — and then naming the planes — is a senior answer. Chapter 1 drew the same lesson with its control plane / data plane split.

2.5 Back-of-the-Envelope Estimation

Chapter 1 defined back-of-the-envelope estimation; the discipline is identical — state assumptions, write them down, invite correction.

Assumptions:

- 1B MAU, 200M DAU. Each daily user views 60 map tiles worth of panning, searches 2 places, and requests 3 routes per day.

- 15M concurrent navigation sessions at peak; each phone uploads one GPS update every 5 seconds while navigating; one update ≈ 150 bytes on the wire.
- The world's routable road network is on the order of **300 million directed edges** (two directions per road piece, worldwide).
- A raster map tile averages 20 KB; the world is mapped at zoom levels 0–20.

Derived numbers:

Quantity	Estimate	How
Map tile requests	≈ 140k QPS avg, 700k peak	200M users × 60 tiles/day ÷ 86,400 s, ×5 peak factor
Place search QPS	≈ 5k avg, 25k peak	200M × 2/day ÷ 86,400, ×5
Route request QPS	≈ 7k avg, 20k peak	200M × 3/day ÷ 86,400, ×3
GPS update ingress	≈ 3M updates/s	15M navigators ÷ 5 s per update
Ingress bandwidth	≈ 450 MB/s	3M updates/s × 150 B
Road graph size	≈ 20 GB	300M edges × 64 B per adjacency record
Naive tile storage	≈ 29 PB	$\sum_{z=0}^{20} 4^z \approx 1.47$ trillion tiles × 20 KB

KEY INSIGHT — What the math tells us

Three conclusions drive everything that follows. First, tile traffic is enormous but **the content barely changes** — a perfect caching workload; Section 2.6 exists to make 95%+ of those 700k QPS vanish into a CDN. Second, the road graph (20 GB) fits in the RAM of a handful of machines, so routing is an **in-memory** problem — but 20k route QPS × 3 CPU-seconds for a naive shortest-path search would need **60,000 CPU cores**, which no fleet absorbs; Section 2.8's whole purpose is to shave those 3 seconds down to 1 millisecond, at which point 20 cores suffice. Third, 3M GPS updates per second is far too much for any request/response design — it demands a streaming pipeline, which Section 2.12 builds.

2.6 Core Challenge I: Serving the Map

A user opens the app and sees their city. They drag the map; new streets slide in. They pinch; the view dives from the whole country to one neighborhood. Each of those moments requires the **right piece of the planet, rendered, in tens of milliseconds**. Rendering the world on the fly for every gesture of 200 million daily users is impossible; the entire solution rests on one idea: **precompute the map as tiny, independently addressable squares, and cache them everywhere.**

DEFINITION — Map tile

A small, fixed-size square of the map — conventionally 256×256 pixels (as an image) or the equivalent bundle of geometry (as data) — covering a specific rectangular patch of the Earth's surface. A screen full of map is assembled from a grid of tiles, like mosaic pieces. Tiles are the unit of storage, caching, and transfer for every mainstream map service.

DEFINITION — Zoom level

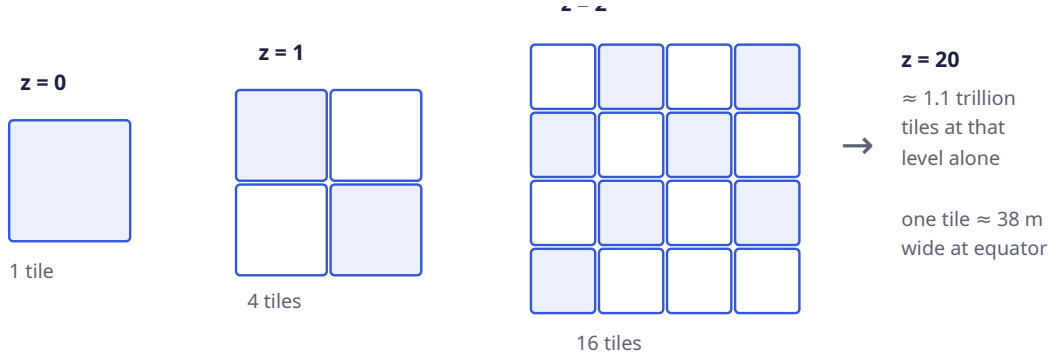
The map's scale, expressed as an integer z from 0 upward. At zoom 0 the **entire world** is one tile. Each step down splits every tile into four children, so zoom z has 2^z tiles per axis and 4^z tiles in total. Zoom 5

is roughly a country; zoom 10 a city; zoom 15 a neighborhood; zoom 20 shows a patch about 38 meters wide at the equator — individual buildings.

DEFINITION — Slippy-map tiling scheme

The near-universal convention for addressing tiles: every tile is identified by three integers **(z, x, y)** — its zoom level and its grid coordinates, counted from the top-left of the world. The Earth's surface is first flattened with the Web Mercator projection (which maps the sphere to a square and caps latitude near ±85.05°), then cut into the $2^z \times 2^z$ grid. Given a latitude/longitude and a zoom, **anyone can compute which tile contains it** with a closed-form formula — Section 2.14 implements it. The addressing is deterministic, so the client never asks the server "which tiles do I need?"; it already knows.

The pyramid shape is the whole storage story. Four zooms, drawn:



Our estimate said storing **all** of that naively costs 29 PB. Two observations make it tractable:

1. **Most tiles are uniform.** Roughly 70% of the planet is ocean; enormous areas are desert, ice, or forest. A solid-blue 256×256 square compresses to almost nothing — and, crucially, **it is the same square everywhere**. Storing tiles content-addressed (Section 2.10) means every identical tile in the world is stored **once**; the trillions collapse to the few percent of tiles that actually contain roads and labels.
2. **Nobody looks at most of the world, most of the time.** Demand is violently skewed toward populated areas and common zooms. That skew is what caching feeds on.

DEFINITION — Content Delivery Network (CDN)

A geographically distributed fleet of **edge caches**: servers in hundreds of cities, each holding copies of popular content close to users. A request is routed to the nearest edge location; on a **cache hit** the content is served locally (single-digit milliseconds, zero load on our infrastructure); on a **miss** the edge fetches from our origin once and caches it for the next requester. CDNs turn a read-heavy, mostly-static workload into someone else's bandwidth.

DEFINITION — TTL (time to live) / cache hit ratio

A cached entry's *TTL* is how long an edge may serve it before revalidating with the origin — the dial between freshness and load. The *hit ratio* is the fraction of requests served from cache. For base map tiles we choose long TTLs (days; roads move slowly) and expect hit ratios above 95%; the **traffic overlay** tiles of Section 2.12 get TTLs of a minute or two, because stale traffic is worthless.

COMMON PITFALL — Serving tiles from your own fleet

> The single most common junior mistake on this problem: drawing a "tile service" that renders or reads tiles on demand per request. At 700k peak QPS that fleet is vast, slow for distant users, and pointless — the content is static and cacheable. The correct shape is: **pre-render tiles offline, push them to object storage, put a CDN in front, and let the origin see a single-digit percentage of traffic.** Rendering is a batch job, not a request path.

One design fork is worth naming here and deciding later (Section 2.17): tiles can be **raster** (pre-rendered images) or **vector** (compact geometry the client renders on its GPU). We will serve vector tiles to modern clients — roughly half the bytes, restyleable without re-rendering, and one tile set serves every zoom-adjacent gesture smoothly — while keeping raster fallbacks for old clients. Either way, the tiling, addressing, and CDN story is identical.

Place search (FR-2) rides on the same geospatial ideas: a **spatial index** lets us find "all places inside this patch of Earth" without scanning 200M records. Section 2.7 defines the indexing structure once, because routing needs it too.

2.7 Core Challenge II: Modeling the Road Network

Directions require a mathematical object we can search. That object is a graph, built from the road network:

> **DEFINITION — Graph / weighted directed graph**
>
> A *graph* is a set of **nodes** connected by **edges**. It is *directed* when edges have a direction (a one-way street is traversable only along its arrow) and *weighted* when every edge carries a number measuring the **cost** of crossing it. We model intersections as nodes and stretches of road between them as edges, and we call one such directed road piece a **segment**.

> **DEFINITION — Segment**
>
> The atomic unit of the road network: one directed piece of road between two nodes, carrying metadata — geometry (the polyline of its shape), length, road class (residential, arterial, highway), speed limit, and turn restrictions at its far end. Segments are also the unit of **traffic**: when we say "this road is slow," we mean "this segment's current crossing time is high," and every GPS update we receive is attributed to exactly one segment (Section 2.12).

The crucial choice is the edge **weight**. Distance is stable but wrong for drivers; what users minimize is **time**. So an edge's weight is its **expected crossing time**: `length / current expected speed`. "Current expected speed" is where live traffic enters — the weight is `length / speed_limit` on an empty road, and degrades as Section 2.12's pipeline reports slower observed speeds. Routing and traffic are thereby one mechanism: **traffic is just edge weights that move.**

> **DEFINITION — Adjacency list**
>
> The standard memory layout for sparse graphs: for every node, a list of its outgoing edges. Routing graphs are extremely sparse (an intersection has 3–6 outgoing segments), so an adjacency list stores 300M small records — the 20 GB from Section 2.5 — instead of the quadratically-sized matrix a dense representation would need. Our routing engine holds adjacency lists **in RAM**; there is no database on the per-request path.

A subtlety interviewers enjoy: a graph edge cannot express "you may enter this intersection from Main St but not turn left onto 1st Ave." Turn restrictions are handled either by splitting nodes (one

node per incoming direction, with only legal turns as edges) or by per-edge metadata the search consults. Either way, the model — nodes, directed edges, time weights — survives intact.

2.8 Core Challenge III: Shortest Paths at Planetary Scale

DEFINITION — Shortest path problem

Given a weighted graph and two nodes, find the path between them whose total edge weight is minimal. With time-weighted segments, the shortest path is the **fastest route**, and its total weight is the route's ETA. This is the single computation at the heart of FR-3 and FR-4.

The canonical algorithm, and the one the interviewer expects you to derive:

DEFINITION — Dijkstra's algorithm

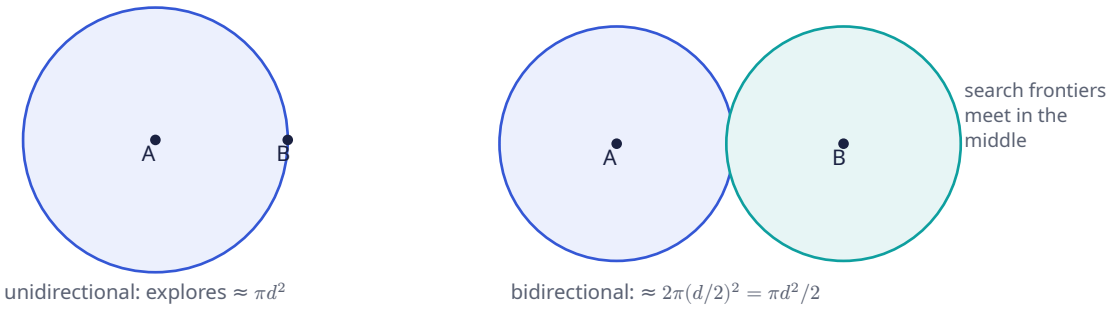
Explores the graph outward from the source in order of increasing distance from it. Maintain for each node the best known distance (initially ∞, 0 for the source); repeatedly take the not-yet-finalized node with the smallest best-known distance, declare it final, and **relax** its edges — for each outgoing edge, check whether reaching its neighbor through this node beats the neighbor's best-known distance, and if so update it. With a min-priority queue selecting the next node, the running time is $O((V + E) \log V)$. It is provably exact for non-negative weights — and crossing times are always non-negative.

DEFINITION — Priority queue (min-heap)

A data structure supporting "insert with a priority" and "remove the minimum-priority element," both in $O(\log n)$. A binary heap inside an array is the standard implementation. Dijkstra's algorithm spends its life asking "which unfinished node is currently closest?" — exactly this operation.

Dijkstra is correct — and unusable here. A cross-country query on 100M nodes takes seconds of CPU; Section 2.5 showed that times 20k QPS is 60,000 cores. Three refinements, each a layer of the same idea — **search less of the graph**:

Refinement 1 — bidirectional Dijkstra. Run two searches simultaneously: one forward from the origin, one **backward** from the destination over reversed edges. Stop when the frontiers meet; the route is the two half-paths joined. Why it helps is geometric:



A forward search from A must explore roughly every node within distance d of A — a ball whose node count grows like the **area** πd^2. Two half-searches each explore a ball of radius $d/2$; together that is $2 \cdot \pi(d/2)^2 = \pi d^2/2$ — about **half** the work in a uniform graph, and better in road networks, where the frontiers approach each other along highways. Same exact answer, half the CPU, and trivially parallelizable across two threads.

Refinement 2 — A* search. Dijkstra explores equally in **all** directions — including straight away from the destination, which is obviously wasted. A* steers the search by reordering the priority queue.

DEFINITION — Heuristic / admissible heuristic

In A*, each node's queue priority is `known distance from source + h(node)`, where the *heuristic* `h` estimates the remaining distance to the destination. The heuristic is *admissible* if it never **overestimates** the true remaining cost. Admissibility preserves Dijkstra's exactness guarantee while pulling the search frontier toward the goal — nodes in the right direction get popped first.

For a road network, the admissible heuristic is the **straight-line distance to the destination divided by the fastest speed anywhere in the network**: no legal route can beat the crow flying at top speed, so this `h` never overestimates. Computing it needs distances between points on a sphere:

DEFINITION — Haversine distance

The great-circle distance between two latitude/longitude points on a sphere — the length of the shortest path over the Earth's surface ("as the crow flies"). A closed-form trigonometric formula computes it in microseconds; Section 2.14 implements it. Haversine is our heuristic's yardstick and our fallback whenever "how far apart are these coordinates?" appears.

Refinement 3 — hierarchy: mega-segments, then contraction hierarchies. Even A* explores every side street near the origin and destination. But observe how **you** drive cross-country: local streets for a minute, then a highway for three hundred miles, then local streets again. The middle of a long route almost never touches small roads. Turn that observation into a mechanism: precompute and **aggregate**.

DEFINITION — Mega-segment

A precomputed synthetic edge that summarizes a chain of ordinary segments — for example, one edge meaning "highway, exit 12 to exit 47, 52 km, 31 minutes at current speeds." A long-distance search can then hop between on-ramps and off-ramps on mega-segments instead of expanding thousands of small segments. The source talk builds its long-range routing this way: segments roll up into mega-segments, which roll up further, so a query quickly climbs from street level to highway level, crosses the country on a handful of edges, and descends again. Mega-segment weights are sums of member segment weights — so when traffic updates a segment (Section 2.12), the change **bubbles up** to every mega-segment containing it.

The literature's formal, optimal version of the same idea:

DEFINITION — Contraction hierarchy (CH)

A preprocessed form of the road graph. Offline, order all nodes by importance (highway junctions outrank cul-de-sacs) and **contract** them in that order: removing a low-importance node, and — whenever the shortest path between two of its neighbors ran through it — inserting a **shortcut edge** with the summed weight, so all shortest-path distances are preserved. At query time, run bidirectional Dijkstra with one rule: only follow edges that go **up** in importance. The two upward searches meet at the most important node on the route. Preprocessing takes hours and adds 30–100% more edges; queries drop from seconds to **microseconds-to-milliseconds** — the roughly 1000× speedup Section 2.5's arithmetic demands. Production engines (OSRM, Valhalla, and the systems the source talk gestures at with mega-segments) all run variants of this playbook.

The full ladder, with the numbers that justify it:

Algorithm	Preprocessing	Query time	Verdict
Dijkstra	none	seconds (continental)	Teaches the principle; too slow to serve
Bidirectional Dijkstra	none	2× faster, still seconds	Free win; still too slow
A* + haversine	none	often 5–10× faster	Great for short trips; long trips still expand too much
Mega-segments / CH	hours, offline	≈ 0.1–1 ms	**Production answer**: the 1000× that makes 20k QPS fit on 20 cores

KEY INSIGHT — Precomputation is the theme of the whole chapter

Tiles are pre-rendered so reads are cache hits (Section 2.6); the graph is pre-contracted so queries touch only highways-in-the-middle (this section); traffic speeds are pre-aggregated per segment so ETAs are table lookups (Section 2.12). A maps system is a machine for moving work **off** the request path and into batch and stream pipelines. When an interviewer asks "how does it answer so fast?", the honest one-word answer is: **earlier**.

2.9 API & Protocol Design

Chapter 1 split its API into a control plane and a data plane; the same split applies, with a twist: our heaviest "API" is not an API at all.

Tiles are plain GETs — and mostly not ours. The client computes the (z, x, y) address of every tile in view (Section 2.6) and issues ordinary `GET /v1/tiles/{z}/{x}/{y}` requests against a tile hostname. Nearly all of these terminate at a CDN edge and never reach us. This is why the tile endpoint has no interesting request parameters: all the intelligence is in the **addressing scheme**.

DEFINITION — Polyline encoding

A compact ASCII encoding of a sequence of coordinates — Google's variant encodes latitude/longitude deltas as variable-length base-64-ish text, so a 500-point route is a few hundred bytes instead of a JSON array of floats. Routes are transmitted as encoded polylines and decoded client-side; the format exists because a route crosses a route-length's worth of geography but must fit in one small response.

The remaining request/response endpoints (REST, as defined in Chapter 1):

Method	Path	Purpose
`GET`	`/v1/tiles/{z}/{x}/{y}`	Map tiles; served by CDN edge, origin on miss
`GET`	`/v1/search?q=&near=lat,lng`	Place search: text + spatial filter, ranked results
`GET`	`/v1/search/autocomplete?q=`	Prefix suggestions as the user types
`POST`	`/v1/routes`	Directions: origin, destination, mode, departure time → candidate routes with distance, ETA, ETA-in-traffic, steps, encoded polyline
`POST`	`/v1/geocode`	Address → coordinates (and reverse: coordinates → address)

A route request and its response:

```
POST /v1/routes
{
  "origin":      { "lat": 37.7749, "lng": -122.4194 },
  "destination": { "lat": 37.3861, "lng": -122.0839 },
  "mode": "driving",
  "depart_at": "now"
}

200 OK
{
  "routes": [{
    "route_id": "rt_9c31",
    "distance_m": 56300,
    "eta_seconds": 2820,
    "eta_in_traffic_seconds": 3450,
    "polyline": "a~l~Fjk~uOwIb@_...",
    "steps": [
      { "instruction": "Head north on Market St",
        "segment_ids": ["seg_7712", "seg_7713"],
        "distance_m": 400, "eta_seconds": 61 }
    ]
  }]
}
```

Navigation (FR-5) is a session, not a request: the phone streams its position up and the server streams guidance down. That is a bidirectional, long-lived, low-latency channel — the WebSocket data plane from Chapter 1, reused verbatim.

Message	Direction	Payload & semantics
`nav_start`	client → server	`{route_id, auth}` — begin the session on the chosen route
`location_update`	client → server	`{lat, lng, speed_mps, heading_deg, t}` — one GPS fix, every 5 s; drives guidance **and** the traffic pipeline
`instruction`	server → client	`{text, voice, distance_to_maneuver_m}` — the next turn, delivered early enough to speak
`eta_update`	server → client	`{eta_seconds, remaining_m}` — refreshed as segment weights move
`reroute`	server → client	`{reason, new_route}` — deviation or a newly faster alternative
`nav_end`	either	Arrived, cancelled, or the connection dropped

Two protocol notes. First, `location_update` is **one stream feeding two masters**: live guidance for this user, and (anonymized, aggregated) the traffic pipeline for everyone else — Section 2.12 draws that fork. Second, navigation is the one place we push **down** to a moving phone, which raises the question "which gateway holds this user's connection?" — answered by the connection manager in Section 2.13.

2.10 Data Model & Storage

DEFINITION — Polyglot persistence

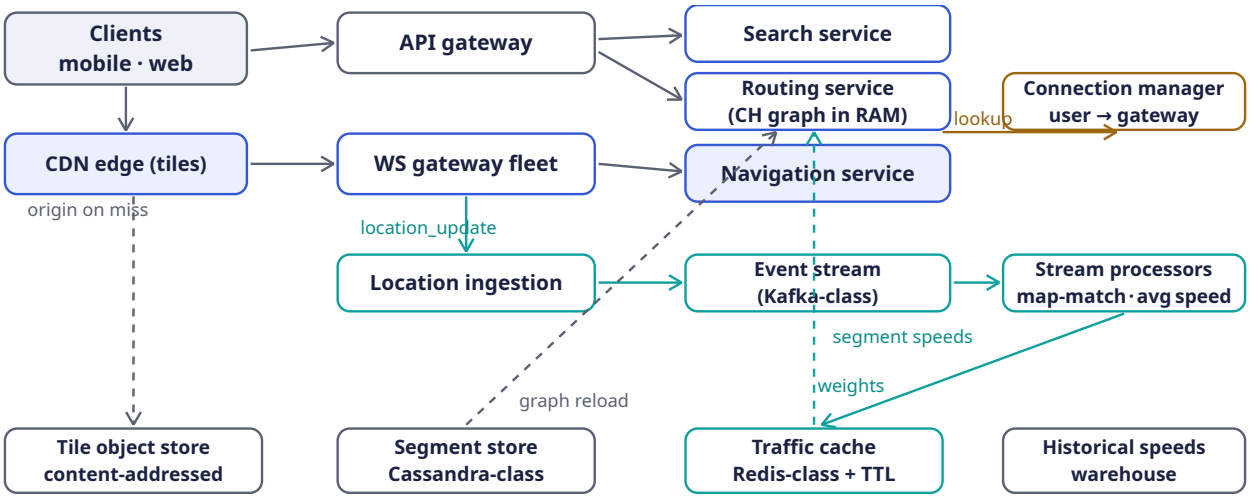
Using different storage engines for different access patterns within one system, instead of forcing every entity into one database. The cost is operational complexity; the benefit is that each workload gets the structure it actually needs. A maps system is the canonical case — our six entities below want six different shapes.

Entity	Contents	Store
`Tile`	`(z,x,y)` → rendered image or vector geometry	Object store behind the CDN; **content-addressed** so identical tiles are stored once
`Place`	name, category, address, location, ratings, hours	Text-inverted index × spatial index; 200M records
`Segment`	geometry, length, road class, speed limit, turn restrictions	Wide-column store (Cassandra-class), sharded by geography; read-heavy, batch-written
`Road graph`	adjacency lists + current weights + shortcuts	**In RAM** on routing nodes; rebuilt/swapped from the segment store + traffic cache
`Traffic`	`segment_id` → current average speed, sample count, updated-at	In-memory key-value cache (Redis-class) with a TTL — absence means "no fresh data," triggering the historical fallback
`Historical speeds`	`segment_id × (day-of-week, time bucket)` → average speed	Analytics warehouse, batch-computed nightly from the GPS archive
`Nav session`	route, progress along it, last fix, ETA	In-memory on the navigation node; ephemeral like Chapter 1's presence data

The ideas interviewers reward here: tiles are **content-addressed** (the 29 PB problem of Section 2.5 collapses because duplicate oceans are stored once); the routing graph lives **in memory**, with stores acting only as its source of truth during reloads; and the traffic cache's TTL **is** the freshness guarantee — a segment whose entry has expired simply stops claiming live data.

2.11 High-Level Architecture

Section 2.4 promised separate planes; here they are. The **read plane** (tiles, search, routes) is stateless and cache-fronted. The **streaming plane** (GPS in, traffic out) is a pipeline. Navigation sits astride both.



Read plane: stateless, cache-fronted. Streaming plane: GPS in → traffic out. Dashed: control/reload paths.

Component responsibilities, and the reason each exists:

Component	Responsibility	Why it is shaped this way
CDN edge	Serve 95%+ of tile requests	Tiles are static and skewed — the textbook cache workload (Section 2.6)
API gateway	Auth, rate limiting, routing for REST endpoints	Stateless; also the paid-API front door, where per-customer quotas are enforced
Search service	Text × spatial place lookup	Read-heavy over a slowly-changing index; replicas scale it
Routing service	Compute routes + ETAs on the CH graph	Holds the graph **in RAM**; scales by region shards and replicas (Section 2.15)
WS gateway fleet	Hold millions of navigation connections	Stateless connection holders, exactly as in Chapter 1
Navigation service	Per-session guidance: progress, instructions, ETA refresh, reroutes	Stateful per session but sessions are small and independent
Connection manager	Registry: user → which gateway	So the navigation service can push `reroute` to the right socket (Section 2.13)
Location ingestion	Absorb 3M updates/s, validate, anonymize	A buffer that decouples phone bursts from processing (Section 2.12)
Event stream	Durable, replayable pipe of GPS updates	Chapter 2 defines it below; lets many consumers read the same firehose

Component	Responsibility	Why it is shaped this way
Stream processors	Map-match pings to segments; average speeds; write traffic cache; bubble weights up mega-segments	The batch-quality work of traffic, done continuously

2.12 Deep Dive: The Live Traffic Loop

This is the subsystem that turns "a map" into "**current** conditions," and it is the heart of the source talk. Follow one GPS update until it changes someone's ETA.

DEFINITION — Event streaming platform (Kafka-class)

A distributed, durable, append-only log organized into **topics**; producers append events, and any number of independent **consumers** read them at their own pace. Topics are **partitioned** (we partition by geographic key, e.g. geohash prefix) so hundreds of consumer instances process the stream in parallel, each owning a disjoint slice of the world. It absorbs our 3M updates/second, decouples ingestion from processing, and — because it is replayable — lets us rebuild derived state after failures.

DEFINITION — Stream processing

Computing over data **as it arrives**, event by event or in small windows, rather than in scheduled batch runs over stored data. Here: every GPS update is map-matched and folded into a per-segment running average within seconds of leaving the phone.

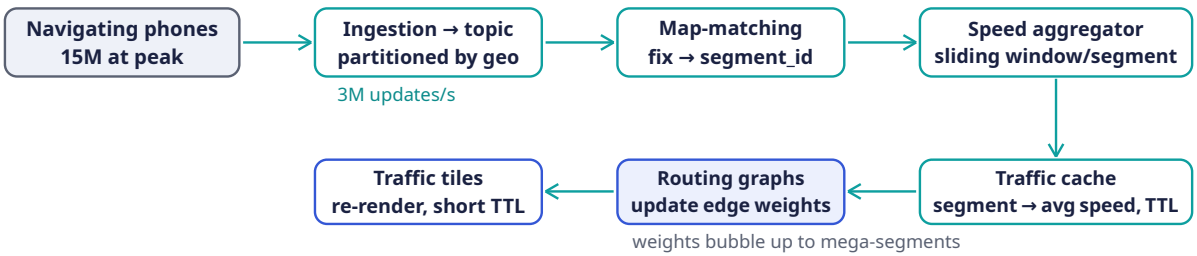
DEFINITION — Map-matching

Snapping a noisy GPS fix to the road segment the vehicle is most likely actually on. Raw GPS is wrong by 5–50 meters — enough to place a car on the wrong street or inside a building. The production-standard approach models the trace as a **hidden Markov model** (hidden states = candidate segments near each fix; emission scores = closeness of fix to segment; transition scores = plausibility of the implied movement) and solves it with the Viterbi algorithm. Map-matching is why the traffic pipeline's first stage is a **processor**, not a database write.

DEFINITION — Sliding window

A moving time interval over a stream — "the last 15 minutes," recomputed as time advances. A per-segment sliding-window average of observed speeds is our definition of "current speed": old enough data falls out of the window and stops influencing the present. Section 2.14 implements the window.

The loop, end to end:



Every few minutes, reality → graph. ETAs and reroutes on new routes then use fresh weights automatically.

Three design details carry the interview:

1. **Speed, not position, is the aggregate.** Per segment and window, we average the speeds of map-matched vehicles. One slow vehicle is noise (a delivery van at the curb); the **average** over dozens of vehicles in 15 minutes is signal. This is also why privacy and accuracy align: nobody's individual trace is stored in the traffic cache — only segment statistics.
2. **Sparse segments fall back to history.** At 3 a.m. on a rural road, the window holds too few samples. Below a minimum count we blend toward the **historical** speed for that segment, day-of-week, and time-of-day — Tuesday-8 a.m. traffic is remarkably similar to last Tuesday-8 a.m. Live data where it exists, patterns where it does not, speed limits as the floor of last resort.
3. **Third-party feeds are scored, not trusted.** We also ingest incident and flow feeds from external providers. The talk's discipline: validate each feed against our own observed reality — if a provider claims congestion on a corridor where our measured speeds never dropped, that claim is wrong, and a provider that is wrong often gets **down-weighted or ignored** at the organization level. Every external signal is guilty until statistically innocent.

The same machinery paints the traffic overlay (FR-6): segment speeds become green/yellow/red classes, rendered into a **separate tile layer** with a TTL of a minute or two — short-lived tiles on top of the long-lived base map, each layer cached at its own freshness.

2.13 Deep Dive: The Turn-by-Turn Navigation Session

A navigation session is a small state machine per user, held in the navigation service: `ON_ROUTE → OFF_ROUTE_SUSPECTED → REROUTING → ON_ROUTE → ARRIVED`. Each `location_update` advances it:

1. **Project.** Map-match the fix onto the planned route's polyline: how far along the route is the user, and how far **off** it?
2. **Guide.** From progress along the route, emit the next `instruction` early enough to be spoken ("in 300 meters, turn right").
3. **Refresh.** Recompute the ETA over the remaining segments using **current** weights from the traffic cache; push `eta_update` when it moves by more than a small threshold (users notice a jumping ETA; hysteresis is a feature).
4. **Reroute.** If the fix sits beyond a distance threshold from the polyline **and** heading disagrees with the route, **sustained** for a few consecutive updates (one bad fix must not trigger a reroute — GPS noise is constant), recompute from the current position and push `reroute` with the new route. Section 2.14 implements this decision.

The push path needs one piece of plumbing: the navigation service knows **what** to send but not **where the user's socket lives**. The **connection manager** — a replicated key-value registry mapping `user_id → gateway_id` — answers that: gateways register each connection on connect and remove it on drop; the navigation service looks the user up and hands the message to that gateway. If the registry entry is stale (a gateway died), the client's reconnect registers a fresh one within seconds, and at worst one push is retried. This is Chapter 1's stateless-gateway story, completed with its directory.

> **COMMON PITFALL — Rerouting on a single GPS fix**
>
> GPS in a downtown canyon can teleport a user two blocks sideways for one update. A reroute fired on that ghost fix is a terrible user experience — and it also **pollutes the traffic pipeline** with a phantom slow-down on a road the user never touched. Both loops therefore consume **map-matched, sustained** signals, never raw single fixes. The same discipline, twice.

> **INTERVIEW TIP — Navigation availability is a degradation story, not an uptime story**

> Four nines on the navigation path does not mean "the server never dies"; it means **the phone copes when it does**. The client caches its route, its upcoming tiles, and its step list; if the session drops, guidance continues locally from the last known state while a new session is established. Saying "the client is the final fallback layer of the server" is exactly the kind of answer the 99.99% requirement is fishing for.

2.14 Deep Dive: Rust Reference Implementations

Four pieces of this chapter are small enough — and interview-critical enough — to show in full: tile addressing, the routing core, the traffic window, and the reroute decision.

2.14.1 Tile addressing: lat/lng → tile → quadkey

The slippy-map formulas of Section 2.6, plus the quadkey encoding that makes tiles prefix-searchable:

```rust
/// Web-Mercator slippy-map addressing (Section 2.6).

/// Which tile (x, y) at `zoom` contains this lat/lng?
pub fn lat_lng_to_tile(lat: f64, lng: f64, zoom: u8) -> (u32, u32) {
    let n = 2f64.powi(zoom as i32);              // tiles per axis: 2^z
    let x = ((lng + 180.0) / 360.0 * n) as u32;
    let lat = lat.to_radians();
    let y = ((1.0 - lat.tan().asinh() / std::f64::consts::PI) / 2.0 * n) as u32;
    let max = n as u32 - 1;
    (x.min(max), y.min(max))                     // clamp the antimeridian edge
}

/// The same address as a quadkey: one digit per zoom level, '0'..='3',
/// interleaving y/x bits from the most significant down. Neighboring tiles
/// share prefixes — a quadkey IS a quadtree path — so a plain sorted
/// key-value store can answer "all tiles in this area" as a range scan.
pub fn quadkey(x: u32, y: u32, zoom: u8) -> String {
    let mut key = String::with_capacity(zoom as usize);
    for level in (0..zoom).rev() {
        let mask = 1u32 << level;
        let mut digit = 0u8;
        if x & mask != 0 { digit += 1; }
        if y & mask != 0 { digit += 2; }
        key.push((b'0' + digit) as char);
    }
    key
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn whole_world_is_tile_zero() {
        assert_eq!(lat_lng_to_tile(0.0, 0.0, 0), (0, 0));
    }

    #[test]
    fn san_francisco_zoom_16() {
        // Verified against the slippy-map formulas.
        assert_eq!(lat_lng_to_tile(37.7749, -122.4194, 16), (10482, 25331));
        assert_eq!(quadkey(10482, 25331, 16), "0230102033330032");
    }
```

```
    #[test]
    fn bing_canonical_quadkey() {
        // The documentation example: tile (3, 5) at level 3 -> "213".
        assert_eq!(quadkey(3, 5, 3), "213");
    }
}
```

The quadkey is the bridge to place search: index every place under the quadkey of its location, and "places near me" becomes "places whose key shares my tile's prefix," widened one ring of neighbors at a time.

2.14.2 The routing core: haversine, Dijkstra, A*

Section 2.8's algorithm ladder, executable. Weights are integer **milliseconds** of expected crossing time — exact, and exactly what the traffic loop updates.

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

pub type NodeId = u32;

/// One directed segment leaving a node (Section 2.7).
#[derive(Clone, Copy, Debug)]
pub struct Edge {
    pub to: NodeId,
    pub millis: u32, // expected crossing time — the weight traffic moves
}

/// In-memory adjacency-list road graph (Section 2.7).
pub struct RoadGraph {
    pub adj: Vec<Vec<Edge>>,
    pub lat: Vec<f64>, // node coordinates, for haversine and A*
    pub lng: Vec<f64>,
}

/// Great-circle distance in meters (Section 2.8).
pub fn haversine_m(a: (f64, f64), b: (f64, f64)) -> f64 {
    const R: f64 = 6_371_000.0; // Earth radius, meters
    let (la1, la2) = (a.0.to_radians(), b.0.to_radians());
    let dla = (b.0 - a.0).to_radians();
    let dlo = (b.1 - a.1).to_radians();
    let h = (dla / 2.0).sin().powi(2) + la1.cos() * la2.cos() * (dlo / 2.0).sin().powi(2);
    2.0 * R * h.sqrt().asin()
}

/// Dijkstra: exact shortest path by total crossing time.
pub fn dijkstra(g: &RoadGraph, source: NodeId, target: NodeId) -> Option<(u64, Vec<NodeId>)>
{
    let mut dist = vec![u64::MAX; g.adj.len()];
    let mut prev = vec![u32::MAX; g.adj.len()];
    let mut heap = BinaryHeap::new();
    dist[source as usize] = 0;
    heap.push(Reverse((0u64, source)));
    while let Some(Reverse((d, u))) = heap.pop() {
        if u == target { break; }             // popped => finalized => optimal
        if d > dist[u as usize] { continue; } // stale heap entry
        for e in &g.adj[u as usize] {
            let nd = d + e.millis as u64;     // relax the edge
            if nd < dist[e.to as usize] {
                dist[e.to as usize] = nd;
```

```
                prev[e.to as usize] = u;
                heap.push(Reverse((nd, e.to)));
            }
        }
    }
    if dist[target as usize] == u64::MAX { return None; }
    let mut path = vec![target];              // walk predecessors back
    while *path.last().unwrap() != source {
        path.push(prev[*path.last().unwrap() as usize]);
    }
    path.reverse();
    Some((dist[target as usize], path))
}

/// A*: the same search, re-prioritized by an admissible heuristic —
/// remaining straight-line distance at the fastest plausible speed, so it
/// never overestimates (and, on a road graph, is *consistent*, which is what
/// licenses the early exit when the target is popped).
pub fn astar(g: &RoadGraph, source: NodeId, target: NodeId) -> Option<(u64, Vec<NodeId>)> {
    const MAX_SPEED_MPS: f64 = 70.0; // ~250 km/h: faster than any legal road
    let t = target as usize;
    let h = |n: NodeId| -> u64 {
        let i = n as usize;
        (haversine_m((g.lat[i], g.lng[i]), (g.lat[t], g.lng[t])) / MAX_SPEED_MPS * 1000.0)
as u64
    };
    let mut dist = vec![u64::MAX; g.adj.len()];
    let mut prev = vec![u32::MAX; g.adj.len()];
    let mut heap = BinaryHeap::new();
    dist[source as usize] = 0;
    heap.push(Reverse((h(source), 0u64, source))); // (priority, dist, node)
    while let Some(Reverse((_, d, u))) = heap.pop() {
        if u == target { break; }
        if d > dist[u as usize] { continue; }
        for e in &g.adj[u as usize] {
            let nd = d + e.millis as u64;
            if nd < dist[e.to as usize] {
                dist[e.to as usize] = nd;
                prev[e.to as usize] = u;
                heap.push(Reverse((nd + h(e.to), nd, e.to)));
            }
        }
    }
    if dist[t] == u64::MAX { return None; }
    let mut path = vec![target];
    while *path.last().unwrap() != source {
        path.push(prev[*path.last().unwrap() as usize]);
    }
    path.reverse();
    Some((dist[t], path))
}
```

And the test that demonstrates the chapter's slogan — **traffic is just edge weights that move**:

```
#[cfg(test)]
mod tests {
    use super::*;

    /// A tiny city: two ways from node 0 to node 4, edges ~1.1–1.6 km.
    ///   0 -> 1 -> 2 -> 4 :  60 s +  60 s +  60 s = 180 s
```

```
    ///   0 -> 3 -> 4      :  30 s + 120 s        = 150 s   (normally fastest)
    /// (Times are consistent with the A* speed cap of 70 m/s, so the
    /// haversine heuristic stays admissible on this graph.)
    fn tiny_city() -> RoadGraph {
        let mut adj = vec![Vec::new(); 5];
        let e = |adj: &mut Vec<Vec<Edge>>, from: u32, to: u32, millis: u32| {
            adj[from as usize].push(Edge { to, millis });
            adj[to as usize].push(Edge { to: from, millis }); // two-way streets
        };
        e(&mut adj, 0, 1, 60_000);
        e(&mut adj, 1, 2, 60_000);
        e(&mut adj, 2, 4, 60_000);
        e(&mut adj, 0, 3, 30_000);
        e(&mut adj, 3, 4, 120_000);
        RoadGraph {
            adj,
            lat: vec![0.0, 0.0, 0.01, -0.01, 0.0],   // rough grid around origin
            lng: vec![0.0, 0.01, 0.01, 0.01, 0.02],
        }
    }

    #[test]
    fn dijkstra_and_astar_agree() {
        let g = tiny_city();
        assert_eq!(dijkstra(&g, 0, 4).unwrap().0, 150_000);
        let (millis, path) = astar(&g, 0, 4).unwrap();
        assert_eq!(millis, 150_000);
        assert_eq!(path, vec![0, 3, 4]);
    }

    #[test]
    fn traffic_reroutes() {
        let mut g = tiny_city();
        // Congestion report: the 3 -> 4 segment now takes 600 s.
        g.adj[3].iter_mut().find(|e| e.to == 4).unwrap().millis = 600_000;
        g.adj[4].iter_mut().find(|e| e.to == 3).unwrap().millis = 600_000;
        // No code path changes — the "traffic-aware" router is the same
        // algorithm reading different weights.
        assert_eq!(dijkstra(&g, 0, 4).unwrap().1, vec![0, 1, 2, 4]);
    }
}
```

2.14.3 The traffic window: from GPS fixes to edge weights

The aggregator at the heart of Section 2.12: a per-segment sliding window of observed speeds, the live/historical blend, and the weight the graph stores.

```
use std::collections::VecDeque;

/// Per-segment sliding-window speed average (Section 2.12).
pub struct SpeedWindow {
    samples: VecDeque<(u64, f64)>, // (timestamp_secs, observed speed m/s)
    window_secs: u64,              // e.g. 15 * 60
}

impl SpeedWindow {
    pub fn new(window_secs: u64) -> Self {
        Self { samples: VecDeque::new(), window_secs }
    }
```

```rust
    pub fn add(&mut self, now: u64, speed_mps: f64) {
        self.samples.push_back((now, speed_mps));
        self.evict(now);
    }

    fn evict(&mut self, now: u64) {
        while let Some(&(t, _)) = self.samples.front() {
            if now.saturating_sub(t) > self.window_secs { self.samples.pop_front(); }
            else { break; }
        }
    }

    /// Current average, or None when there is too little live data —
    /// the caller then blends in the historical pattern (Section 2.12).
    pub fn average(&mut self, now: u64, min_samples: usize) -> Option<f64> {
        self.evict(now);
        if self.samples.len() < min_samples { return None; }
        let sum: f64 = self.samples.iter().map(|&(_, s)| s).sum();
        Some(sum / self.samples.len() as f64)
    }
}

/// Effective speed for a segment: live average where we trust it,
/// the historical pattern for this day/time otherwise, and never above
/// the posted limit (we do not route people as if speeding were planned).
pub fn effective_speed(live: Option<f64>, historical: f64, limit: f64) -> f64 {
    live.unwrap_or(historical).min(limit)
}

/// The edge weight the routing graph stores (Section 2.7): expected
/// crossing time. Clamped away from zero so weights stay finite and
/// Dijkstra's non-negativity requirement is never endangered.
pub fn weight_millis(segment_length_m: f64, speed_mps: f64) -> u32 {
    (segment_length_m / speed_mps.max(0.5) * 1000.0) as u32
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn window_evicts_old_samples() {
        let mut w = SpeedWindow::new(900);          // 15-minute window
        w.add(1_000, 30.0);                          // highway at full speed
        w.add(1_700, 8.0);                           // jam begins
        w.add(1_800, 9.0);
        // The t=1000 sample is now 800 s old: inside the window.
        assert_eq!(w.samples.len(), 3);
        w.add(2_000, 10.0);
        // t=1000 is 1000 s old: evicted. Average over the jam only.
        let avg = w.average(2_000, 3).unwrap();
        assert!((avg - 9.0).abs() < 0.01);
    }

    #[test]
    fn sparse_segments_fall_back_to_history() {
        let mut w = SpeedWindow::new(900);
        w.add(100, 22.0);                            // one car at 3 a.m.
        assert_eq!(w.average(200, 10), None);        // not enough samples
        // Historical pattern says 20 m/s here at this day/time; limit 25.
```

```
        assert_eq!(effective_speed(None, 20.0, 25.0), 20.0);
        // Live data says 30 m/s, but we never plan on speeding.
        assert_eq!(effective_speed(Some(30.0), 20.0, 25.0), 25.0);
    }

    #[test]
    fn weight_is_crossing_time() {
        assert_eq!(weight_millis(1_000.0, 10.0), 100_000); // 1 km at 10 m/s
        assert!(weight_millis(1_000.0, 0.0) > 0);          // clamped, finite
    }
}
```

2.14.4 The reroute decision

Section 2.13's state machine, reduced to its durable core: distance from the planned polyline, sustained across fixes. (Distance uses a local flat-earth approximation — within a few hundred meters of the route, curvature is noise; production systems run the full map-matching of Section 2.12 here.)

```
/// Approximate meters-per-degree at a given latitude: 111.32 km per degree
/// of latitude everywhere; longitude degrees shrink with cos(latitude).
fn meters_per_deg(lat: f64) -> (f64, f64) {
    (111_320.0, 111_320.0 * lat.to_radians().cos())
}

/// Distance from point p to segment a-b, in meters (local approximation).
fn point_segment_m(p: (f64, f64), a: (f64, f64), b: (f64, f64)) -> f64 {
    let (m_lat, m_lng) = meters_per_deg(p.0);
    let (px, py) = (p.1 * m_lng, p.0 * m_lat);
    let (ax, ay) = (a.1 * m_lng, a.0 * m_lat);
    let (bx, by) = (b.1 * m_lng, b.0 * m_lat);
    let (dx, dy) = (bx - ax, by - ay);
    let len2 = dx * dx + dy * dy;
    let t = if len2 == 0.0 { 0.0 } else {
        (((px - ax) * dx + (py - ay) * dy) / len2).clamp(0.0, 1.0)
    };
    let (cx, cy) = (ax + t * dx, ay + t * dy);   // closest point on a-b
    ((px - cx).powi(2) + (py - cy).powi(2)).sqrt()
}

/// Distance from a fix to the nearest point anywhere on the route polyline.
pub fn distance_to_route_m(p: (f64, f64), route: &[(f64, f64)]) -> f64 {
    route.windows(2)
        .map(|w| point_segment_m(p, w[0], w[1]))
        .fold(f64::MAX, f64::min)
}

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Guidance { OnRoute, OffRouteSuspected, Reroute }

/// The hysteresis filter: ONE noisy fix must never trigger a reroute
/// (Section 2.13's pitfall). `strikes` counts consecutive off-route fixes.
pub struct RerouteFilter {
    pub distance_threshold_m: f64, // e.g. 40 m
    pub required_consecutive: u8,  // e.g. 3 fixes
    strikes: u8,
}

impl RerouteFilter {
    pub fn new(distance_threshold_m: f64, required_consecutive: u8) -> Self {
        Self { distance_threshold_m, required_consecutive, strikes: 0 }
```

```
    }

    pub fn update(&mut self, fix: (f64, f64), route: &[(f64, f64)]) -> Guidance {
        if distance_to_route_m(fix, route) > self.distance_threshold_m {
            self.strikes += 1;
        } else {
            self.strikes = 0;
        }
        if self.strikes >= self.required_consecutive { Guidance::Reroute }
        else if self.strikes > 0 { Guidance::OffRouteSuspected }
        else { Guidance::OnRoute }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    // A straight eastbound route along lat 37.0.
    fn route() -> Vec<(f64, f64)> { vec![(37.0, -122.0), (37.0, -121.9)] }

    #[test]
    fn on_route_fix_stays_quiet() {
        let mut f = RerouteFilter::new(40.0, 3);
        assert_eq!(f.update((37.0001, -121.95), &route()), Guidance::OnRoute);
    }

    #[test]
    fn sustained_deviation_reroutes() {
        let mut f = RerouteFilter::new(40.0, 3);
        // ~56 m north of the route: beyond threshold.
        let lost = (37.0005, -121.95);
        assert_eq!(f.update(lost, &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update(lost, &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update(lost, &route()), Guidance::Reroute);
    }

    #[test]
    fn one_ghost_fix_recovers() {
        let mut f = RerouteFilter::new(40.0, 3);
        assert_eq!(f.update((37.0005, -121.95), &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update((37.0, -121.95), &route()), Guidance::OnRoute);
    }
}
```

2.15 Scaling & Geographic Sharding

Chapter 1 defined sharding; here the shard key is **geography itself**.

Tier	Sharding strategy	Why
Tiles	No sharding logic at all — the `(z,x,y)` address space partitions naturally; CDN edges cache by URL	Addressing **is** the partition scheme; popularity skew does the rest
Road graph	Partitioned by region (continent → subregion); each shard is a full in-	A route is overwhelmingly inside one region; replicas add both capacity and failover

Tier	Sharding strategy	Why
	memory copy of its region's CH graph, replicated	
Cross-region routes	Solved hierarchically: mega-segments cross region borders at a handful of highway gateways	The hierarchy of Section 2.8 doubles as the distributed-systems answer
GPS stream	Topic partitioned by geohash prefix; each stream-processor instance owns a disjoint set of cells	Map-matching and aggregation are embarrassingly parallel over geography
Traffic cache	Sharded by `segment_id` hash; total size 500 MB	Tiny, hot, and latency-critical — keep it in memory, close to routing
Place index	Replicated per region; sharded by quadkey prefix within a region	Search is read-heavy and geographically scoped (Section 2.14)

KEY INSIGHT — Geography is a forgiving shard key

Unlike user data (Chapter 1's documents), geography does not move, grow legs, or go viral. A region's road graph changes on the scale of weeks; its traffic weights change on the scale of minutes but are **regenerated**, not migrated. The one genuinely hot spot — a stadium emptying at 11 p.m. — is absorbed inside one partition by the aggregation pipeline, not spread across the system.

2.16 Failure Modes & Recovery

Failure	Handling
WS gateway dies	Clients reconnect to any gateway; the connection manager re-registers them; the navigation service resumes pushes to the new socket. Unacked client updates are retried and deduplicated (Chapter 1's idempotency discipline).
Navigation node dies	Sessions are small (route + progress). The client's reconnect carries its last known position; a fresh node rebuilds the session from the route store and continues. User sees a pause, not a crash.
Stream backpressure / Kafka lag	Freshness degrades first and **visibly**: per-segment entries age past their TTL, and the system slides to historical speeds automatically — the fallback of Section 2.12 is the degradation mode, by design. Recovery replays the log.
Traffic cache loss	Rebuilt from the stream within one window (15 min); historical speeds serve in the meantime.
Routing node dies	Each region shard is replicated; traffic shifts to a replica. A lost node reloads its graph from the segment store plus the traffic cache (Section 2.10).
Tile origin down	The CDN keeps serving — 95%+ of requests never noticed the origin existed. `stale-while-revalidate` semantics make even expired tiles safe, since the base map changes weekly at worst.

Failure	Handling
Bad or spoofed GPS input	Per-segment sample thresholds ignore singletons; per-source scoring quarantines systematically wrong feeds (Section 2.12).
Full region outage	The phone is the last fallback: cached route, tiles, and step list keep guiding offline (Section 2.13) while sessions re-home to another region.

2.17 Trade-offs & Alternatives

Decision	Benefit purchased	Cost accepted
Vector tiles over raster	50% fewer bytes; restyle without re-render; smooth zoom	Client GPU does the work; a raster fallback must exist for old clients
CH / mega-segments over plain A*	1000× query speedup; the only way 20k QPS fits a small fleet	Hours of preprocessing; shortcut weights must be rebuilt or bubbled-up when traffic moves — extra machinery (Section 2.8)
WebSocket push for navigation	Reroutes and ETA updates arrive instantly; one connection also carries GPS upstream	Millions of long-lived connections to hold and to register (connection manager)
Hybrid live + historical traffic	Coverage everywhere, graceful under sparse data and pipeline lag	Two data paths to keep consistent; blending logic to get right
Quadkey/geohash indexing	Proximity becomes a string-prefix range scan in ordinary KV stores	Cell-boundary artifacts — always query the neighbor ring too
Precompute over on-demand render	Tiles and hierarchy make reads O(cache hit)	A planet of batch jobs to run, version, and roll back

2.18 Observability & SLOs

DEFINITION — MAPE (mean absolute percentage error)

The average of `|predicted − actual| / actual` over many predictions — the standard accuracy metric for forecasts, and the natural SLI for ETA quality. Per **completed** trip, we know both the ETA we gave and the realized travel time, so ETA MAPE is measurable exactly, per city, per hour, per route class. The source talk is emphatic on this point: you cannot improve what you do not score, and ETA accuracy is the metric this product lives by.

What we instrument, at minimum: **ETA error distribution** (predicted vs. realized, per region — the chapter's headline SLI, targeting the ±10% of Section 2.4); **route acceptance rate** (how often users actually drive the recommended route — a recommendation nobody takes is a signal something is wrong with the routes, per the talk's analytics discussion); reroute rate per session; tile cache hit ratio

and origin QPS; route latency p50/p99; pipeline lag (fix → weight update); traffic freshness (median age of live segment speeds); navigation session drops and recovery times.

2.19 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- *"Offline maps?"* — Ship regional bundles (tiles + contracted graph) to the device; routing runs locally; traffic and reroute quality silently degrade until reconnect. Everything we precomputed server-side is exactly what the bundle contains.
- *"Public transit?"* — A **time-dependent** graph (edges exist only when a vehicle does); Dijkstra generalizes, but production transit uses schedule-based algorithms (RAPTOR and friends). Name it, don't build it.
- *"GPS noise in urban canyons?"* — The HMM map-matching of Section 2.12, plus sensor fusion (accelerometer, gyroscope, wheel ticks where available).
- *"Better ETAs?"* — Features into a learned model: current and historical speeds, weather, events, turn and signal penalties; graph neural networks model congestion **spreading** along the road graph. Google reported up to 50% ETA-error reductions in some cities from exactly this move — say the number as motivation, then defend the simple hybrid as the baseline it must beat.
- *"Walking and cycling?"* — Same graph, same algorithms, different edge eligibility and weights (stairs, bike lanes, no highways). The design's generality is the answer.
- *"Location privacy?"* — Aggregate, don't store: segment statistics instead of traces, retention limits on raw updates, minimum sample counts that double as k-anonymity for sparse segments.

If you remember five things:

1. The map is a precomputed pyramid of addressable squares; the CDN, not your fleet, serves it.
2. Roads are a time-weighted directed graph, and **traffic is just edge weights that move**.
3. Dijkstra is exact and unusable; bidirectional and A* are free wins; hierarchy (mega-segments / contraction hierarchies) is the 1000× that makes it a product.
4. Live traffic is a streaming pipeline: GPS fix → map-match → per-segment windowed average → weights and overlay tiles, with history as the fallback.
5. Navigation correctness lives on the phone: the client caches the route and survives the server.

2.20 Summary & Further Reading

We designed a maps and navigation service for 1B monthly users: a pre-rendered, CDN-served tile pyramid addressed by `(z, x, y)` and quadkeys; a time-weighted road graph held in memory and searched with an algorithm ladder that ends at contraction hierarchies; a live traffic loop that turns 3M GPS updates per second into per-segment speeds, edge weights, and overlay tiles, with historical patterns as the fallback; and a turn-by-turn navigation service that guides, refreshes ETAs, and reroutes with hysteresis — degrading to the phone when the network or we fail.

Primary source for this chapter:

- **"13: Google Maps" — Systems Design Interview Questions With Ex-Google SWE (Jordan has no life)** — the walkthrough this chapter expands.

Foundations worth reading:

- The OSRM and Valhalla open-source routing engines — contraction hierarchies in production code, not just papers.
- Geisberger et al., *Exact Routing in Large Road Networks Using Contraction Hierarchies* (2008) — the CH formulation.
- Newson & Krumm, *Hidden Markov Map Matching Through Noise and Sparseness* (2009) — the map-matching standard.

- Google's S2 geometry library documentation, and Uber's H3 — the real spatial indexes behind quadkey-style addressing.
- The Bing Maps tile system documentation — the canonical quadkey reference.
- Google DeepMind's write-up on learning ETAs with graph neural networks (2020) — the ML direction for Section 2.19's follow-up.

2.21 Chapter 2 Glossary

A one-glance index of every term this chapter defined. Chapter 1's glossary is assumed; later chapters assume both.

Term	Meaning in one line
ETA	Predicted travel time / arrival time; the quality bar of this chapter
Freshness	Age of the data behind an answer; traffic targets minutes
Map tile	Fixed 256×256 square of map; the unit of storage, cache, transfer
Zoom level	Scale integer *z*; each step splits every tile into four
Slippy-map scheme	Deterministic (z, x, y) tile addressing over Web Mercator
Quadkey	Tile address as a digit string; a quadtree path, prefix-searchable
CDN	Planet-wide edge caches that absorb static reads near the user
TTL / hit ratio	How long cache entries may live / fraction of requests they absorb
Content addressing	Identical tiles (oceans) stored once, by content hash
Polyline encoding	Compact ASCII encoding of a route's coordinate sequence
Graph	Nodes + edges; directed and weighted for roads
Segment	One directed road piece between nodes; the unit of traffic
Adjacency list	Per-node outgoing-edge lists; the in-memory graph layout
Shortest path	Minimum-total-weight route; with time weights, the fastest path
Dijkstra's algorithm	Exact breadth-by-distance search with a priority queue
Priority queue	O(log n) "give me the current minimum" structure
Bidirectional search	Two half-searches meeting mid-route; half the work
A* / heuristic	Dijkstra steered by a remaining-distance estimate
Admissible heuristic	Never overestimates; preserves exactness
Haversine distance	Great-circle distance between coordinates
Mega-segment	Precomputed synthetic edge summarizing a segment chain
Contraction hierarchy	Node-importance ordering + shortcuts; µs–ms queries
Kafka-class stream	Durable partitioned event log; decouples and replays
Stream processing	Computing over data as it arrives, not in batch
Map-matching	Snapping noisy GPS fixes to the true road segment
Sliding window	Moving time interval defining "current" speed per segment
Polyglot persistence	Different engines for different access patterns

Term	Meaning in one line
Connection manager	Registry of which gateway holds which user’s socket
MAPE	Mean absolute percentage error; the ETA accuracy SLI

CHAPTER 3

Designing a Distributed Rate Limiter

PROBLEM SOURCE

This chapter solves the problem posed in the talk **"7: Design a Rate Limiter"** from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The talk designs a rate-limiting service for a public API: why limiting is a business feature and not just a shield, the classic limiting algorithms, and how to enforce limits correctly across a fleet of stateless servers. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol details, and Rust reference implementations.

3.1 The Problem Statement

The interviewer looks up and says:

> *"Design a rate limiter. Our public API is used by a million developers, and we need to make sure no single caller can overwhelm the service — or use more than their plan allows."*

Chapters 1 and 2 designed user-facing products. This chapter designs a piece of **infrastructure** — a component that sits in front of every request and decides, in well under a millisecond, whether the request may proceed. The difficulty is not the concept ("count requests, reject past a threshold" is a five-line program on one machine). The difficulty is doing it **correctly and cheaply on the hot path of a distributed system**: fifty gateway servers must enforce one shared limit per caller, under concurrency, without meaningful latency, and without the limiter itself becoming the outage it exists to prevent.

DEFINITION — Rate limiting / throttling

Controlling the **rate** at which a caller may invoke a system: at most n requests per time window per identity (API key, user, IP address, tenant). Calls beyond the limit are rejected (or queued, or slowed — hence *throttling*). Rate limiting serves three distinct masters: **protection** (abuse, brute force, accidental stampede), **capacity** (one noisy caller must not starve the rest), and **monetization** (usage tiers are limits with a price tag). Keep all three in mind: they produce different requirements.

DEFINITION — Quota vs. rate limit

A *quota* is a budget over a long period ("100,000 calls per month") used for billing and plan enforcement; it tolerates approximate, batched accounting. A *rate limit* governs short windows ("100 calls per second") to shape live traffic; it must be enforced **now**, on the request's critical path. This chapter is about rate limits; Section 3.18 discusses how quotas differ.

3.2 Scope & Clarifying Questions

The prompt spans everything from login-page brute-force protection to planetary DDoS absorption. Narrow it:

Speaker	Dialogue
Candidate	"What are we limiting — HTTP API calls, or something else like messages or bytes?"
Interviewer	"HTTP requests to our public REST API. Limit per API key."
Candidate	"Are limits the same for everyone, or tiered — free keys vs. paid plans?"
Interviewer	"Tiered. Rules must be configurable per key and per route, and changeable without redeploying anything."
Candidate	"Scale of the API being protected?"
Interviewer	"Peak 200,000 requests per second, spread across a fleet of about 50 stateless gateway servers. One million registered API keys."
Candidate	"How exact must enforcement be? If a key is limited to 100 requests per second, is 101 a scandal?"
Interviewer	"Never reject a caller who is under their limit. Small overshoot during failures is acceptable; systematic overshoot is not."
Candidate	"How much latency may the check add to each request?"
Interviewer	"A millisecond or two at most — this runs on every single request."
Candidate	"And if the limiter's own state store is down — block everything, or let traffic through?"
Interviewer	"Great question. Decide, and defend the decision."

AGREED SCOPE

Per-API-key rate limits on a public REST API; tiered limits, configurable per key and per route, hot-reloadable; enforced **globally** across 50 stateless gateway nodes at **200k RPS** peak; ≤1–2 ms added latency; **no false rejections**, bounded overshoot; and an explicit, defended policy for limiter failure (fail-open vs. fail-closed — Section 3.15).

INTERVIEW TIP — The last question is the interview

"What do we do when the limiter breaks?" separates candidates who build components from candidates who build **systems**. Ask it yourself if the interviewer doesn't — every subsequent design decision (centralized store, local fallbacks, timeouts) is downstream of the answer.

3.3 Functional Requirements

Chapter 1 defined functional requirements; ours:

1. **FR-1 — Enforcement.** A caller exceeding its limit receives HTTP **429 Too Many Requests** with a `Retry-After` hint; everyone else passes through.
2. **FR-2 — Configurable rules.** Limits are defined per API key, optionally per route or method, with named tiers (free, pro, enterprise); an admin API changes them at runtime.

3. **FR-3 — Transparency.** Responses carry the caller's limit state (limit, remaining, reset time) in standard headers, so well-behaved clients can self-throttle **before** being rejected.
4. **FR-4 — Global enforcement.** The limit holds across the entire gateway fleet, not per server — a caller with 100 req/s cannot get 5,000 req/s by being load-balanced across 50 nodes.
5. **FR-5 — Burst policy.** Short bursts above the sustained rate are allowed up to a defined allowance (real traffic is bursty; rejecting every microburst punishes normal clients).
6. **FR-6 — No false rejections.** A caller under its limit is never rejected. (Overshoot is a soft failure; false rejection is a hard one — it breaks innocent customers deterministically.)

3.4 Non-Functional Requirements

Three qualities dominate, and they pull against each other — naming the tension is half the answer:

Quality	Target
Added latency	≤ 1 ms in-region, p99 — the check is on **every** request's critical path
Accuracy	Zero false rejections; steady-state overshoot within 1% of the limit; degraded-mode overshoot explicitly bounded
Availability of the API	The limiter must never be a new single point of failure: if the limiter fails, the API stays up (fail-open), with a local safety cap
Scale	200k RPS enforcement across 50 nodes; 1M keys; 100M+ active keys per day tolerable
Config freshness	Rule changes propagate to all gateways within seconds

DEFINITION — False rejection / overshoot

The two ways a limiter can be wrong. A *false rejection* denies a caller who is under the limit — an availability bug, visible and unforgivable. *Overshoot* allows more than the limit — a protection degradation, tolerable in small doses. Accuracy requirements should always be stated in this asymmetric vocabulary, because the two errors have opposite costs and different causes.

KEY INSIGHT — Latency, accuracy, availability: pick the trade-off consciously

Exact global counting wants a synchronous round trip to a strongly consistent store (latency + availability cost). **Zero added latency** wants purely local counting (accuracy cost: 50 nodes × local limit). **Maximum availability** wants fail-open everywhere (protection cost). Every real design is a point in this triangle; the rest of the chapter locates ours and prices it.

3.5 Back-of-the-Envelope Estimation

Assumptions (stated, per Chapter 1's discipline):

- Peak gateway traffic: **200k requests/sec**; every request needs exactly one limit check.
- 1M registered keys; up to **100M keys active in a day** (many keys are used once and idle for weeks).
- Typical limit: 100 req/s per key; token-bucket state per active key is two numbers plus the key itself — 50–60 bytes.
- A modern in-memory store serves 100k simple ops/sec per shard.

Derived numbers:

Quantity	Estimate	How
Check operations	200k ops/s peak	one per request; reads+writes combined
Store shards needed	4–6 (+ replicas)	200k ops/s ÷ 100k ops/s per shard, with headroom
State memory (counters)	≈ 6 GB worst case	100M active keys × 60 B
State memory (sliding log)	≈ 800 GB worst case	100M keys × up to 1,000 time-stamps × 8 B — **infeasible**
Added latency budget	≤ 1 ms	one in-region round trip, or zero with local state
Config size	a few MB	1M keys × rule record; trivially cacheable everywhere

KEY INSIGHT — The scarce resource is round trips, not hardware

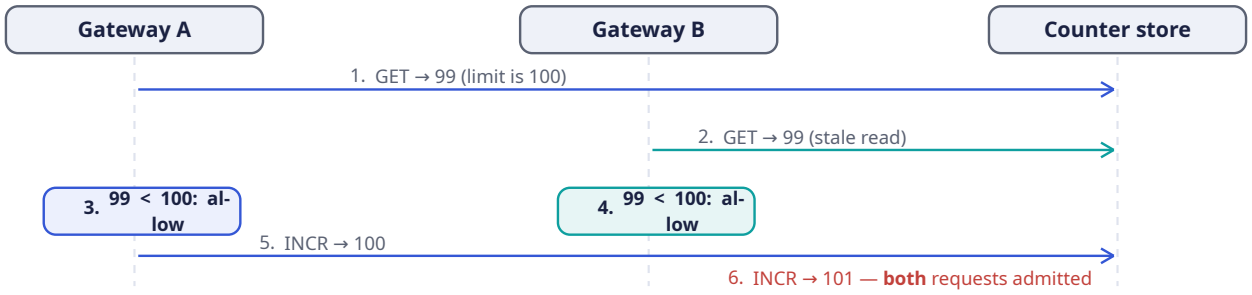
Six gigabytes of state and a few hundred thousand ops per second are, by Chapter 2's standards, nothing. The entire design problem is: **where does the check happen relative to the request?** A centralized exact check costs one network round trip (1 ms in-region — the whole budget). A local check costs zero round trips but is approximate. Estimation tells you this is a **latency topology** problem, not a capacity problem — design accordingly.

3.6 The Core Challenge: Counting Correctly Under Concurrency

The limit check is a read-modify-write: **read** the caller's count, **decide**, **write** the incremented count. Two naive placements fail in instructive ways.

Naive strategy 1 — count locally on each gateway. No shared state, zero added latency. But the limit is **per key**, and a key's requests land on all 50 gateways. If each node enforces 100 req/s locally, the caller effectively has **5,000 req/s** — FR-4 is unmet. Lowering each node's limit to 100/50 = 2 req/s misfires the other way: a caller whose traffic happens to hit one node gets strangled. Per-instance limits cannot express a global limit.

Naive strategy 2 — a shared counter store with plain reads and writes. Put the counters in one shared, in-memory store; each gateway does `GET`, compare, `INCR`. This fails under concurrency:



DEFINITION — Race condition / check-then-act (TOCTOU)

A *race condition* is a bug whose outcome depends on the uncontrolled interleaving of concurrent operations. The specific species above is **time-of-check to time-of-use**: the decision ("count is 99, under the limit") is based on state that another actor changes before our write lands. Both gateways checked

honestly; both were correct **at check time**; the limit was still exceeded. No amount of retries fixes a TOCTOU hole — the check and the update must become **one indivisible operation**.

DEFINITION — Atomicity / compare-and-swap (CAS)

An operation is *atomic* if it executes entirely or not at all, with no observable intermediate state. *Compare-and-swap* is the primitive form: "set this value to V_2 only if it currently equals V_1," performed as one hardware/server-side step. Our options for making the check atomic: a server-side script executed atomically by the store (Section 3.12), a CAS loop, or a single atomic increment whose **returned value** is the decision input. The last two need no scripting at all — `INCR` already returns the post-increment count, which **is** the atomic observation we need.

So the core challenge decomposes into two questions that the rest of the chapter answers: **what exactly is the count** (which algorithm — Section 3.7), and **how is the check made atomic across the fleet** (Section 3.12).

3.7 The Algorithm Zoo

Five algorithms cover the entire design space. Each is a different answer to "what does 'at most *n* per window' mean, exactly?"

DEFINITION — Fixed window counter

Divide time into fixed windows (e.g., each clock minute) keyed `rl:{key}:{window_id}`; each request increments the current window's counter; reject when it exceeds the limit. O(1) memory per key, one atomic increment per request — and a famous flaw: **the boundary burst**. A caller can fire 100 requests at 0:59.9 and 100 more at 1:00.0 — 200 requests in 0.1 seconds, all legal, because the two bursts sit in different windows.

DEFINITION — Sliding window log

Store the **timestamp of every request** in a per-key log; on each request, drop timestamps older than the window and reject if the remaining count reaches the limit. Perfectly accurate, no boundary problem — but memory is O(limit) per key. Section 3.5 priced this at 800 GB at our scale: the log is the algorithm you describe to show you know the trade-off, not the one you ship.

DEFINITION — Sliding window counter

The fixed-window/log hybrid. Keep the current and previous window counters only, and estimate the sliding window as:

```
estimate = current_count + previous_count × (1 − elapsed / window)
```

If 15 seconds of a 60-second window have elapsed, the previous window's 84 requests count as `84 × 0.75 = 63`. O(1) memory, one increment per request, no boundary burst — at the price of being an **estimate** (it assumes the previous window's traffic was spread evenly, which is fair on aggregate; published analyses put the misclassification rate well under 1%).

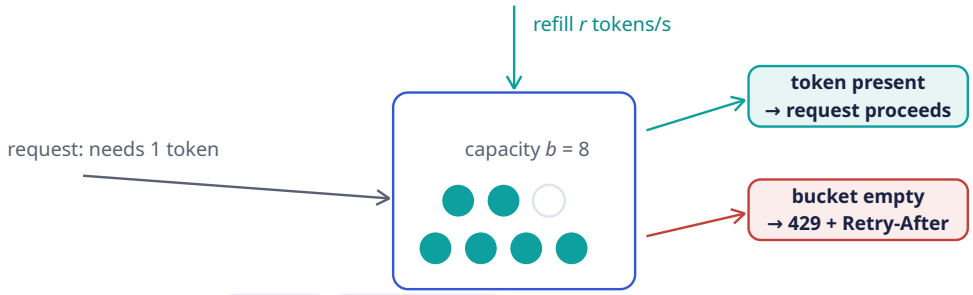
DEFINITION — Token bucket

Each key owns a bucket holding up to *b* tokens, refilled continuously at *r* tokens per second. A request takes one token; an empty bucket rejects. The parameters map directly onto product language: *r* is the sustained rate, *b* is the burst allowance (FR-5). O(1) memory (two numbers: tokens, last-refill

timestamp), and refill is computed **lazily** on each request — no timers, no background jobs. This is the industry's default for API limiting, and our choice; Section 3.11 deep-dives it and Section 3.13 implements it.

DEFINITION — Leaky bucket / GCRA

Requests enter a conceptual queue and "leak" out at a fixed rate; a request is admitted only if the queue has room. Where the token bucket **permits** bursts, the leaky bucket **forbids** them — output is perfectly smooth. Its stateless formulation is the **Generic Cell Rate Algorithm**: store one theoretical-arrival-time per key; admit if now is not earlier than that time minus tolerance. The right tool when you meter **into** a fragile downstream (payment gateways, SMS providers), overkill for request admission.



Two numbers per key — `tokens`, `last_refill` — and no timers: refill is computed when a request arrives.

The comparison that ends the discussion:

Algorithm	Memory/key	Exact?	Bursts?	Notes
Fixed window	O(1)	window-granular	2× at boundary	Simplest; the boundary burst is the interview trap
Sliding log	O(limit)	exact	no	Memory kills it at scale (Section 3.5)
Sliding counter	O(1)	1% error	smoothed	Best accuracy-per-byte when bursts are unwanted
Token bucket	O(1)	exact accounting	**allowed** up to *b*	**Our choice**: burst policy is a product feature here
Leaky / GCRA	O(1)	exact pacing	forbidden	Choose when downstream needs smooth flow

3.8 API & Protocol Design

A rate limiter's "API" is mostly **other people's responses**. The contract that matters:

DEFINITION — HTTP 429 / Retry-After

Status **429 Too Many Requests** (RFC 6585) means "you, specifically, are calling too fast." The `Retry-After` header tells the caller how many seconds to wait before retrying. Together they convert

> a rejection into a negotiation: a well-built client backs off exactly as instructed instead of hammering harder. A limiter that rejects without `Retry-After` trains clients to retry blindly — worse for everyone.

Header	Meaning
`X-RateLimit-Limit`	The caller's limit for this route, e.g. `100`
`X-RateLimit-Remaining`	Requests left in the current window / tokens left
`X-RateLimit-Reset`	When the budget refills (epoch seconds or seconds-from-now)
`Retry-After`	On 429 only: minimum wait before retry, seconds

(An IETF draft standardizes these as `RateLimit-*` fields; the `X-` forms remain the de-facto convention. Either is defensible — knowing both exist is the point.)

A rejection response, end to end:

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 45
Retry-After: 1
Content-Type: application/json

{ "error": "rate_limit_exceeded", "limit": 100, "window": "1s", "plan": "free" }
```

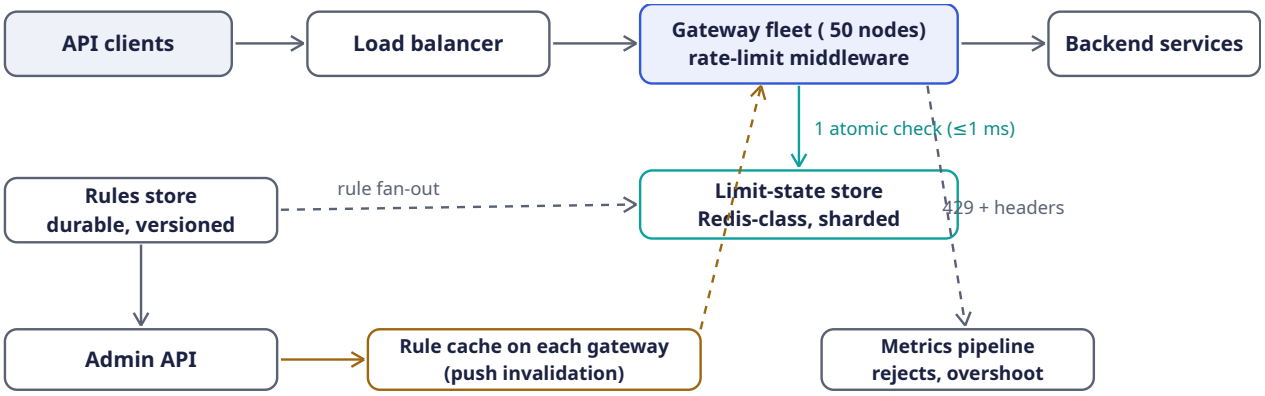
The **admin** surface (FR-2, FR-6) is ordinary REST: rules as records — `{ api_key or tier, route pattern, algorithm, limit/rate, burst, window }` — created and updated through an admin API, versioned, and pushed to every gateway within seconds (Section 3.10).

3.9 Data Model & Storage

Entity	Contents	Store
Limit state	`rl:{api_key}:{route}` → `{tokens, last_refill_ms}` (bucket) or window counters; TTL = window so idle keys self-clean	In-memory store cluster (Redis-class), sharded by key hash
Rule	tier or key → {algorithm, rate, burst, window, route pattern}; versioned	Durable KV/relational store, fanned out to gateway-local caches
Quota ledger	monthly usage per key for billing	Append-only log → batch aggregation; **not** on the request path (Section 3.2)

Two decisions to name. First, state entries carry a **TTL** (Chapter 2) equal to the window, so the 100M-active-keys problem is self-cleaning: idle keys vanish without a garbage collector. Second, rules are cached **on each gateway** with push-based invalidation — a per-request rule lookup must never add a second network hop.

3.10 High-Level Architecture



The middleware is a read-modify-write on the hot path: one atomic store op per request, rules read locally.

Component	Responsibility	Why it is shaped this way
Rate-limit middleware	First code every request touches on the gateway: load rule → atomic check → pass or 429	Middleware placement means **no backend code changes** and one consistent policy for every route
Limit-state store	Atomic counters/buckets for all active keys	One shared store is what makes the limit **global** (FR-4); sharded by key hash, replicated for availability
Rules store + admin API	Versioned rule records; runtime updates	Config is tiny and read-hot: push it to gateway memory, never fetch per request
Gateway rule cache	Rules in process memory, invalidated by push	Zero added hops for config (Section 3.9)
Metrics pipeline	Every decision emitted as an event	Section 3.17: you cannot tune what you cannot see

> **KEY INSIGHT — The one-op rule**
>
> Per request, the design allows itself **exactly one** atomic store operation and zero synchronous config reads. Everything — the algorithm choice (O(1) state), the TTL self-cleaning, the gateway rule caches — exists to preserve that invariant. When an interviewer pushes on any component, return to it.

3.11 Deep Dive: The Token Bucket, Precisely

The token bucket's elegance is that **time is never advanced by a timer**. State is two numbers, updated lazily when a request arrives:

1. `tokens` — current balance, capped at capacity *b*.
2. `last_refill_ms` — when the balance was last recomputed.

On each request at time `now`:

```
elapsed_s   = (now - last_refill_ms) / 1000
tokens      = min(b, tokens + elapsed_s * r)   // lazy refill, capped
```

```
last_refill = now
if tokens >= 1: tokens -= 1; ALLOW (remaining = floor(tokens))
else:           DENY  (retry_after_ms = (1 - tokens) / r * 1000)
```

Reading the parameters as product language: r = sustained rate ("100 req/s"), b = burst allowance ("… with bursts up to 150"). The retry hint is free: `retry_after` is exactly how long until one token exists.

A worked trace (limit 5 req/s, burst 8), to have in hand at the whiteboard:

Time	Event	Balance after	Why
`t = 0.0 s`	8 requests arrive together	8 → 0	Full bucket: all 8 pass — the burst allowance
`t = 0.0 s`	9th request	0	Empty: 429, `Retry-After: 1` (1/5 s until one token)
`t = 1.0 s`	1 request	5 → 4	One second refilled 5 tokens
`t = 3.0 s`	1 request	8 → 7	Two more seconds refill 10, capped at capacity 8

COMMON PITFALL — Refill by background timer

Implementing refill as a cron or timer per key means a million timers, clock drift between timer and request paths, and state that never cleans itself up. Lazy refill has none of these: no timers, no drift (one clock, read once per request), and a TTL on the key reclaims idle state for free. If you find yourself scheduling work per key, you have re-invented the sliding log's costs inside the token bucket.

NOTE — One clock, and make it monotonic

Distributed machines disagree about wall-clock time (clock skew), and even on one machine the wall clock can jump backwards (NTP corrections) — a backward jump would **refund** tokens. Two defenses: (1) perform the bucket update **inside the store** with a server-side script, so one clock governs every gateway (Section 3.12); (2) where local time is unavoidable, read a **monotonic** clock (one that never moves backwards), as Section 3.13's Rust does.

3.12 Deep Dive: Distributed Enforcement & the Race

Section 3.6's race came from splitting **check** and **update** across a network. Three production-grade fixes, in increasing order of sophistication:

Fix 1 — atomic increment-as-decision. For window algorithms, one atomic `INCR` already returns the post-increment value: the store's returned count **is** the check. Reject when the returned value exceeds the limit. One round trip, no client-side race, and the first increment of a window sets the key's TTL in the same atomic step. Fixed window and sliding counter ship this way.

Fix 2 — server-side script for read-modify-write. Token bucket needs read-compute-write (refill, compare, decrement). Executing that sequence **on the store**, as one atomic script, removes the race identically: the store serializes script execution, so no two requests ever observe the same balance. Section 3.13 shows the client side, with the script embedded as data.

Fix 3 — approximate local counting. The radical option: skip the store on the hot path entirely. Each gateway keeps local counters and periodically **publishes** them; a background aggregator broadcasts fleet-wide totals, and each node denies when `local_estimate + fleet_total ≥ limit`. Zero added latency, and the store is off the critical path (fail-open for free) — but between syncs, overshoot is

bounded by roughly `limit × (sync_lag / window)`. Choose it when protection matters more than precision (abuse mitigation); never for billing-adjacent limits.

Approach	Latency	Accuracy	Choose when
Centralized, atomic op	+1 RTT (1 ms)	exact	Default. Paid tiers, contractual limits
Centralized, non-atomic	+1 RTT	races under concurrency	Never — Section 3.6
Local + async sync	0	bounded overshoot	Abuse protection at extreme scale, or store-failure fallback
Sticky routing + local	0	near-exact	Only if your load balancer can pin keys to nodes — fragile on failover

INTERVIEW TIP — Say the quiet part: exactness is a product decision

"How exact does this need to be?" has no technical answer — it depends on whether the limit protects revenue (exact: paid tiers), infrastructure (approximate fine: abuse), or other users' experience (approximate fine: fairness). Volunteering this framing turns an algorithm comparison into a senior-level requirements discussion.

3.13 Deep Dive: Rust Reference Implementations

Three pieces, all executable: the token bucket with deterministic-time tests, the fixed-window/sliding-counter pair showing the boundary burst and its cure, and the atomic store check with the server-side script embedded as data.

3.13.1 Token bucket with a testable clock

Time is injected through a `Clock` trait so tests control it exactly — the difference between a test that **suggests** correctness and one that **proves** it.

```rust
use std::sync::Mutex;

/// Time source, injected so tests can control it (Section 3.11).
pub trait Clock: Send + Sync {
    fn now_ms(&self) -> u64;
}

/// Production clock: monotonic — it can never jump backwards and refund
/// tokens (Section 3.11's note).
pub struct MonotonicClock(std::time::Instant);
impl MonotonicClock {
    pub fn new() -> Self { Self(std::time::Instant::now()) }
}
impl Clock for MonotonicClock {
    fn now_ms(&self) -> u64 { self.0.elapsed().as_millis() as u64 }
}

/// The decision every algorithm returns: pass, or reject with a retry hint
/// that becomes the Retry-After header (Section 3.8).
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Decision {
```

```
    Allow { remaining: u64 },
    Deny { retry_after_ms: u64 },
}

/// Token bucket (Section 3.7): `capacity` tokens, refilled lazily at
/// `refill_per_sec`. The Mutex makes the read-modify-write atomic *within*
/// one process; Section 3.12 handles the fleet-wide version.
pub struct TokenBucket<C: Clock> {
    capacity: f64,
    refill_per_sec: f64,
    state: Mutex<(f64, u64)>, // (tokens, last_refill_ms)
    clock: C,
}

impl<C: Clock> TokenBucket<C> {
    pub fn new(capacity: f64, refill_per_sec: f64, clock: C) -> Self {
        let now = clock.now_ms();
        Self { capacity, refill_per_sec,
               state: Mutex::new((capacity, now)), clock }
    }

    pub fn try_acquire(&self) -> Decision {
        let now = self.clock.now_ms();
        let mut s = self.state.lock().unwrap();
        let elapsed_s = (now - s.1) as f64 / 1000.0;
        let mut tokens = (s.0 + elapsed_s * self.refill_per_sec).min(self.capacity);
        s.1 = now;
        if tokens >= 1.0 {
            tokens -= 1.0;
            s.0 = tokens;
            Decision::Allow { remaining: tokens as u64 }
        } else {
            s.0 = tokens;
            let wait_ms = ((1.0 - tokens) / self.refill_per_sec * 1000.0).ceil() as u64;
            Decision::Deny { retry_after_ms: wait_ms.max(1) }
        }
    }
}
```

Tests, including the concurrency proof that `Mutex` buys us exactly-once accounting across threads:

```
#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::{Arc, atomic::{AtomicU64, Ordering}};

    /// A clock the test controls explicitly.
    struct TestClock(AtomicU64);
    impl Clock for TestClock {
        fn now_ms(&self) -> u64 { self.0.load(Ordering::SeqCst) }
    }

    #[test]
    fn burst_then_reject_then_refill() {
        let clock = Arc::new(TestClock(AtomicU64::new(0)));
        let b = TokenBucket::new(8.0, 5.0, clock.clone()); // 8 burst, 5/s
        for i in 0..8 {
            assert!(matches!(b.try_acquire(), Decision::Allow { .. }), "burst #{i}");
        }
        // Bucket empty: denied, and told to wait 1/5s for one token.
```

```
        assert_eq!(b.try_acquire(), Decision::Deny { retry_after_ms: 200 });
        // One second passes: 5 tokens return (capped at capacity).
        clock.0.store(1000, Ordering::SeqCst);
        assert_eq!(b.try_acquire(), Decision::Allow { remaining: 4 });
    }

    #[test]
    fn exactly_capacity_across_threads() {
        let clock = Arc::new(TestClock(AtomicU64::new(0)));
        let b = Arc::new(TokenBucket::new(100.0, 1.0, clock));
        let allowed = Arc::new(AtomicU64::new(0));
        std::thread::scope(|s| {
            for _ in 0..8 {
                let (b, allowed) = (b.clone(), allowed.clone());
                s.spawn(move || {
                    for _ in 0..25 {    // 8 threads x 25 = 200 attempts
                        if matches!(b.try_acquire(), Decision::Allow { .. }) {
                            allowed.fetch_add(1, Ordering::SeqCst);
                        }
                    }
                });
            }
        });
        assert_eq!(allowed.load(Ordering::SeqCst), 100); // never one more
    }
}
```

3.13.2 Fixed window, its boundary burst, and the sliding counter's cure

```
/// Fixed window counter (Section 3.7): one counter per window per key.
pub struct FixedWindow {
    window_ms: u64,
    count: u64,
    window_start_ms: u64,
}

impl FixedWindow {
    pub fn new(window_ms: u64) -> Self {
        Self { window_ms, count: 0, window_start_ms: 0 }
    }

    pub fn try_acquire(&mut self, now_ms: u64, limit: u64) -> bool {
        let w = now_ms / self.window_ms * self.window_ms;   // window start
        if w != self.window_start_ms {                      // new window
            self.window_start_ms = w;
            self.count = 0;
        }
        self.count += 1;
        self.count <= limit
    }
}

/// Sliding window counter: previous window, weighted by overlap (Section 3.7).
pub struct SlidingWindowCounter {
    window_ms: u64,
    prev_count: u64,
    curr_count: u64,
    curr_window_start_ms: u64,
}
```

```
impl SlidingWindowCounter {
    pub fn new(window_ms: u64) -> Self {
        Self { window_ms, prev_count: 0, curr_count: 0, curr_window_start_ms: 0 }
    }

    pub fn try_acquire(&mut self, now_ms: u64, limit: u64) -> bool {
        let w = now_ms / self.window_ms * self.window_ms;
        if w != self.curr_window_start_ms {
            self.prev_count = if w == self.curr_window_start_ms + self.window_ms {
                self.curr_count                          // adjacent window: carry over
            } else { 0 };                                // gap longer than a window
            self.curr_count = 0;
            self.curr_window_start_ms = w;
        }
        let elapsed = (now_ms - w) as f64 / self.window_ms as f64;
        let estimate = self.curr_count as f64
                     + self.prev_count as f64 * (1.0 - elapsed);
        if estimate + 1.0 <= limit as f64 {
            self.curr_count += 1;
            true
        } else {
            false
        }
    }
}
```

And the test that **demonstrates the flaw** — fixed window admits a 2× burst at the boundary, sliding counter rejects it:

```
#[cfg(test)]
mod window_tests {
    use super::*;

    #[test]
    fn fixed_window_admits_boundary_burst() {
        let mut fw = FixedWindow::new(60_000);        // 100 req/min
        // 100 requests at t = 59.5 s .. 59.9 s: all pass (window 0).
        for i in 0..100 {
            assert!(fw.try_acquire(59_500 + i, 100));
        }
        // 100 MORE at t = 60.0 s .. 60.4 s: all pass (window 1).
        // 200 requests in under a second — legal. That is the bug.
        for i in 0..100 {
            assert!(fw.try_acquire(60_000 + i * 4, 100));
        }
    }

    #[test]
    fn sliding_counter_rejects_the_same_burst() {
        let mut sw = SlidingWindowCounter::new(60_000);
        for i in 0..100 {
            assert!(sw.try_acquire(59_500 + i, 100));
        }
        // Just past the boundary the estimate is ~100 x overlap: the
        // previous window still counts almost fully. Burst denied.
        assert!(!sw.try_acquire(60_200, 100));
        assert!(!sw.try_acquire(61_000, 100));
        // Half a window later, weight has decayed: traffic flows again.
```

```
        let mut ok = 0;
        for i in 0..60 {
            if sw.try_acquire(90_000 + i * 500, 100) { ok += 1; }
        }
        assert!(ok > 40, "expected roughly half the limit to be available");
    }
}
```

3.13.3 The fleet-wide atomic check

Within one process, a `Mutex` makes read-modify-write atomic. Across fifty gateways, the atomicity must live **in the store**: the whole check ships as one server-side script that the store executes without interleaving. In production Rust this is a string constant handed to the client library — the script is data; the engineering is Rust:

```
/// Fleet-wide fixed-window check (Section 3.12, Fix 1). Executed atomically
/// by the store, so the interleaving of Section 3.6 is impossible.
/// KEYS[1] = "rl:{api_key}:{route}:{window_id}", ARGV[1] = window seconds.
const FIXED_WINDOW_LUA: &str = r#"
    local n = redis.call("INCR", KEYS[1])
    if n == 1 then redis.call("EXPIRE", KEYS[1], ARGV[1]) end
    return n
"#;

/// One gateway node's decision: exactly one round trip, and the *returned*
/// count is the check — no client-side read-modify-write at all.
pub async fn allowed(
    con: &mut redis::aio::MultiplexedConnection,
    api_key: &str,
    route: &str,
    limit: u64,
    window_secs: u64,
    now_secs: u64,
) -> redis::RedisResult<Decision> {
    let window_id = now_secs / window_secs;
    let key = format!("rl:{api_key}:{route}:{window_id}");
    let n: u64 = redis::Script::new(FIXED_WINDOW_LUA)
        .key(key)
        .arg(window_secs)
        .invoke_async(con)
        .await?;
    Ok(if n <= limit {
        Decision::Allow { remaining: limit - n }
    } else {
        let reset = (window_id + 1) * window_secs;
        Decision::Deny { retry_after_ms: (reset - now_secs) * 1000 }
    })
}
```

Note what the code makes structural: the key **embeds the window id**, so window rollover is just a new key (old windows expire by TTL — self-cleaning, Section 3.9); and the retry hint falls out of the window arithmetic for free.

3.13.4 Where the check sits

The middleware contract, sketched against a generic handler: rules come from the gateway-local cache (zero hops), the decision from the fleet store (one hop), and `Deny` maps to the 429 contract of Section 3.8.

```
```

```rust
/// Gateway middleware shape (Section 3.10). `handler` is the real API.
pub async fn handle(
    req: Request,
    rules: &RuleCache,            // gateway-local, push-refreshed
    store: &mut Store,            // fleet-wide limit state
) -> Response {
    let rule = rules.for_key_and_route(req.api_key(), req.route());
    match store.check(req.api_key(), req.route(), &rule).await {
        Ok(Decision::Allow { remaining }) => {
            let mut resp = handler(req).await;
            resp.headers_mut().insert("X-RateLimit-Limit", rule.limit.into());
            resp.headers_mut().insert("X-RateLimit-Remaining", remaining.into());
            resp
        }
        Ok(Decision::Deny { retry_after_ms }) => Response::too_many_requests(
            retry_after_ms,          // -> Retry-After + X-RateLimit-Reset
            &rule,
        ),
        Err(_) => fail_open(req).await, // Section 3.15: never take the API down
    }
}
```

3.14 Scaling & Sharding

Sharding (Chapter 1) is by API key — the natural key, since every limit check is scoped to exactly one:

- **Limit-state store**: keys hash-sharded across 4–6 shards with replicas (Section 3.5's arithmetic). Each check touches exactly one key, so there is no cross-shard coordination on the hot path.
- **Gateways**: stateless; any request can be checked anywhere, because the state lives in the shared store.
- **Rules**: fully replicated to every gateway — config is megabytes, and the read pattern (every request) demands local memory.

> **DEFINITION — Hot key**
>
> A single key whose traffic concentrates on one shard hard enough to matter — here, one viral API key doing tens of thousands of checks per second. A single atomic `INCR` or script call is O(1) and microseconds; a hot key is far more likely to be **rejected** fast than to melt a shard. If a key ever outgrows one shard, split its counter across sub-shards (`rl:key#0..15`) and sum at check time — the same trick Chapter 2's stream used per segment.

Multi-region note. A truly global limit across regions forces synchronous cross-region round trips on the hot path — hundreds of milliseconds, violating the latency NFR. Production answer: enforce per-region limits sized to divide the global one, and reconcile asynchronously (Chapter 1's PACELC framing: consistency would cost latency, so availability and latency win, and the **bounded overshoot** vocabulary of Section 3.4 is how you state the price).

3.15 Failure Modes & Recovery

> **DEFINITION — Fail-open / fail-closed**
>
> What a dependent system does when its dependency fails. *Fail-open* serves traffic anyway (availability over enforcement); *fail-closed* refuses (enforcement over availability). The choice is per-route: abuse protection on a public API fails open with a local cap; a login endpoint defending against brute force

fails **closed-ish** — refusing logins during a store outage is cheaper than admitting a credential-stuffing wave.

Failure	Handling
Limit-state store shard down	Fail over to the replica. If all replicas are gone: **fail open** with a per-gateway local token bucket sized `limit / fleet_size` — traffic flows, protection degrades to a bounded multiple of the limit, metrics scream (Section 3.17).
Store unreachable from some gateways (partition)	Those gateways degrade to local caps; the rest enforce exactly. Converges on heal, no manual action.
Clock skew between gateways	Structurally impossible to matter: bucket/window math runs on the **store's** clock via the server-side script (Section 3.12); local code uses monotonic time (Section 3.13).
Bad rule pushed	Rules are versioned; roll back to the previous version. Rate of change is small, so a human-in-the-loop push is fine.
Rule fan-out stalls	Gateways enforce last-known rules and alert; a stale rule for 60 s beats a missing check for 60 s.
Retry storm after a 429 wave	`Retry-After` spreads retries; add jitter client-side guidance in docs. A limiter that herds retries into the same second recreates the burst it just rejected.
Hot key	Sub-shard the counter and sum (Section 3.14). Rare in practice — hot keys are usually already over their limit.

3.16 Trade-offs & Alternatives

Decision	Benefit purchased	Cost accepted
Token bucket	Bursts as a product feature; O(1) state; lazy refill needs no timers	Two-number accounting must be atomic fleet-wide (script in the store)
Sliding window counter	No boundary burst, still O(1)	1% estimate error; slightly more math at check time
Fixed window alone	One atomic INCR; trivially explainable	2× boundary burst (Section 3.13's test proves it)
Centralized exact check	FR-4 truly holds; no false rejections	+1 RTT on every request; store on the critical path

Decision	Benefit purchased	Cost accepted
Local + async sync	Zero added latency; fail-open for free	Bounded overshoot; unusable for billing-adjacent limits
Fail-open default	The API never dies because the limiter did	During store outages, protection is a bounded approximation

3.17 Observability & SLOs

SLIs and SLOs (Chapter 1) for a limiter are unusual in one way: the system **working** looks like rejections. Instrument accordingly:

- **Decision rates** per rule: allows vs. 429s, over time. A 429 spike after a rule change is a config bug until proven otherwise.
- **Overshoot audit**: sampled per-key realized rates vs. limits — the direct measurement of Section 3.4's accuracy NFR.
- **Check latency** p50/p99, split by path (local rules lookup vs. store call); SLO: the ≤1 ms budget of Section 3.4.
- **Store health**: shard ops/sec, script latency, replica lag — the one dependency on the hot path.
- **Degraded-mode counters**: how many gateways are currently failing open, and with what local cap. This number should be zero; when it is not, paging is legitimate.
- **Top rejected keys**: the abuse list and the sales list are the same list — chronically limited keys are upgrade candidates (limits as monetization, Section 3.1).

3.18 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- *"Per-key AND per-IP AND per-route at once?"* — Composite rules: evaluate each applicable limit; deny if any denies. Order checks cheapest-first.
- *"Monthly quotas for billing?"* — Different system: append-only usage log, batch aggregation, delayed enforcement. Rate limits shape traffic; quotas shape invoices (Section 3.1).
- *"Limiter as a shared service vs. a library?"* — A library removes a hop but re-introduces per-language drift and per-instance state; a sidecar/service centralizes policy at the price of the hop. State which you picked and why.
- *"Exactly-distributed without a central store?"* — Gossiped counters or CRDTs (Chapter 1) give you **approximate** global counts with zero coordination; exactness without coordination is not a thing, and saying so is the answer.
- *"L3/L4 DDoS?"* — Out of scope by layer: SYN floods and volumetric attacks are absorbed at the CDN/edge (Chapter 2) with connection-level defenses. This design is L7, per-authenticated-caller.
- *"Adaptive limits under load?"* — Load-shedding's cousin: when the backend browns out, tighten limits dynamically by a global multiplier. Nice extension; say it, don't build it live.

If you remember five things:

1. A rate limiter is a read-modify-write on the hot path; the design is the art of making that operation atomic and ≤ 1 ms.
2. Local counters cannot express a global limit; a non-atomic shared counter races. Atomicity lives **in the store**.
3. Token bucket = sustained rate + burst allowance, with lazy refill and self-cleaning TTL state.
4. Accuracy is asymmetric: never false-reject, bound the overshoot — and know that exactness is a product decision, not a technical one.
5. The limiter must never be the outage: fail open with a local cap, and page on degraded mode.

3.19 Summary & Further Reading

We designed a distributed rate limiter for a 200k-RPS public API: a gateway middleware making exactly one atomic check per request against a sharded in-memory store; the algorithm zoo (fixed window, sliding log, sliding counter, token bucket, leaky/GCRA) with token bucket chosen for its burst semantics; the TOCTOU race and its three fixes (atomic increment-as-decision, server-side scripts, approximate local counting); the 429 + `Retry-After` contract that turns rejections into negotiations; runtime-reloadable tiered rules pushed to gateway memory; and a defended fail-open policy with bounded local caps.

Primary source for this chapter:

- **"7: Design a Rate Limiter" — Systems Design Interview Questions With Ex-Google SWE (Jordan has no life)** — the walkthrough this chapter expands.

Foundations worth reading:

- Stripe's engineering blog on rate limiters — production limiter design at API-company scale.
- Figma's *An alternative approach to rate limiting* — the leaky-bucket-in-Redis variant, honestly costed.
- The IETF draft *RateLimit header fields for HTTP* — the emerging standard response contract.
- Brandur Leach's writing on GCRA and the Redis-cell module — the stateless leaky bucket, implemented.
- NGINX's rate-limiting documentation — the leaky bucket hiding inside a commodity reverse proxy.

3.20 Chapter 3 Glossary

A one-glance index of every term this chapter defined. Chapters 1–2 are assumed; later chapters assume all three.

Term	Meaning in one line
Rate limiting / throttling	Capping requests per identity per window; protection, capacity, monetization
Quota	Long-period budget for billing; batched, approximate accounting
False rejection	Denying an under-limit caller — the unforgivable error
Overshoot	Admitting over-limit traffic — the bounded, tolerable error
Race condition / TOCTOU	Check and use separated in time; state changes in between
Atomicity / CAS	All-or-nothing operation / conditional swap as one step
Fixed window counter	One counter per window; O(1), but 2× boundary bursts
Sliding window log	Every timestamp kept; exact; memory O(limit) — infeasible at scale
Sliding window counter	Two windows weighted by overlap; O(1), 1% error, no burst
Token bucket	Capacity b refilled at r/s; bursts allowed; lazy refill
Leaky bucket / GCRA	Constant-rate drip; bursts forbidden; smooth output
HTTP 429 / Retry-After	"Too fast" status + how long to wait — rejection as negotiation
X-RateLimit-* headers	Limit, remaining, reset — client-side self-throttling data
API gateway middleware	First code every request meets; policy without backend changes
Server-side script (Lua)	Read-modify-write executed atomically inside the store

Term	Meaning in one line
Local + async sync	Zero-latency approximate enforcement with bounded overshoot
Monotonic clock	Time source that never moves backwards; no refunded tokens
Fail-open / fail-closed	Dependency down: serve anyway / refuse — chosen per route
Hot key	One key concentrating load on one shard; sub-shard and sum
Tiered limits	Different limits per plan; limits as pricing
Rule cache	Gateway-local copy of config; zero hops per request
Key TTL self-cleaning	Idle keys expire with their window; no garbage collector
Boundary burst	Two legal window-fulls fired at a window edge — 2× in seconds
Degraded-mode cap	Local emergency limit while the fleet store is down

CHAPTER 4

Designing a Logging & Metrics Platform

PROBLEM SOURCE

This chapter solves the problem posed in the talk **"14: Distributed Logging & Metrics Framework"** from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The talk designs the observability backbone for a microservices fleet: how telemetry leaves thousands of hosts, where it is buffered, how logs and metrics are stored and queried, and how alerts fire. This chapter follows the same arc, deepened with full definitions, capacity mathematics, query and retention design, and Rust reference implementations.

4.1 The Problem Statement

The interviewer looks up and says:

> *"We run thousands of services across tens of thousands of machines. When something breaks at 3 a.m., engineers need to see what happened — search every log, graph every metric, and get paged before users notice. Design the logging and metrics platform that makes that possible."*

The previous chapters designed systems that **serve users**. This one designs the system that watches all of them — including, recursively, the ones from Chapters 1–3. It is a data pipeline problem wearing an operations costume: an enormous, never-ending write stream on one side, and on the other a handful of humans asking very expensive questions at the worst possible moment (during an outage, when both data volume and query volume spike together).

DEFINITION — Observability / telemetry

The ability to answer questions about a system's internal state from its **outputs**: the data it emits as it runs. *Telemetry* is that emitted data. The canonical "three pillars" are **logs** (timestamped event records), **metrics** (numeric measurements over time), and **traces** (the path of one request through many services). This chapter designs the platform for the first two; traces are a follow-up (Section 4.18).

DEFINITION — Log vs. metric

A *log record* is a discrete, semi-structured event: "at 03:14:22.017, service `checkout` on host `i-9f2` logged `ERROR payment timeout user=…`" — high volume, low per-record value, queried by **searching text**. A *metric* is a named numeric measurement repeated over time: `http_requests_total{service="checkout", status="500"} = 87341` — small, structured, queried by **range scans and aggregations**. They look like cousins and demand opposite storage engines. Holding that distinction is the core of this chapter (Section 4.6).

4.2 Scope & Clarifying Questions

Speaker	Dialogue
Candidate	"What's in scope — logs, metrics, distributed tracing, all three?"
Interviewer	"Logs and metrics. Tracing is a future extension; don't design it, but don't preclude it."
Candidate	"Scale of the fleet being observed?"
Interviewer	"About 50,000 service instances across a few thousand hosts, in several regions."
Candidate	"How quickly must emitted data be queryable?"
Interviewer	"Logs within about a minute; metrics within about thirty seconds. Alerts within a minute of the condition."
Candidate	"How long do we keep data?"
Interviewer	"Logs: two weeks fast-searchable, three months retrievable. Metrics: thirteen months, and nobody needs per-15-second resolution on data from last year."
Candidate	"Is some data loss acceptable under extreme overload?"
Interviewer	"Metrics — yes, they're statistical; drop samples rather than fall over. Logs — avoid it; engineers notice gaps. State how you'd degrade."
Candidate	"Who queries, and how often?"
Interviewer	"A few thousand engineers. Dashboards refresh continuously; searches spike during incidents."

AGREED SCOPE

Logs + metrics for a **50k-instance** fleet. Logs: full-text search, queryable ≤ 1 min after emission, 2 weeks hot + 3 months cold. Metrics: label-filtered queries and dashboards, fresh ≤ 30 s, 13 months retained with downsampling. Alerting on both, notification ≤ 1 min from condition. Metrics may drop under extreme load; logs degrade by **sampling**, never silent gaps.

4.3 Functional Requirements

1. **FR-1 — Collection.** Every instance's logs and metrics flow to the platform automatically, within seconds of emission, with no per-service plumbing by engineers.
2. **FR-2 — Log search.** Engineers query logs by full text, structured fields, and time range; results over hot data return in seconds.
3. **FR-3 — Metric queries & dashboards.** Engineers query series by name and labels over time ranges, with aggregations (rate, sum, percentile), rendered as auto-refreshing dashboards.
4. **FR-4 — Alerting.** Rules over logs and metrics evaluate continuously; when a condition holds for its configured duration, notifications fire — once, not repeatedly (Section 4.12).
5. **FR-5 — Retention & tiering.** Data ages automatically through hot → warm → cold → deleted, per configured policy, with no manual intervention.
6. **FR-6 — Self-service config.** Teams register services, set retention, and define alerts through an API/UI — the platform team is not in the loop.

4.4 Non-Functional Requirements

DEFINITION — Ingestion latency (freshness, telemetry sense)

The time from an event being emitted on a host to it being **queryable**. Chapter 2's freshness measured data age; this one measures pipeline speed. Our targets: logs ≤ 60 s, metrics ≤ 30 s — fast enough that an engineer watching a deploy sees its effect while watching.

Quality	Target
Ingestion throughput	10M log events/s average, 3× that in bursts; 0.7M metric samples/s (Section 4.5)
Ingestion latency	Logs queryable ≤ 60 s; metrics ≤ 30 s
Query latency	Hot log search p95 ≤ 5 s; dashboard refresh ≤ 2 s
Durability	Logs: at-least-once, gaps are visible and rare. Metrics: loss-tolerant by nature (sampling tolerates gaps)
Availability	Ingestion path ≥ 99.9% — it is the fleet's black-box recorder; query path may degrade first
Cost discipline	Storage dominates; compression and tiering are first-class requirements, not optimizations

KEY INSIGHT — The system's worst day is everyone else's

When the fleet has an incident, two things happen at once: services emit **more** telemetry (error storms, stack traces) and engineers query **harder** (everyone opens dashboards). The platform must be sized for the correlated spike — the day it is needed most is the day it is loaded most. This observation drives the buffer-first architecture of Section 4.11.

4.5 Back-of-the-Envelope Estimation

Assumptions:

- 50k service instances; each emits 200 log lines/s average (quiet services less, chatty ones more), 200 B per line.
- Each instance exposes 200 metric series (name + label combinations); sampled every 15 s.
- Dashboards/searches: 2k engineers, spiking to 5k queries/s during incidents.

Derived numbers:

Quantity	Estimate	How
Log event rate	≈ 10M events/s avg, 30M burst	50k instances × 200 lines/s, ×3 incident factor
Log bandwidth	≈ 2 GB/s avg	10M × 200 B
Raw logs per day	≈ 170 TB	2 GB/s × 86,400 s
Hot log tier (14 days)	≈ 0.5–1 PB	sampling/filtering trims to 30–50 TB/day indexed; × 14 days, plus index overhead
Metric sample rate	≈ 0.7M samples/s	50k × 200 series ÷ 15 s

Quantity	Estimate	How
Metric storage per day	≈ 120 GB **compressed**	0.7M/s × 86,400 s × 2 B per compressed sample (Section 4.9)
Metrics, 13 months	tens of TB	after rollups; an order of magnitude less than logs' **daily** raw volume
Buffer capacity	≈ 2–5 GB/s sustained	a Kafka-class cluster of 20–30 brokers, 200 partitions

KEY INSIGHT — What the math tells us

Two facts shape everything. First, **logs are the cost center**: 170 TB/day raw versus metrics' 120 GB/day — three orders of magnitude apart. Storage design for logs is really **cost** design (sampling, tiering, compression); storage design for metrics is **query-shape** design (fast range scans over tiny values). Second, the write rate utterly dwarfs the query rate, so the system is organized as a pipeline: absorb first, index second, query third.

4.6 The Core Challenge: One Firehose, Two Shapes of Data

Logs and metrics arrive on the same wire from the same hosts — and must part ways immediately, because they are different kinds of data with different query patterns and different economics:

Property	Logs	Metrics
Record	Semi-structured text event	Name + labels + (timestamp, number)
Volume	170 TB/day raw	120 GB/day compressed
Per-record value	Low individually; priceless during one incident	Low individually; aggregates are the product
Query pattern	Full-text search, field filters, recent time ranges	Range scan by name/labels; aggregations over time
Index needed	Inverted index over tokens (Section 4.8)	Compressed time-ordered blocks (Section 4.9)
Loss tolerance	Low — gaps break investigations	High — statistics tolerate dropped samples

COMMON PITFALL — One store for both

The classic junior move: "just put it all in one database." Indexing metrics as log lines wastes the log pipeline's expensive text machinery on tiny numbers; storing logs in a time-series engine forfeits text search entirely. The chapter's architecture forks **right after the buffer** into two purpose-built stores — Section 4.8 and 4.9 exist because this fork exists.

4.7 Data Ingestion: Agents, Push vs. Pull, and the Buffer

DEFINITION — Telemetry agent

A small daemon running on every host (or beside every workload as a sidecar), owned by the platform team: it tails log files, collects or receives metrics, batches, compresses, and ships both upstream — buffering locally when upstream is unavailable. Agents are how collection (FR-1) happens "automatically": a service writes to stdout and exposes a metrics endpoint; the agent does the rest.

The one genuine design fork in collection is **who initiates the metrics transfer**:

DEFINITION — Push vs. pull (scrape)

Push: the source (or its agent) sends metrics upstream when it has them. *Pull*: the collector periodically **requests** metrics from each target's endpoint ("scrapes" `/metrics` every 15 s). Pull gives the collector control of sample timing, automatic **up/down detection** (a target that doesn't answer is itself a signal), and easy correctness (one place enforces the schedule). Push fits short-lived jobs that vanish before they can be scraped, and crossing trust boundaries outward. Production fleets commonly run both: pull for services, push gateways for batch jobs.

DEFINITION — Backpressure

What a pipeline does when a downstream stage is slower than upstream: the pressure must propagate backward and be **absorbed somewhere** — a queue grows, a producer is slowed, or data is deliberately shed. A pipeline without a backpressure plan converts every slow consumer into a cascading outage. Our answer: a durable buffer between agents and processors, plus explicit drop policies per data class (metrics shed first, logs sampled — Section 4.2's degradation agreement).

KEY INSIGHT — The buffer is the shock absorber

A Kafka-class log (Chapter 2) between agents and processors buys four things at once: spikes are absorbed (the incident-storm insight of Section 4.4); indexing can lag and replay after outages; a bad deploy of the indexing fleet costs **lag**, not data; and new consumers (the future tracing pipeline, a billing tap) attach without touching producers. Decoupling ingestion from indexing is the single highest-leverage decision in the chapter.

And a deliberate callback: the buffer's tenants are protected from each other with per-team **ingestion quotas** — Chapter 3's rate limiter, applied to telemetry. One team's debug-level flood must not evict everyone else's logs.

4.8 Log Storage: The Inverted Index

Log search answers "find the records containing these tokens, in this time range, with these field values, among billions of records" in seconds. A scan is impossible; the index must be inverted.

DEFINITION — Inverted index / posting list

The data structure behind every text search engine: a map from **token** to the sorted list of records containing it (the *posting list*). `"payment"` → [rec 17, rec 203, rec 8811, …]. A query like `payment AND timeout` intersects two posting lists — set intersection over sorted lists, not a scan of the data. Building the index is work done **at write time** so that reads stay fast: logs are a write-once, read-rarely, read-expensively workload, and the inverted index prices exactly that.

DEFINITION — Tokenization / structured logging

Tokenization splits raw text into searchable terms (lower-cased, split on punctuation). *Structured logging* emits logs as JSON records —

`{"level":"ERROR","service":"checkout","msg":"payment timeout",...}` — so fields are indexed **as fields** (`service:checkout`) rather than dug out of text at query time. The platform ingests both; structure makes queries faster and sampling smarter, and FR-6's self-service story nudges teams toward it.

Three storage decisions carry the load:

1. **Immutable segments, merged in the background.** Incoming records are buffered into small index **segments** written to disk immutably; a background process merges small segments into larger ones. Appends and merges are sequential I/O — the cheapest disk work there is (same lesson as Chapter 1's operation log).
2. **Sharding by time.** The index is partitioned into time buckets (e.g., one index per day per tenant class). Queries prune whole days by timestamp before touching a posting list — and **retention becomes deletion of an old index file**, not a billion per-record deletes. Time-sharding turns FR-5 into `rm`.
3. **Hot / warm / cold tiers.** Today's indices live on fast SSD nodes with replicas (hot); last week's on cheaper nodes (warm); months-old data as compressed archives in object storage (cold), rehydrated on the rare historical query. Cost per GB falls by an order of magnitude per tier — Section 4.5 made clear this **is** the design.

4.9 Metrics Storage: The Time-Series Database

DEFINITION — Time series / sample / label set

A *time series* is a named, labeled sequence of **(timestamp, value)** pairs: `http_requests_total{service="checkout", status="500"}` sampled every 15 s. A *sample* is one such pair. The *labels* (tags) are the dimensions queries filter and group by. The number of **distinct series** — the product of all label-value combinations — is the *cardinality*, and it is the metric system's one true nemesis.

COMMON PITFALL — Cardinality explosion

Labels must be **dimensions**, never **identifiers**. `status="500"` is a dimension (a handful of values); `user_id="u_91827"` is an identifier — millions of series, each seen once, index structures bloated beyond RAM, queries scanning garbage. Rule of thumb: a label whose value set grows with **traffic or users** does not belong on a metric; it belongs in a log or a trace. Every metrics interview eventually arrives here — arrive first.

Why a purpose-built store: samples of one series arrive in time order and change slowly, so they compress almost to nothing:

DEFINITION — Delta encoding / delta-of-delta

Store differences, not values. Timestamps sampled every 15 s delta-encode to a constant 15000 ms; the **delta-of-delta** (difference of consecutive deltas) is 0 almost always — compressible to a single bit per timestamp in the production scheme (Facebook's Gorilla, which also XOR-encodes values so only meaningful bits are stored). 16-byte samples become 1–2 bytes. Section 4.13 implements the delta+zigzag+varint core of the idea.

DEFINITION — Downsampling / rollup

> Replacing old high-resolution samples with precomputed aggregates — keep 15-second resolution for a week, 1-minute for a quarter, 1-hour beyond: `avg/min/max/sum/count` per bucket, computable incrementally. Nobody asks for 15-second data from last year (Section 4.2), and rollups turn 13 months of retention from a petabyte problem into a terabyte one.

Queries are then range scans over compressed blocks, sharded by metric-name hash: fetch the blocks for matching series in the range, decompress, aggregate. Dashboard panels are exactly this, cached and refreshed.

4.10 API & Query Design

Three surfaces — ingestion, query, config — each deliberately boring:

Method	Path	Purpose
`POST`	`/v1/logs:batch`	Agent push: compressed batch of log records with source metadata
`GET`	`/metrics` (on targets)	The scrape endpoint pull collectors call every 15 s
`POST`	`/v1/metrics:write`	Push path for short-lived jobs
`GET`	`/v1/logs/search?q=&from=&to=`	Log search: query string, time range, cursor-paginated
`POST`	`/v1/metrics/query`	Instant and range queries over series
`POST`	`/v1/alerts/rules`	Create/update alert rules (FR-4, FR-6); versioned like Chapter 3's rules

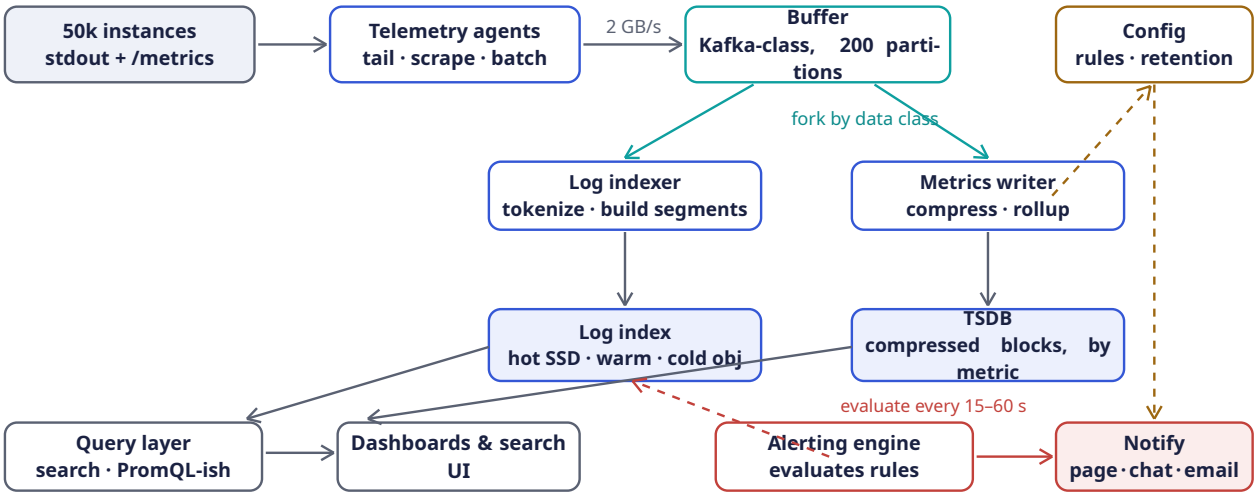
Query languages, sketched to show the shape (not to design in full):

```
-- logs: field filters + free text + time range
service:checkout AND level:ERROR AND "payment timeout"   @ last 1h

-- metrics: series selector + aggregation over a range
rate(http_requests_total{service="checkout", status=~"5.."}[5m])
  |> sum by (region)
```

Alert rules are records: `{ name, query, threshold, comparison, for_duration, severity, notify_channels }` — evaluated by the alerting engine of Section 4.12.

4.11 High-Level Architecture



One write path, two stores. The buffer absorbs incident storms; the query layer federates across both stores.

Component	Responsibility	Why it is shaped this way
Telemetry agents	Tail logs, scrape/collect metrics, batch + compress, ship	Per-host daemon = zero per-service work (FR-1); local buffer rides out upstream blips
Buffer (Kafka-class)	Durable, replayable queue between agents and processors	The shock absorber of Section 4.7: spikes, replays, new consumers
Log indexer	Parse, tokenize, build inverted-index segments	Stateless consumers of the log topic; scale with partition count
Log index cluster	Hot/warm/cold inverted indices, time-sharded	Search needs posting lists; retention needs cheap tiers (Section 4.8)
Metrics writer	Compress samples into blocks; maintain rollups	Stateless consumers of the metrics topic
TSDB cluster	Compressed time-series blocks; label index	Range scans over 2 B samples (Section 4.9)
Query layer	Federate queries across both stores; cache dashboard panels	One door for engineers; caches absorb dashboard refresh storms
Alerting engine	Evaluate rules continuously; notify without flapping	Section 4.12 — its correctness is the product's voice
Config service	Rules, retention, quotas; versioned, pushed	Chapter 3's lesson: config never fetched on the hot path

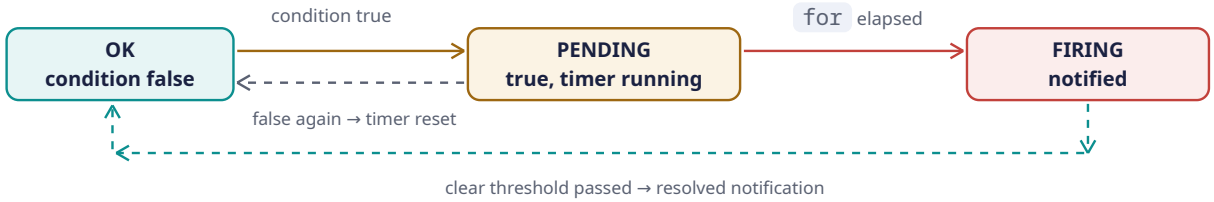
4.12 Deep Dive: The Alerting Engine

Alerting is where the metrics store earns its keep. The engine's loop is trivially stated — **every evaluation interval (15–60 s), run every rule's query, compare against the threshold** — and the difficulty is entirely in not crying wolf.

DEFINITION — Flapping / hysteresis / "for" duration

> *Flapping*: an alert that oscillates OK → FIRING → OK → FIRING as the metric jitters around the threshold, paging a human every oscillation until the rule is muted and the signal lost. The cures: a *"for" duration* — the condition must hold continuously for N minutes before firing (a PENDING state); and *hysteresis* — firing and clearing use **different** thresholds (fire at p99 > 500 ms for 5 min, clear only when p99 < 450 ms for 10 min), so a metric straddling one line cannot cross both. Alert fatigue is a **system failure mode**, and it is engineered against, exactly like throughput.

The per-rule state machine:



Three more engineering points:

- **Deduplication and grouping.** One dying dependency fires forty service alerts; the engine groups by root labels (`region`, `dependency`) into **one** page with the forty attached as context. Notifications are facts about **state transitions**, not about every evaluation.
- **Evaluation must be independent of ingestion health of the thing it watches.** If the alerting engine shares a failure domain with the metrics pipeline, the one outage that must page is the one that can't. It runs on separate infrastructure and — critically — can fire on **absence** of data (Section 4.15).
- **Self-service with guardrails (FR-6).** Teams write rules in config; the platform validates them (query parses, series cardinality bounded, threshold sane) and versions every change, exactly the Chapter 3 rule-lifecycle pattern.

4.13 Rust Reference Implementations

Four distilled pieces, each with a deterministic test. They are teaching skeletons of what runs at platform scale: a mini inverted index for log search, timestamp compression for the TSDB, incremental rollups, and the alert state machine.

4.13.1 A Mini Inverted Index for Log Search

Structured records are tokenized; each token maps to a sorted posting list of record ids. A two-term `AND` query intersects the lists in linear time.

```rust
use std::collections::HashMap;

/// One structured log record, as the indexer sees it after parsing.
#[derive(Debug, Clone)]
pub struct LogRecord {
    pub id: u64,
    pub timestamp_ms: u64,
    pub service: String,
    pub level: String,
    pub message: String,
}

/// Lower-case, split on anything that isn't alphanumeric:
/// "payment TIMEOUT after 3s" -> ["payment", "timeout", "after", "3s"].
fn tokenize(text: &str) -> Vec<String> {
    text.to_lowercase()
```

```
        .split(|c: char| !c.is_alphanumeric())
        .filter(|t| !t.is_empty())
        .map(str::to_string)
        .collect()
}

/// token -> sorted ids of records containing it ("posting list").
/// Also index the searchable fields under `field:value` tokens so that
/// `service:checkout` is just another posting-list lookup.
#[derive(Default)]
pub struct InvertedIndex {
    postings: HashMap<String, Vec<u64>>,
}

impl InvertedIndex {
    pub fn add(&mut self, rec: &LogRecord) {
        let mut tokens = tokenize(&rec.message);
        tokens.push(format!("service:{}", rec.service.to_lowercase()));
        tokens.push(format!("level:{}", rec.level.to_lowercase()));
        tokens.sort();
        tokens.dedup();
        for tok in tokens {
            let list = self.postings.entry(tok).or_default();
            // ids are appended in increasing order, so lists stay sorted;
            // a real engine would also store positions for phrase queries.
            list.push(rec.id);
        }
    }

    pub fn lookup(&self, token: &str) -> &[u64] {
        self.postings.get(token).map(Vec::as_slice).unwrap_or(&[])
    }

    /// Intersection of two sorted lists: the heart of `A AND B`.
    pub fn and(&self, a: &str, b: &str) -> Vec<u64> {
        let (xs, ys) = (self.lookup(a), self.lookup(b));
        let (mut i, mut j, mut out) = (0, 0, Vec::new());
        while i < xs.len() && j < ys.len() {
            match xs[i].cmp(&ys[j]) {
                std::cmp::Ordering::Less => i += 1,
                std::cmp::Ordering::Greater => j += 1,
                std::cmp::Ordering::Equal => {
                    out.push(xs[i]);
                    i += 1;
                    j += 1;
                }
            }
        }
        out
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn rec(id: u64, service: &str, level: &str, msg: &str) -> LogRecord {
        LogRecord { id, timestamp_ms: id, service: service.into(),
                    level: level.into(), message: msg.into() }
    }
```

```
    #[test]
    fn search_intersects_posting_lists() {
        let mut ix = InvertedIndex::default();
        ix.add(&rec(1, "checkout", "ERROR", "payment timeout after 3s"));
        ix.add(&rec(2, "checkout", "INFO",  "payment authorized"));
        ix.add(&rec(3, "search",   "ERROR", "timeout talking to index"));
        ix.add(&rec(4, "checkout", "ERROR", "payment timeout after 5s"));

        // free text
        assert_eq!(ix.lookup("timeout"), &[1, 3, 4]);
        // field query
        assert_eq!(ix.lookup("service:checkout"), &[1, 2, 4]);
        // the incident query: payment AND timeout, checkout only
        assert_eq!(ix.and("payment", "timeout"), &[1, 4]);
        assert_eq!(
            ix.and("service:checkout", "level:error")
                .into_iter()
                .collect::<Vec<_>>(),
            vec![1, 4]
        );
    }
}
```

Production engines add positions (for phrase queries), compression of posting lists, skip pointers for faster intersection, and segment files instead of a `HashMap` — but the **algorithm** the interviewer wants to hear is the sorted intersection above, unchanged since the 1960s.

4.13.2 Delta + Zigzag + Varint: Compressing Timestamps

The honest core of Gorilla-style compression: store the first timestamp, then **delta-of-deltas**; encode each as a zigzag varint so small numbers — including small **negative** corrections — cost one byte.

```
/// Zigzag maps signed -> unsigned so small magnitudes (either sign)
/// become small unsigned ints: 0->0, -1->1, 1->2, -2->3, 2->4, ...
fn zigzag(n: i64) -> u64 {
    ((n << 1) ^ (n >> 63)) as u64
}
fn unzigzag(z: u64) -> i64 {
    ((z >> 1) as i64) ^ -((z & 1) as i64)
}

/// Unsigned varint: 7 bits per byte, high bit = "more bytes follow".
fn write_varint(mut z: u64, out: &mut Vec<u8>) {
    loop {
        let byte = (z & 0x7f) as u8;
        z >>= 7;
        if z == 0 { out.push(byte); break; }
        out.push(byte | 0x80);
    }
}
fn read_varint(bytes: &[u8], pos: &mut usize) -> u64 {
    let (mut z, mut shift) = (0u64, 0);
    loop {
        let byte = bytes[*pos];
        *pos += 1;
        z |= ((byte & 0x7f) as u64) << shift;
        if byte & 0x80 == 0 { return z; }
        shift += 7;
    }
```

```
}

/// Compress a sorted series of millisecond timestamps:
/// first value verbatim, then delta-of-delta as zigzag varints.
pub fn compress_timestamps(ts: &[u64]) -> Vec<u8> {
    assert!(!ts.is_empty());
    let mut out = Vec::new();
    write_varint(ts[0], &mut out);
    let (mut prev_ts, mut prev_delta) = (ts[0] as i64, 0i64);
    for w in ts.windows(2) {
        let delta = w[1] as i64 - prev_ts;
        let dod = delta - prev_delta;                 // 0 for a steady 15s scrape
        write_varint(zigzag(dod), &mut out);
        prev_ts = w[1] as i64;
        prev_delta = delta;
    }
    out
}

pub fn decompress_timestamps(bytes: &[u8], count: usize) -> Vec<u64> {
    let mut pos = 0;
    let first = read_varint(bytes, &mut pos) as i64;
    let mut out = vec![first as u64];
    let (mut prev_ts, mut prev_delta) = (first, 0i64);
    for _ in 1..count {
        let dod = unzigzag(read_varint(bytes, &mut pos));
        let delta = prev_delta + dod;
        prev_ts += delta;
        prev_delta = delta;
        out.push(prev_ts as u64);
    }
    out
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn zigzag_roundtrip_and_small_magnitudes() {
        for n in [-2, -1, 0, 1, 2, 15000, -15000] {
            assert_eq!(unzigzag(zigzag(n)), n);
        }
        assert_eq!(zigzag(-1), 1);
        assert_eq!(zigzag(0), 0);
    }

    #[test]
    fn steady_15s_scrape_compresses_to_about_a_byte_per_point() {
        let base = 1_700_000_000_000u64;           // some epoch-ms
        let ts: Vec<u64> = (0..1000).map(|i| base + i * 15_000).collect();
        let packed = compress_timestamps(&ts);
        assert_eq!(decompress_timestamps(&packed, ts.len()), ts);
        // 1000 x 8-byte timestamps -> ~1007 bytes: ~1 byte per point.
        assert!(packed.len() < 1100, "got {} bytes", packed.len());
    }

    #[test]
    fn jitter_and_gaps_survive_roundtrip() {
        // scrape jitter (+/- 3ms) plus one 30s gap where a sample was lost
```

```
        let mut ts = vec![1_000u64, 16_003, 31_002, 61_001, 75_997];
        let packed = compress_timestamps(&ts);
        assert_eq!(decompress_timestamps(&packed, ts.len()), ts);
        ts.clear(); // prove the bytes alone reconstruct everything
        assert!(ts.is_empty());
    }
}
```

Gorilla additionally XORs consecutive **values** and stores only meaningful bit-runs — the same philosophy: at scale, the difference between 16 bytes and 1.4 bytes per sample is the difference between a petabyte and a rack.

4.13.3 Incremental Rollups for Retention

A rollup bucket carries just enough state to be mergeable: `sum`, `min`, `max`, `count`. Raw samples stream in; the bucket answers the five aggregates without remembering any sample.

```
/// One downsampled bucket: mergeable in O(1) without raw samples.
/// This is why rollups scale: merging two 1-hour buckets is
/// adding sums, taking min/max, adding counts.
#[derive(Debug, Clone, Copy, PartialEq)]
pub struct Bucket {
    pub sum: f64,
    pub min: f64,
    pub max: f64,
    pub count: u64,
}

impl Bucket {
    pub fn empty() -> Self {
        Bucket { sum: 0.0, min: f64::INFINITY, max: f64::NEG_INFINITY, count: 0 }
    }
    pub fn add(&mut self, value: f64) {
        self.sum += value;
        self.min = self.min.min(value);
        self.max = self.max.max(value);
        self.count += 1;
    }
    /// Merge another bucket's aggregates in O(1) — no raw samples needed.
    /// This associativity is exactly why rollups scale: two hour-buckets
    /// combine into a day-bucket with four cheap operations.
    pub fn merge(&mut self, other: &Bucket) {
        if other.count == 0 { return; }
        self.sum += other.sum;
        self.min = self.min.min(other.min);
        self.max = self.max.max(other.max);
        self.count += other.count;
    }
    pub fn avg(&self) -> Option<f64> {
        (self.count > 0).then(|| self.sum / self.count as f64)
    }
}

/// Assign a timestamp (ms) to its bucket of `width_ms`, then fold in.
pub fn rollup(samples: &[(u64, f64)], width_ms: u64)
    -> std::collections::BTreeMap<u64, Bucket>
{
    let mut buckets = std::collections::BTreeMap::new();
    for &(ts, v) in samples {
        buckets.entry(ts / width_ms).or_insert_with(Bucket::empty).add(v);
```

```
    }
    buckets
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn rollup_buckets_match_hand_computation() {
        // 15s samples across two 1-minute buckets; base aligned to a
        // minute boundary so the first four samples share bucket 0.
        let t = 1_700_000_040_000u64; // == 28_333_334 * 60_000
        let samples: Vec<(u64, f64)> = (0..8)
            .map(|i| (t + i * 15_000, (100 + i * 10) as f64))
            .collect();
        let m = rollup(&samples, 60_000);
        let keys: Vec<u64> = m.keys().copied().collect();
        assert_eq!(keys.len(), 2);
        let b0 = m[&keys[0]];
        assert_eq!(b0.count, 4);                        // t, t+15s, t+30s, t+45s
        assert_eq!(b0.sum, 100.0 + 110.0 + 120.0 + 130.0);
        assert_eq!(b0.min, 100.0);
        assert_eq!(b0.max, 130.0);
        assert_eq!(b0.avg(), Some(115.0));
    }

    #[test]
    fn buckets_merge_without_raw_samples() {
        let mut a = Bucket::empty();
        for v in [10.0, 20.0, 30.0] { a.add(v); }
        let mut b = Bucket::empty();
        for v in [5.0, 50.0] { b.add(v); }
        a.merge(&b);
        assert_eq!(a.count, 5);
        assert_eq!(a.sum, 115.0);
        assert_eq!(a.min, 5.0);
        assert_eq!(a.max, 50.0);
        assert_eq!(a.avg(), Some(23.0));
    }
}
```

4.13.4 The Alert Evaluator: OK → PENDING → FIRING

The state machine of Section 4.12, made executable. The test demonstrates the two failure modes it prevents: a single bad spike must **not** page (no `for` violation), and a sustained breach must page **once**, not on every evaluation.

```
/// States of Section 4.12's machine. `pending_since` is Some(ms) while
/// the condition has been continuously true but `for_ms` hasn't elapsed.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum AlertState {
    Ok,
    Pending { since_ms: u64 },
    Firing,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Notification {
    Fired,
```

```
    Resolved,
}

pub struct AlertRule {
    pub threshold: f64,
    pub for_ms: u64,
}

pub struct AlertEvaluator {
    pub rule: AlertRule,
    state: AlertState,
}

impl AlertEvaluator {
    pub fn new(rule: AlertRule) -> Self {
        AlertEvaluator { rule, state: AlertState::Ok }
    }

    /// Feed one evaluation (one query result) at time `now_ms`.
    /// Returns a notification only on *transitions*.
    pub fn evaluate(&mut self, breached: bool, now_ms: u64) -> Option<Notification> {
        use AlertState::*;
        match (self.state, breached) {
            (Ok, true) => {
                self.state = Pending { since_ms: now_ms };
                None
            }
            (Pending { since_ms }, true) => {
                if now_ms - since_ms >= self.rule.for_ms {
                    self.state = Firing;
                    Some(Notification::Fired)          // page once
                } else {
                    None                               // still waiting
                }
            }
            (Firing, true) => None,                    // already paged; stay quiet
            (Firing, false) => {
                self.state = Ok;
                Some(Notification::Resolved)
            }
            (Pending { .. }, false) => {
                self.state = Ok;                       // spike ended: reset timer
                None
            }
            (Ok, false) => None,
        }
    }

    pub fn is_breach(&self, value: f64) -> bool {
        value > self.rule.threshold
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn evaluator() -> AlertEvaluator {
        AlertEvaluator::new(AlertRule { threshold: 500.0, for_ms: 300_000 })
    }
```

```
    #[test]
    fn single_spike_never_pages() {
        let mut ev = evaluator();
        // p99 jumps over threshold for one 15s evaluation, then recovers
        assert_eq!(ev.evaluate(true, 0), None);        // -> Pending
        assert_eq!(ev.evaluate(false, 15_000), None);  // recovered: reset
        assert_eq!(ev.state, AlertState::Ok);
    }

    #[test]
    fn sustained_breach_pages_once_then_resolves() {
        let mut ev = evaluator();
        let mut t = 0u64;
        let step = 60_000;                             // evaluate every minute
        let mut notes = Vec::new();
        // breach for 6 consecutive minutes, `for` = 5 minutes
        for _ in 0..6 {
            if let Some(n) = ev.evaluate(true, t) { notes.push((t, n)); }
            t += step;
        }
        // fired exactly once, when the timer elapsed (t = 5 min)
        assert_eq!(notes, vec![(300_000, Notification::Fired)]);
        // keep breaching: silence
        assert_eq!(ev.evaluate(true, t), None);
        // recovery -> one resolved notification
        assert_eq!(ev.evaluate(false, t + step), Some(Notification::Resolved));
        assert_eq!(ev.state, AlertState::Ok);
    }

    #[test]
    fn threshold_boundary_is_not_a_breach() {
        let ev = evaluator();
        assert!(!ev.is_breach(500.0));                 // strictly greater
        assert!(ev.is_breach(500.1));
    }
}
```

4.14 Scaling the Platform

Layer	Scale axis	Mechanism
Agents	Fleet size (50k+)	Embarrassingly parallel: one per host; sizing is a per-host CPU/RAM budget, not a cluster problem
Buffer	Ingestion GB/s	Partitions (200) spread over 20–30 brokers; keyed by tenant+service so one team's flood is contained and quota'd (Chapter 3 callback)
Log indexers	Events/s	Stateless consumers, one per partition lane; autoscale on consumer lag
Log index	Data volume + query QPS	Time-sharded indices spread over nodes; hot tier replicated; incident

Layer	Scale axis	Mechanism
		query storms absorbed by the query-layer cache
Metrics writers	Samples/s	Stateless consumers; 0.7M samples/s is small — headroom is free
TSDB	Series cardinality × retention	Shard by metric-name hash; rollups shrink old data 100× before it becomes a storage problem
Alerting engine	Rule count × eval rate	Partition rules across evaluators by hash; evaluations are independent, so this scales linearly

KEY INSIGHT — The incident storm is the sizing case

Average load (2 GB/s) is not the design point; **everyone's worst day at once** is. An outage multiplies error logs 3–5×, and two thousand engineers open dashboards simultaneously. The buffer absorbs the write spike; the query cache absorbs the read spike; per-tenant quotas keep one service's panic from evicting the logs everyone else needs to debug it. A telemetry platform sized for the average is a platform that dies exactly when it is needed.

4.15 Failure Modes & Degradation

Failure	Symptom	Response
Telemetry agent dies	A host goes silent	Agents buffer locally, so restarts lose nothing; and silence itself is a signal — a heartbeat-absence alert fires (the alerting engine evaluates **absence**, not just thresholds)
Buffer fills / brokers down	Agents' local buffers grow	Local disk spooling buys hours; on recovery, consumers replay the backlog. Durability is the buffer's whole job
Log indexer lags or crashes	Search results fall behind ingest	Lag is visible as a metric on the buffer (meta-observability, Section 4.17); stateless indexers are replaced and replay from last offset
Index node lost	A shard of one day's index unavailable	Replicas on the hot tier serve queries; the shard is rebuilt from the buffer while it is still retained
TSDB node lost	Gap in dashboards	Replicas; and because metrics are statistics, a brief gap de-

Failure	Symptom	Response
		grades a chart without lying to an alert — `for` durations span it
Alerting engine down	The silent killer	It runs in an **independent failure domain** (separate infra, separate credentials), and a meta-monitor outside the platform watches its heartbeat — Section 4.17
Poison record crashes a parser	One consumer stuck in a crash loop	Dead-letter queue: N failed attempts → the record is parked aside, processing continues, an operator is notified
Storage budget blown	Hot tier at capacity	Enforced sampling on the noisiest tenants (never silent gaps — Section 4.2), accelerated warm/cold migration, quota renegotiation

COMMON PITFALL — Monitoring the monitor

The most dangerous failure of an observability platform is **looking healthy while being blind**: dashboards render cached panels, no alerts fire, and meanwhile ingestion silently stopped. Every layer must therefore emit telemetry about itself into an **independent** pipeline — if the platform's own metrics flow through the platform, its failure erases its own death certificate. Section 4.17 formalizes this as meta-observability.

4.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Metrics collection	Pull (scrape) for services + push gateway for batch jobs	Pure push: loses scrape-time up/down detection and centralized scheduling; pure pull: can't catch short-lived jobs
Log write path	Index at write time (inverted index)	Schema-on-read / index-at-query (cheap writes, brutal query scans) — fine for archive-only data; our FR-2 demands interactive search
Log retention	Hot/warm/cold tiers with time-sharded indices	Uniform hot storage: correct but 10× the cost for data that is queried <1% of the time
Metric resolution	Fixed 15 s + rollup tiers	High-res forever: cardinality × retention makes storage explode; variable/adaptive reso-

Decision	Chosen	Alternative & why not (here)
		lution: complex, inconsistent dashboards
Alert condition	Threshold + `for` duration + clear hysteresis	Fire on every breach: flapping burns trust; anomaly detection everywhere: powerful for **exploration**, too opaque as the paging contract
Buffer	Durable log (Kafka-class) between agents and processors	Direct agent→indexer RPC: no spike absorption, no replay, couples every producer to every consumer's outages
Degradation under overload	Shed metrics first, sample logs, never silently	Hard block (429 to agents): telemetry backs up into **application** hosts — the observer becomes the outage

4.17 SLOs & Meta-Observability

The platform that measures everyone else's SLOs needs its own — measured from **outside**, by a small independent "meta-monitor" in a separate failure domain:

Indicator	Definition	Target
Ingestion availability	Share of batches accepted by the buffer	≥ 99.9%
Log searchability delay	Event timestamp → present in index, p95	≤ 60 s
Metric freshness	Scrape timestamp → queryable, p95	≤ 30 s
Query latency (hot)	Log search / dashboard render, p95	≤ 5 s / ≤ 3 s
Alert latency	Condition true → notification sent, p95	≤ 1 min + `for`
Data loss	Acked bytes later found missing, per month	≈ 0 (buffer replay)
Pipeline health coverage	Hosts whose agent heartbeat is watched	100%

> **INTERVIEW TIP — State the SLO of the telemetry system in the interview**
>
> Candidates design observability for everyone else and forget the platform itself. Naming meta-observability — "who watches the watcher, and from which failure domain" — is a senior-level signal: it shows you have been paged by the monitoring system being down, not just by the systems it monitors.

4.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Add distributed tracing — what changes?"** A third pillar with its own shape: traces are **trees** (spans with parent ids), stored per-trace-id with heavy head-based or tail-based sampling (1–5% head, or tail-sampling that keeps errors and slow outliers). Trace ids are **correlation keys**: log records carry `trace_id` so a log search pivots to the trace, and exemplars link metric spikes to example traces. The platform's buffer and tiering lessons transfer directly.

2. **"How do you cut the log bill in half?"** Rank tenants by bytes; enforce structured logging; sample repetitive INFO lines at the agent (log 1-in-N, keep all WARN+); shorten hot retention for the noisiest services; negotiate quotas. The answer is organizational as much as technical.
3. **"PII ends up in logs — now what?"** Scrubbing is cheapest at the agent (pattern rules before anything leaves the host), enforced again at the indexer; access to raw logs is itself logged; cold-tier encryption keys rotate. Treat telemetry as production data with a blast radius.
4. **"A team creates a label with user ids in it — what breaks?"** Cardinality explosion (Section 4.9): the TSDB's series index balloons, queries slow globally, and you are now storing identifiers you may not be allowed to store. Guardrails: reject writes whose label sets exceed per-metric cardinality budgets, and alert **the platform team** on the growth itself.
5. **"Why not just use a vendor solution?"** Buy-versus-build: the architecture above is exactly what the vendors run internally; the interview question is whether you can reason about their trade-offs. At 170 TB/day the build case is real; at 170 GB/day it usually is not.

4.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> A logging-and-metrics platform is **one firehose, two shapes of data**. Telemetry agents collect from every host (pull for services, push for batch jobs); a durable buffer absorbs the incident storm and decouples ingestion from processing; then the data forks: logs into a time-sharded **inverted index** (tokenization, posting lists, immutable segments, hot/warm/cold tiers that make retention a file deletion), metrics into a **time-series store** (delta-of-delta compression, label cardinality as the nemesis, mergeable rollups). Alerting is a per-rule state machine — OK → PENDING → FIRING — that pages on transitions, never on repetitions, and runs in its own failure domain. The platform watches itself from outside. The sizing insight: design for everyone else's worst day, because that is precisely when this system becomes the most important one in the building.

Further reading.

- The source video: *"14: Distributed Logging & Metrics Framework — Systems Design Interview Questions With Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=p_q-n09B8KA`
- Pelkonen, Franklin, Teller, Cavallaro, Huang, Meza, Veeraraghavan — *"Gorilla: A Fast, Scalable, In-Memory Time Series Database"* (VLDB 2015) — the delta-of-delta / XOR compression scheme Section 4.9 distills.
- The Prometheus documentation — the pull model, the label model, and the `for` / alerting-rule semantics this chapter borrows.
- McCandless, Hatcher, Gospodnetić — *Lucene in Action* — inverted indices, segments, and merges in production depth.
- *Google SRE Book*, chapters "Monitoring Distributed Systems" and "Alerting on SLOs" — symptoms vs. causes, and the philosophy behind Section 4.12.

4.20 Chapter Glossary

Term	Meaning
Alert fatigue	The learned ignoring of alerts caused by excessive false or repeated pages; a system failure mode, engineered against with `for` durations and hysteresis
Backpressure	Propagation of slowness upstream through a pipeline, absorbed by queues, throttling, or deliberate shedding

Term	Meaning
Cardinality	The number of distinct time series a metric's label combinations produce; the metric platform's primary scaling constraint
Cold / warm / hot tier	Storage classes of decreasing cost and speed holding progressively older telemetry; retention is implemented by migrating and deleting time-sharded indices
Dead-letter queue (DLQ)	A side channel where records that repeatedly fail processing are parked so the pipeline proceeds
Delta-of-delta encoding	Compressing a timestamp series by storing the difference of consecutive deltas — zero for a steady scrape, near-zero with jitter
Downsampling / rollup	Replacing old high-resolution samples with precomputed mergeable aggregates (sum/min/max/count) per bucket
Failure domain	The set of components that can fail together; the alerting engine and meta-monitor must live outside the domain they watch
Flapping	An alert oscillating between firing and clearing as a metric jitters around its threshold
Hysteresis	Using different thresholds to enter and leave a state, preventing oscillation at a single boundary
Inverted index	Map from token to the sorted list of records containing it; the data structure behind text search
Label	A key=value dimension on a metric series; dimensions, never identifiers
Meta-observability	Telemetry about the telemetry platform itself, emitted into an independent pipeline so the platform's death cannot erase its own certificate
Observability / telemetry	The practice (and data) of inferring a system's internal state from its outputs; pillars: logs, metrics, traces
Posting list	The sorted list of record ids attached to one token in an inverted index; queries intersect posting lists
Pull (scrape) model	Metrics collection where the collector periodically requests samples from each target's endpoint, gaining up/down detection for free
Push model	Metrics collection where sources send samples upstream; suits short-lived jobs and outbound trust boundaries
Sample	One (timestamp, value) pair of a time series
Sampling (logs)	Keeping a deterministic 1-in-N subset of a repetitive log class; degrades volume without creating silent gaps
Schema-on-read	Indexing little at write time and interpreting data at query time; cheap writes, expensive queries
Segment (index)	An immutable shard of an inverted index; small segments are merged in the background

Term	Meaning
Structured logging	Emitting logs as fielded records (JSON) rather than free text, enabling field queries and smart sampling
Telemetry agent	The per-host daemon that tails logs, collects metrics, batches, buffers, and ships telemetry upstream
Time series	A named, labeled sequence of timestamped values; the atom of the metrics pillar
Tokenization	Splitting text into searchable terms at index time
Zigzag encoding	A signed-to-unsigned integer mapping that makes small magnitudes of either sign varint-cheap

— End of Chapter 4 · Next: Chapter 5—

CHAPTER 5

Designing a Hierarchical Comment System

> **NOTE — Problem source**
>
> This chapter solves the problem posed in the talk *"15: Reddit Comments"* from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the commenting system of a Reddit-scale discussion site — nested reply threads, up/down voting, and the several ways users can sort a thread. The interesting parts are the **data shape** (a forest of unbounded trees) and the **ranking mathematics** (votes plus time decay plus statistical confidence), not raw storage volume — a deliberate contrast with Chapter 4's firehose. All terms are defined before use; all reference code is Rust with deterministic tests.

5.1 The Problem Statement

The interviewer draws a post with a familiar indented conversation underneath it and says:

"Users comment on posts and reply to each other, arbitrarily deep. They upvote and downvote comments. A thread can be sorted by 'best', 'top', 'new', or 'controversial'. Popular posts get tens of thousands of comments, so users page through them and expand subtrees on demand. Design the system that stores, ranks, and serves these comment threads."

The previous chapters designed infrastructure; this one designs a **user-facing product surface** whose entire value is ordering: the same fifty thousand comments are useful or useless depending on which fifteen the reader sees first. Two hard problems hide inside — representing arbitrarily deep trees so that subtrees can be fetched and paged cheaply, and ranking votes in a way that is **statistically honest** and **time-aware** at write rates of thousands of votes per second.

> **DEFINITION — Comment thread / tree**
>
> The replies under one post form a *tree*: the post is the root context, each comment has exactly one parent (the post or another comment), and replies fan out to arbitrary depth and breadth. The set of trees under all posts is a *forest*. "The thread" usually means one post's whole tree, or a rendered window into it.

> **DEFINITION — Score / vote tally**
>
> A comment's *score* is `upvotes - downvotes`, the single number users see. Votes serve two masters: they produce the visible score, and they feed the **ranking functions** (Section 5.8) that decide display order — and those two consumers want different things from the same data, which is where the design gets interesting.

5.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
How deep can nesting go?	Unbounded in principle; the UI collapses past 10 levels with a "continue this thread" link
Edit and delete?	Edits allowed briefly (mark "edited"); deletes are soft — the comment becomes a tombstone but its replies survive
Are votes final?	No — users can change or retract votes; one vote per user per comment, enforced
Real-time?	New comments should appear within seconds on refresh; no live push needed. Counts can lag a little
Moderation?	Assume a separate moderation pipeline flags/removes; we must honor removals fast. Don't design moderation itself
Sort orders?	best, top, new, controversial — "best" is the default and the tricky one
Scale?	Reddit-scale: 70M daily users, viral posts with 10^5 comments
Anonymous reading?	Yes — most reads are logged-out; that population is cache-friendly
Media in comments?	Text plus links only; media uploads are the Maps chapter's CDN problem, already solved

NOTE — Agreed scope

1. **Post and reply** to arbitrary depth; soft-delete with surviving replies.
2. **Vote** up/down/retract; exactly one vote per user per comment.
3. **Sort** a thread by best / top / new / controversial, with sound mathematics behind each.
4. **Page** through large threads: windowed top-level listing, expandable subtrees, collapsed deep chains.
5. **Counts**: post-level comment counts and comment scores may be seconds stale, never lost.
6. Out: live push updates, moderation tooling, rich media, recommendations.

5.3 Functional Requirements

DEFINITION — Ranking mode

One of the supported orderings of a thread's comments. *Top*: highest score. *New*: most recent first. *Best*: highest **statistical confidence** of being liked (Section 5.8's Wilson score — not the same as top!). *Controversial*: most balanced **and** voluminous disagreement. Each mode is a different pure function of the same vote data — an important honesty property.

1. **FR-1 — Create.** Post a top-level comment on a post, or a reply to any existing comment, recording parent, author, timestamp, and body.
2. **FR-2 — Vote.** Cast, change, or retract one up/down vote per user per comment; the score reflects the change within seconds.
3. **FR-3 — Read a thread window.** Given a post, a ranking mode, and a cursor, return the next page of the ranked tree — top-level comments each with the first few levels of their best children.
4. **FR-4 — Expand a subtree.** Given any comment, return the next window of its children ("more replies", "continue this thread").

5. **FR-5 — Soft-delete and removal.** Deleted comments render as a tombstone that preserves thread structure; moderator removals disappear entirely, quickly.
6. **FR-6 — Post-level counters.** Comment counts per post for listing pages, eventually consistent within seconds.

5.4 Non-Functional Requirements

DEFINITION — Read-to-write ratio

The proportion of read requests to write requests a system serves. It dictates architecture: high ratios justify aggressive caching and denormalization (precomputing for reads at write time), because every write is amortized across thousands of reads. Comment systems are extreme — Section 5.5 derives roughly **35:1** on requests, and far higher on bytes.

Requirement	Target & reasoning
Thread read latency	p95 ≤ 200 ms for a cached window, ≤ 700 ms uncached; this is the product's main surface
Write latency	p95 ≤ 300 ms for comment and vote writes; users tolerate a beat before their own comment appears
Availability	Reads ≥ 99.99% (stale cache is acceptable degradation); writes ≥ 99.9%
Durability	A confirmed comment or vote is never lost — it is social content, not telemetry (contrast Chapter 4's metrics)
Vote integrity	One-vote-per-user enforced exactly; tallies reconcile to the true vote log eventually
Consistency	Read-your-own-write for the author; seconds of staleness acceptable for everyone else
Ranking freshness	A vote visibly affects ordering within 1 minute on hot threads; scores tick visibly on refresh
Cost	Storage is cheap here (11 TB/year); spend engineering on read shape and ranking, not capacity

KEY INSIGHT — This is a shape problem, not a size problem

Chapter 4 drowned in bytes (170 TB/day); this chapter stores 30 GB/day — three orders of magnitude less. The difficulty is that the data is a forest of unbounded trees that must be **ranked four ways, paged stably, and re-ranked continuously** as votes arrive. Interviewers use this problem to test data modeling and algorithmic reasoning, not capacity arithmetic.

5.5 Back-of-the-Envelope Estimation

Assumptions (stated, then derived from): 70M daily active users; 20% read comment threads, averaging 10 thread views; 1 in 35 thread readers writes a comment; every comment author and 10 lurkers vote on comments per reader; average comment body 500 bytes.

Quantity	Estimate	Derivation
Thread views	700M/day ≈ 8k/s avg, 40k/s peak	14M readers × 10 views; peak ×5 for a viral moment
Comments written	20M/day ≈ 230/s avg, 1.2k/s peak	20M posts' worth of discussion; 1-in-35 readers comment
Votes cast	200M/day ≈ 2.3k/s avg, 12k/s peak	10 votes per active voter; the write-heavy path
Comment storage	30 GB/day → 11 TB/year	20M × 1.5 KB with ids, indexes, overhead
Vote records	40 GB/day raw, compacted small	200M × 50 B (user, comment, value, time); idempotency needs them
Hot cache	50 GB	Top 1M threads' first ranked windows × 50 KB
Read bandwidth	800 MB/s at avg load	8k views/s × 100 KB rendered window

KEY INSIGHT — Votes, not comments, are the write storm

Comments arrive at 230/s; **votes** at 2.3k/s average and 12k/s peak — and each vote is a read-modify-write on **at least four** things: the voter's idempotency record, the comment's up/down tallies, the cached score, and eventually the ranked order. Naively, that is a hot-row update per vote on viral comments. The vote pipeline (Section 5.11) exists to make the busiest path in the system also the cheapest.

The pipeline of the rest of the chapter: Section 5.6 isolates the two hard problems; 5.7 models the trees; 5.8 does the ranking mathematics; 5.9–5.11 design APIs, architecture, and the vote pipeline; 5.12 serves threads; 5.13 implements all of it in Rust; 5.14–5.20 scale, harden, and review.

5.6 The Core Challenge: Trees and Honest Ordering

Two problems carry this design, and neither is capacity.

Problem one — represent a forest of unbounded trees so that windows of it are cheap to read. Every read asks a deceptively simple question: "give me top-level comments 41–60 by best, each with its first two levels of best children." A relational row per comment answers that badly unless the schema is designed for it; Section 5.7 compares the three classical tree encodings and picks a hybrid.

Problem two — rank honestly. "Best" cannot mean "highest score": a comment at +1 (1 up, 0 down) would outrank none and a comment at +190 (200 up, 10 down) deserves to beat one at +2 (2 up, 0 down) — raw differences ignore **sample size**. Ranking must be a statistical statement about votes, plus a time statement (freshness), and it must be recomputed continuously as votes stream in. Section 5.8 derives the three functions used in production practice.

COMMON PITFALL — Score is not rank

Sorting "best" by `up - down` is the most common wrong answer in this interview. It ranks a 1-up-0-down comment above a 200-up-10-down one (+1 vs +190 is fine, but +2 vs +190 is not — and +1, 0 down beats +190 never, yet **percentage** sorting fails oppositely: 1/1 = 100% beats 200/210 = 95%). Both

naive orders are statistically illiterate; the fix is a confidence bound (Section 5.8). Say this early in the interview — it is the question the interviewer is waiting for.

5.7 Data Model: Encoding Comment Trees

DEFINITION — Adjacency list / materialized path / nested sets

The three classical ways to store trees in a relational store. *Adjacency list*: each row stores `parent_id` — trivial writes, but fetching a subtree needs recursive queries. *Materialized path*: each row stores its whole ancestor chain (`/a1/b7/c3`), so a subtree is one `WHERE path LIKE '/a1/b7/%'` prefix scan and depth is the path length — at the cost of longer keys and painful subtree **moves**. *Nested sets*: each row stores `lft` / `rgt` bounds from a depth-first numbering, making subtrees a range scan but every **insert** renumbers half the tree — catastrophic at comment write rates.

Property	Adjacency list	Materialized path	Nested sets
Insert a reply	One row	One row (path = parent's + own id)	One row + mass renumbering
Fetch a subtree	Recursive	One prefix/range scan	One range scan
Fetch top-level page	Indexed `parent_id IS NULL` scan	`depth = 1` or path prefix	Range union
Depth of a comment	Unknown without walking	Path segment count	Stored or derived
Move a subtree	One update	Rewrite all descendant paths	Renumber the tree
Fit for comments	Good bones	**Best fit** — subtrees are read, rarely moved	Unacceptable writes

The chosen schema — adjacency bones, path muscle, denormalized tallies:

```
comments
  comment_id      bigint pk
  post_id         bigint          -- shard key (Section 5.14)
  parent_id       bigint null     -- adjacency: direct parent
  path            text            -- materialized: /c1/c7/c42 (segment per ancestor)
  depth           smallint        -- = segments in path; drives collapse rules
  author_id       bigint
  body            text            -- null when tombstoned
  created_utc     timestamp
  up / down       bigint          -- denormalized tallies (vote pipeline owns them)
  state           enum            -- live | tombstone | removed
  index (post_id, path)           -- serves every window query in Section 5.12
```

`votes` is a separate table keyed `(user_id, comment_id)` — the idempotency backbone of Section 5.11. A post's `comment_count` lives denormalized on the `posts` row, maintained asynchronously (FR-6).

DEFINITION — Tombstone (soft delete)

A deletion that erases content but keeps structure: body and author are blanked, `state` becomes `tombstone`, and the row **stays** so its replies still have a parent. Rendering shows "[deleted]". Hard deletion would orphan entire subtrees; moderation removals (`removed`) are the exception — they may take the subtree with them per policy.

5.8 Deep Dive: The Ranking Mathematics

Three pure functions over the same two integers (`up`, `down`) plus time. Each answers a different question honestly.

5.8.1 "Top" and "New" — trivial orders

`top` sorts by score `up - down` descending (ties by age, newest first); `new` sorts by `created_utc` descending. Both are indexable directly. Their weakness is statistical, which is exactly why "best" exists.

5.8.2 "Best" — the Wilson lower confidence bound

DEFINITION — Wilson score interval (lower bound)

Treat each comment's votes as `n = up + down` Bernoulli trials estimating the **true probability** `p` that a random viewer upvotes it. A confidence interval around the observed ratio `p̂ = up/n` narrows as `n` grows; the *lower bound* of the Wilson interval (at z = 1.96, ≈95%) is a pessimistic estimate of `p` that fairly compares comments with **different sample sizes**. Small-sample comments are penalized toward 0; as `n` → ∞ the bound approaches `p̂`. It needs only `up` and `down`, is monotonic in both, and costs a few flops — computable at vote time and stored.

$$\text{wilson}(u,d) = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p}) + \frac{z^2}{4n}}{n}}}{1 + \frac{z^2}{n}}, \quad n = u + d, \hat{p} = u/n$$

The numbers that make the point: a comment with 1 up / 0 down scores ≈ 0.21; with 10 / 0, ≈ 0.72; with 100 / 0, ≈ 0.96. And 100 up / 100 down (p̂ = 50% but n = 200, ≈ 0.42) correctly beats 1 / 0 — volume of evidence matters more than a lone perfect record. Section 5.13 implements and tests exactly these.

5.8.3 "Controversial" — balanced disagreement, weighted by volume

$$\text{controversial}(u,d) = \begin{cases} 0 & \text{if } \min(u,d) = 0 \\ (u+d)^{\min(u,d)/\max(u,d)} & \text{otherwise} \end{cases}$$

A 50/50 split on 100 votes (`balance` = 1, magnitude 100) crushes a 105-vote comment with a 100/5 split (balance = 0.05 → magnitude ≈ 1.26): near-even splits dominate, and among equal splits the **louder** argument wins.

5.8.4 "Hot" — score with time decay (for posts and fast threads)

DEFINITION — Time-decay ranking ("hot")

Reddit's post ranking: `"hot" = log_10 max(|s|, 1) + "sign"(s) · (t - t_0) / 45000`, with `s = up - down`, `t` the comment's epoch seconds, and `t_0` an arbitrary fixed epoch. The log makes the first 10 votes worth as much as the next 100 (diminishing returns); dividing age by 45,000 s (12.5 h) means a post must be **10× more popular** to tie one 12.5 hours newer. Fresh content cycles through the front page; score advantages decay gracefully instead of cliffing.

KEY INSIGHT — Rank is a stored, recomputed column — not a query-time sort

All four functions are pure over `(up, down, created_utc)`. So the vote pipeline recomputes a comment's rank keys **on every vote** and stores them; reads are then index scans, never sorts. "Best" order changes only when votes change — and votes arrive at 2.3k/s, exactly the pipeline built in Section 5.11. Sorting 50k comments per page-view would be malpractice.

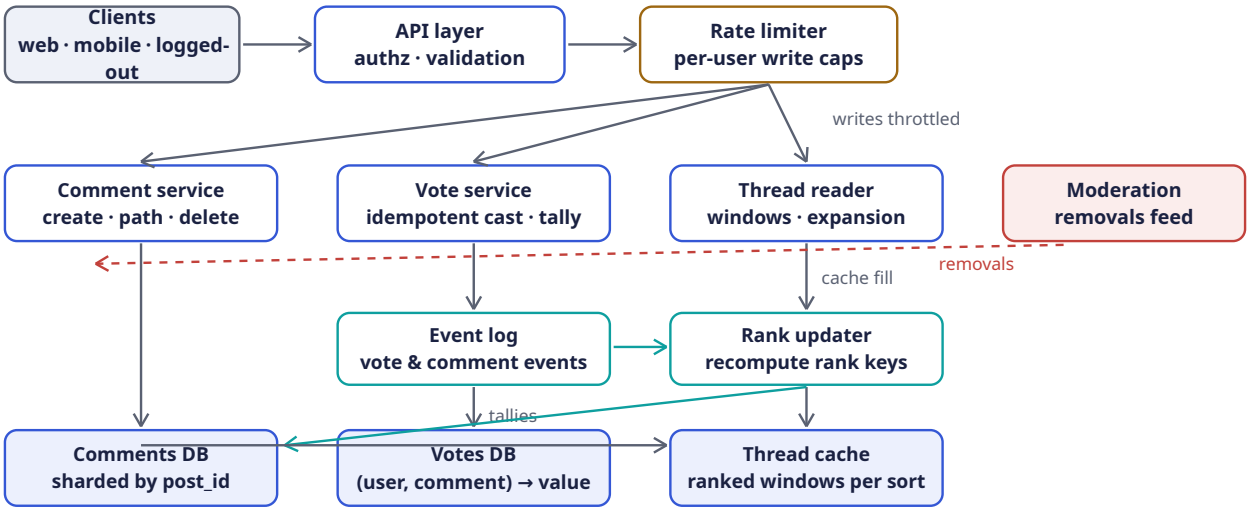
5.9 API Design

Method	Path	Purpose
`POST`	`/v1/posts/{id}/comments`	Top-level comment (FR-1); body, author
`POST`	`/v1/comments/{id}/replies`	Reply to a comment (FR-1); server computes path, depth
`PUT`	`/v1/comments/{id}/vote`	Cast/change/retract vote: `{value: 1|-1|0}` — idempotent (FR-2)
`GET`	`/v1/posts/{id}/comments?sort=&cursor=&limit=`	Ranked thread window with shallow children (FR-3)
`GET`	`/v1/comments/{id}/children?cursor=&limit=`	Expand a subtree window (FR-4)
`DELETE`	`/v1/comments/{id}`	Soft-delete → tombstone (FR-5); author-only
`GET`	`/v1/posts/{id}/comments/count`	Post-level counter (FR-6); cacheable, seconds-stale

Two deliberate choices:

- **Opaque cursors, never offsets.** A cursor encodes the last seen rank key and comment id (`base64(wilson, id)`); offsets break the moment a new comment inserts above the reader's position — Chapter 1's cursor lesson applied to ranked lists. Ties are impossible: `(rank_key, comment_id)` is a total order.
- **Votes are `PUT`, not `POST`.** Repeating or flipping a vote is one idempotent operation on a `(user, comment)` resource, not an event stream the client can double-fire. Retries are free.

5.10 High-Level Architecture



Component	Responsibility	Why it is shaped this way
API layer	Auth, validation, routing	Stateless; scales horizontally behind a load balancer
Rate limiter	Per-user caps on comment & vote writes	Chapter 3 verbatim: comments/hour, votes/minute — the first anti-brigading wall
Comment service	Create replies (computing `path` / `depth`), edits, tombstones	Owns the tree invariants; the only writer of `comments` rows
Vote service	Idempotent cast/change/retract; owns `votes` table	One writer per vote record; the idempotency enforcer (Section 5.11)
Event log	Durable stream of vote & comment events	Decouples tally/rank recomputation from the write path — Chapter 4's buffer lesson
Rank updater	Recompute tallies, Wilson, hot, controversial; write rank keys; bump post counters	Turns 2.3k votes/s into batched, coalesced rank updates
Comments DB	Tree rows, sharded by `post_id`	One post's whole thread lives on one shard — window queries stay single-shard
Votes DB	`(user, comment) → value`	Sharded by `comment_id`; the ground truth for reconciliation
Thread cache	Pre-rendered ranked windows per post × sort	35:1 read ratio + logged-out majority = the tier that actually serves traffic
Moderation feed	Removals pushed to comment service	Honored in seconds (agreed scope); separate pipeline, not designed here

5.11 Deep Dive: The Vote Pipeline

The busiest path in the system deserves the most engineering. Requirements: one vote per user (exactly), changeable, never lost, score visible in seconds, rank keys refreshed continuously.

> **DEFINITION — Idempotent write / natural idempotency key**
>
> A write that can be applied any number of times with the same effect. Retries, double-clicks, and client bugs make **at-least-once** delivery the honest assumption, so the handler must deduplicate. Here the key is free: `(user_id, comment_id)` is the natural primary key of a vote — one row per pair, upserted, is the deduplication mechanism itself. No tokens, no expiry windows (contrast Chapter 3's idempotency keys on payment APIs).

The write path for `PUT /v1/comments/c42/vote {value: -1}`:

1. **Rate-limit** the user (Chapter 3: sliding window, votes/minute).
2. **Upsert** `votes[(u, c42)] = -1` in one statement returning the previous value — the atomic read-modify-write that Chapter 3's TOCTOU section warns about; the unique key makes it safe:

```
-- one atomic statement; all clauses share one snapshot, so `old`
-- reads the pre-upsert value
WITH old AS (
  SELECT value FROM votes WHERE user_id = $1 AND comment_id = $2
)
INSERT INTO votes (user_id, comment_id, value, updated_utc)
VALUES ($1, $2, $3, now())
ON CONFLICT (user_id, comment_id)
DO UPDATE SET value = $3, updated_utc = now()
RETURNING (SELECT value FROM old) AS prev_value;
```

3. **Emit** a `VoteChanged{user, comment, prev, new}` event to the log. The request returns here — the write path is **two** durable operations.
4. **Rank updater** consumes events, **coalesces** bursts per comment (200 votes in a second become one recompute), recomputes tallies `up += (new==1) - (prev==1)`, `down += (new==-1) - (prev==-1)`, re-derives Wilson/hot/controversial, writes them back, and **version-stamps** the thread cache so the next read repopulates.

> **KEY INSIGHT — Coalescing is the whole trick**
>
> A viral comment can take thousands of votes per minute, but its **rank key** only needs recomputing a few times a minute — readers cannot perceive faster change, and sorting is stable between recomputes. Treating votes as a stream to be compacted (latest-wins per user; batch-sum per comment) turns the hottest write path into a trickle of rank updates. The score users see may lag the truth by seconds — the agreed scope allows exactly that, and spends the savings on reads.

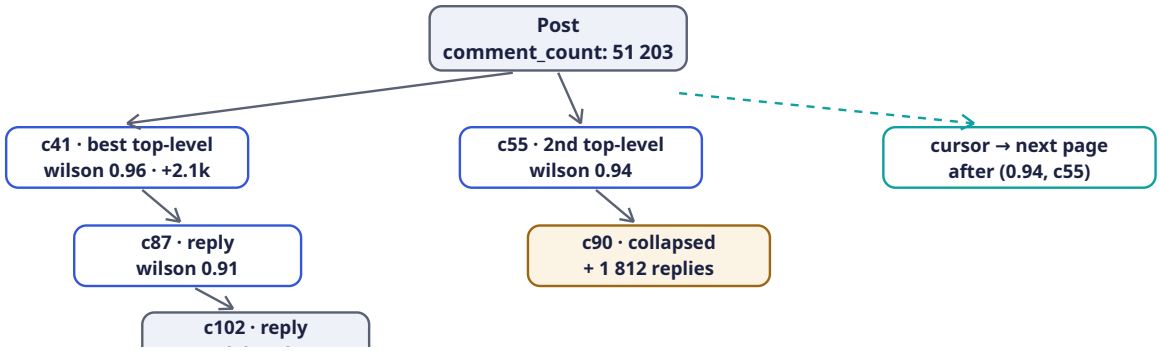
Reconciliation. Tallies are derived state; `votes` is ground truth. A nightly job (and a per-comment repair tool) recomputes tallies from `votes` and fixes drift — the same discipline as Chapter 4's buffer replays: **derived numbers must be rebuildable from the event record.**

5.12 Deep Dive: Reading a Thread Window

A thread read assembles a **window** of the tree: a page of top-level comments, each carrying its first levels of best children, with everything past the depth cap collapsed behind an affordance. With the schema of Section 5.7 the whole window is two queries:

1. **Top-level page**: `WHERE post_id = $1 AND depth = 1 ORDER BY rank_key DESC, comment_id DESC LIMIT $2` with the cursor supplying the rank-key boundary — a single index range scan on `(post_id, rank)` per sort mode.
2. **Children fill**: for the ≤20 parents in the page, one batched prefix scan `WHERE post_id = $1 AND path ~ ANY ('/c41/%', '/c87/%', ...) AND depth <= 4 ORDER BY path` — materialized paths turn "first two levels of every parent on the page" into **one** query instead of N recursive ones (the N+1 that sinks the naive adjacency-list design).

Assembly is in memory: parents in rank order, children nested under them by path prefix, tombstones rendered as "[deleted]" placeholders, subtrees beyond depth 10 replaced by a `{continue_thread: comment_id}` token the client expands through `/children` (FR-4).



Caching. The rendered window JSON is cached under `(post_id, sort, cursor)` for logged-out readers — the majority (Section 5.2) — and the rank updater **version-stamps** the post on every coalesced update, so stale windows age out within seconds without any invalidation fanout. Logged-in readers get the same window plus a per-user vote overlay (`SELECT comment_id, value FROM votes WHERE user_id = $1 AND comment_id = ANY(...)`) merged at read time — personalization over a shared cache, instead of a cache per user.

> **INTERVIEW TIP — Window, don't tree**
>
> The API never returns "the thread" — only windows: ranked top-level pages, shallow children, explicit expansions. A 50k-comment thread is 50k rows the user will never scroll; serving the **shape** of the conversation (best subthreads first, depth capped) is both the product-correct and the systems-correct answer. Candidates who serialize whole trees lose the room.

5.13 Rust Reference Implementations

Four pieces with deterministic tests: the ranking trio, the tree model with windowing, the idempotent vote service, and cursor pagination.

5.13.1 The Ranking Functions

```rust
/// z = 1.96 -> ~95% confidence. The lower Wilson bound of the true
/// upvote probability, from (up, down) alone. Pessimistic for small n.
pub fn wilson_lower(up: u64, down: u64) -> f64 {
    let n = (up + down) as f64;
    if n == 0.0 { return 0.0; }
    let z: f64 = 1.96;
    let phat = up as f64 / n;
    let denom = 1.0 + z * z / n;
```

```
    let centre = phat + z * z / (2.0 * n);
    let margin = z * ((phat * (1.0 - phat) + z * z / (4.0 * n)) / n).sqrt();
    (centre - margin) / denom
}

/// Reddit's "hot": log score + age bonus. 45_000s = 12.5h; a comment must
/// be 10x more popular to tie one 12.5h newer. t0 is a fixed epoch.
pub fn hot(up: u64, down: u64, epoch_secs: i64) -> f64 {
    const T0: i64 = 1_134_028_003; // Reddit's original launch epoch
    let s = up as i64 - down as i64;
    let order = s.unsigned_abs().max(1) as f64;
    let order = order.log10();
    let sign = (s > 0) as i8 as f64 - (s < 0) as i8 as f64;
    order + sign * (epoch_secs - T0) as f64 / 45_000.0
}

/// Balanced disagreement weighted by volume: (u+d)^(min/max).
pub fn controversial(up: u64, down: u64) -> f64 {
    let (lo, hi) = (up.min(down), up.max(down));
    if lo == 0 { return 0.0; }
    ((up + down) as f64).powf(lo as f64 / hi as f64)
}

#[cfg(test)]
mod ranking_tests {
    use super::*;

    #[test]
    fn wilson_grows_with_sample_size_at_same_ratio() {
        // all 100% approval, increasing n: 0.21 -> 0.72 -> 0.96
        let (w1, w10, w100) =
            (wilson_lower(1, 0), wilson_lower(10, 0), wilson_lower(100, 0));
        assert!((w1 - 0.2065).abs() < 1e-3, "w1 = {w1}");
        assert!((w10 - 0.7225).abs() < 1e-3, "w10 = {w10}");
        assert!((w100 - 0.9630).abs() < 1e-3, "w100 = {w100}");
        assert!(w1 < w10 && w10 < w100);
    }

    #[test]
    fn wilson_prefers_proven_mediocre_to_lucky_perfect() {
        // 100/100 (phat 50%, n 200) beats 1/0: evidence over luck.
        let proven = wilson_lower(100, 100);
        assert!((proven - 0.4307).abs() < 1e-3, "{proven}");
        assert!(proven > wilson_lower(1, 0));
        assert!(wilson_lower(0, 0) == 0.0);
        assert!(wilson_lower(0, 5) < wilson_lower(1, 5));
    }

    #[test]
    fn hot_decay_needs_10x_score_per_12h30m() {
        let t = 1_700_000_000;
        let older = hot(100, 0, t);
        let newer = hot(10, 0, t + 45_000);
        assert!((older - newer).abs() < 1e-9);
        assert!(hot(0, 10, t) < hot(0, 1, t)); // downvotes push below zero-side
    }

    #[test]
    fn controversial_needs_both_balance_and_volume() {
        let even_split = controversial(50, 50);      // 100^1.0
```

```
        let loud_blowout = controversial(100, 5);    // 105^0.05
        assert!((even_split - 100.0).abs() < 1e-9);
        assert!((loud_blowout - 1.2619).abs() < 1e-3, "{loud_blowout}");
        assert!(even_split > loud_blowout);
        assert_eq!(controversial(0, 100), 0.0);      // no disagreement
    }
}
```

5.13.2 The Tree Model: Paths, Depth, Tombstones

```
use std::collections::HashMap;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum State { Live, Tombstone }

#[derive(Debug, Clone)]
pub struct Comment {
    pub id: u64,
    pub parent_id: Option<u64>,
    pub created_ms: u64,
    pub up: u64,
    pub down: u64,
    pub state: State,
    pub body: String,
}

impl Comment {
    pub fn score(&self) -> i64 { self.up as i64 - self.down as i64 }

    /// Materialized path segment: parent path + own id.
    pub fn path(&self, parent_path: &str) -> String {
        format!("{}/{}", parent_path, self.id) // "" -> "/1" -> "/1/2" -> ...
    }

    pub fn render_body(&self) -> &str {
        match self.state {
            State::Live => &self.body,
            State::Tombstone => "[deleted]",
        }
    }
}

/// Depth-first flatten of a thread for rendering: parents before children,
/// siblings ranked by score (stand-in for "best" at read time). Tombstones
/// keep their position so replies are not orphaned.
pub fn flatten_thread(rows: &[Comment]) -> Vec<(usize, &Comment)> {
    let mut children: HashMap<Option<u64>, Vec<&Comment>> = HashMap::new();
    for c in rows {
        children.entry(c.parent_id).or_default().push(c);
    }
    for kids in children.values_mut() {
        kids.sort_by(|a, b| {
            b.score()
                .cmp(&a.score())
                .then(b.created_ms.cmp(&a.created_ms)) // newer wins ties
                .then(a.id.cmp(&b.id))                 // total order: no ambiguity
        });
    }
    let mut out = Vec::with_capacity(rows.len());
```

```
    let mut stack: Vec<(usize, &Comment)> = Vec::new();
    if let Some(roots) = children.get(&None) {
        for c in roots.iter().rev() {
            stack.push((1, *c)); // top-level comments have depth 1
        }
    }
    while let Some((depth, c)) = stack.pop() {
        out.push((depth, c));
        if let Some(kids) = children.get(&Some(c.id)) {
            for k in kids.iter().rev() {
                stack.push((depth + 1, *k));
            }
        }
    }
    out
}

#[cfg(test)]
mod tree_tests {
    use super::*;

    fn c(id: u64, parent: Option<u64>, up: u64, state: State) -> Comment {
        Comment { id, parent_id: parent, created_ms: id * 1000,
                  up, down: 0, state, body: format!("body {id}") }
    }

    #[test]
    fn window_is_dfs_with_ranked_siblings() {
        let rows = vec![
            c(1, None, 10, State::Live),
            c(2, Some(1), 5, State::Live),
            c(3, Some(1), 8, State::Live),
            c(4, Some(2), 1, State::Live),
            c(5, None, 20, State::Tombstone),
            c(6, Some(5), 3, State::Live),
        ];
        let flat = flatten_thread(&rows);
        let ids: Vec<u64> = flat.iter().map(|(_, c)| c.id).collect();
        // roots by score: 5 (20) then 1 (10); children of 1: 3 (8) then 2 (5)
        assert_eq!(ids, vec![5, 6, 1, 3, 2, 4]);
        // depth tracks nesting
        assert_eq!(flat.iter().find(|(_, c)| c.id == 4).unwrap().0, 3);
        assert_eq!(flat.iter().find(|(_, c)| c.id == 1).unwrap().0, 1);
    }

    #[test]
    fn tombstone_hides_body_but_keeps_subtree() {
        let rows = vec![c(5, None, 20, State::Tombstone),
                        c(6, Some(5), 3, State::Live)];
        let flat = flatten_thread(&rows);
        assert_eq!(flat[0].1.render_body(), "[deleted]");
        assert_eq!(flat[1].1.render_body(), "body 6");
    }

    #[test]
    fn materialized_paths_compose() {
        let (c1, c2, c4) = (c(1, None, 0, State::Live),
                            c(2, Some(1), 0, State::Live),
                            c(4, Some(2), 0, State::Live));
        let p1 = c1.path("");
```

```
        let p2 = c2.path(&p1);
        assert_eq!(c4.path(&p2), "/1/2/4");
        assert_eq!(p2.matches('/').count(), 2); // segment count == depth
    }
}
```

5.13.3 The Idempotent Vote Service

```
use std::collections::HashMap;

/// Natural idempotency key: (user, comment). One row per pair; upserts.
#[derive(Default)]
pub struct VoteService {
    votes: HashMap<(u64, u64), i8>, // value in {-1, 0, +1}; 0 == retracted
}

impl VoteService {
    /// Cast / change / retract a vote. Returns (delta_up, delta_down) for
    /// the rank updater to apply to the comment's tallies — the whole
    /// effect of the vote in two integers.
    pub fn cast(&mut self, user: u64, comment: u64, value: i8) -> (i64, i64) {
        assert!((-1..=1).contains(&value));
        let prev = self.votes.insert((user, comment), value).unwrap_or(0);
        let delta = |old: i8, new: i8, side: i8| {
            ((new == side) as i64) - ((old == side) as i64)
        };
        (delta(prev, value, 1), delta(prev, value, -1))
    }
}

#[cfg(test)]
mod vote_tests {
    use super::*;

    #[test]
    fn repeat_and_flip_and_retract() {
        let mut svc = VoteService::default();
        assert_eq!(svc.cast(7, 42, 1), (1, 0));   // new upvote
        assert_eq!(svc.cast(7, 42, 1), (0, 0));   // retry: no double count
        assert_eq!(svc.cast(7, 42, -1), (-1, 1)); // flip: score swings by 2
        assert_eq!(svc.cast(7, 42, 0), (0, -1));  // retract
        assert_eq!(svc.cast(8, 42, 1), (1, 0));   // another user unaffected
    }

    #[test]
    fn folding_deltas_reproduces_true_tallies() {
        let mut svc = VoteService::default();
        let deltas = [svc.cast(7, 42, 1), svc.cast(7, 42, -1),
                      svc.cast(8, 42, 1)];
        let (up, down) = deltas
            .iter()
            .fold((0i64, 0i64), |(u, d), &(du, dd)| (u + du, d + dd));
        assert_eq!((up, down), (1, 1)); // 7's flip + 8's upvote
    }
}
```

5.13.4 Cursor Pagination over a Ranked List

```rust
/// Map an f64 to bits whose unsigned order matches the float's order —
/// so rank keys can live in opaque cursors and database indexes.
pub fn ordered_bits(f: f64) -> u64 {
    let b = f.to_bits();
    if b & (1 << 63) == 0 { b | (1 << 63) } else { !b }
}

/// Total order per comment: (rank desc, id desc). No ties, ever.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub struct RankKey {
    pub wilson: u64, // ordered_bits(wilson_lower(up, down))
    pub id: u64,
}

pub type Cursor = RankKey; // the client treats it as opaque base64

/// Next page strictly after `after` from a list sorted descending.
pub fn page_after(
    sorted_desc: &[RankKey],
    after: Option<Cursor>,
    limit: usize,
) -> (Vec<RankKey>, Option<Cursor>) {
    let start = match after {
        None => 0,
        Some(c) => sorted_desc
            .iter()
            .position(|&k| k == c)
            .map(|i| i + 1)
            .unwrap_or(sorted_desc.len()),
    };
    let page: Vec<RankKey> =
        sorted_desc.iter().skip(start).take(limit).copied().collect();
    let next = match page.last() {
        Some(&last) if start + page.len() < sorted_desc.len() => Some(last),
        _ => None,
    };
    (page, next)
}

#[cfg(test)]
mod page_tests {
    use super::*;
    use crate::wilson_lower; // from the ranking listing (same crate)

    fn key(up: u64, down: u64, id: u64) -> RankKey {
        RankKey { wilson: ordered_bits(wilson_lower(up, down)), id }
    }

    #[test]
    fn ordered_bits_preserves_float_order() {
        assert!(ordered_bits(0.72) > ordered_bits(0.21));
        assert!(ordered_bits(-1.0) < ordered_bits(0.0));
    }

    #[test]
    fn pages_cover_everything_once_and_are_stable() {
        let mut keys = vec![
            key(100, 0, 1), // 0.963
            key(10, 0, 2),  // 0.722
            key(10, 0, 3),  // 0.722 — tie with id 2, broken by id desc
```

```
            key(1, 0, 4),   // 0.207
            key(0, 1, 5),   // 0.0
        ];
        keys.sort_by(|a, b| b.cmp(a)); // (wilson desc, id desc)

        let (p1, c1) = page_after(&keys, None, 2);
        let (p2, c2) = page_after(&keys, c1, 2);
        let (p3, c3) = page_after(&keys, c2, 2);
        let ids: Vec<u64> =
            [p1.as_slice(), &p2, &p3].concat().iter().map(|k| k.id).collect();
        assert_eq!(ids, vec![1, 3, 2, 4, 5]); // no skips, no duplicates
        assert_eq!(c1.unwrap().id, 3);        // cursor = last item of page
        assert!(c3.is_none());                // exhausted
    }
}
```

5.14 Scaling the Platform

Layer	Scale axis	Mechanism
API / services	Request rate	Stateless; horizontal scaling behind load balancers
Comments DB	Posts × comments	Shard by `post_id` — a whole thread on one shard, so every window query is single-shard; viral posts are one **hot** shard, handled by cache, not resharding
Votes DB	Users × votes	Shard by `comment_id` — tallies per comment stay local; lookup by user is the rare path (vote overlay), served by a secondary index
Event log / rank updater	Vote rate 2.3k/s avg	Partitioned by `comment_id`; coalescing collapses bursts — consumer count tracks partitions, not votes
Thread cache	Concurrent readers	Window JSON is immutable between version stamps → any number of replicas; logged-out traffic is fully cacheable
Read bandwidth	800 MB/s avg	Edge/CDN for logged-out windows; the origin only serves logged-in overlays and cache misses

KEY INSIGHT — The viral-post problem is a cache problem, not a database problem

A post that makes the front page concentrates a meaningful share of the **entire site's** read QPS onto one shard's rows. Sharding cannot help — one post has one home. What helps: the ranked-window cache (one fill serves millions of reads between version stamps), read replicas of the hot shard for

cache misses, and coalesced rank updates so the vote storm does not translate into row-write storms. Measure success as **origin QPS during a viral event** staying flat.

5.15 Failure Modes & Degradation

Failure	Symptom	Response
Tally drift	Displayed score ≠ vote log sum	Expected in small doses (coalescing, replays); nightly reconciliation recomputes tallies from `votes`; a repair endpoint fixes single comments
Rank updater lag	Scores update, order doesn't	Visible as consumer lag (Chapter 4's meta-metrics); degraded mode = windows re-sorted by stored (stale) rank keys — the product blurs, never breaks
Event log outage	Writes accepted, ranks frozen	Vote service buffers events locally (Chapter 4's agent pattern); on recovery the backlog replays and coalesces
Thread cache loss	Cache cold on hot posts	Stampede protection: single-flight fill per window (one miss rebuilds, the rest wait), stale-while-revalidate serving
Replica lag on reads	User posts, refreshes, comment missing	Read-your-own-write for authors: route their reads to the primary, or merge their pending writes client-visible via token
Votes DB shard loss	Votes on one slice of comments fail	Fail **closed** for integrity (a queued vote is better than a lost vote); tallies already computed remain served
Brigading wave	Hundreds of votes/s on one thread from fresh accounts	Chapter 3's rate limiter plus velocity anomaly detection (Chapter 4's metrics!); flagged threads shift to slower rank recompute while investigated

5.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Tree encoding	Adjacency + materialized path hybrid	Pure adjacency: recursive subtree reads; nested sets: write amplification on every insert

Decision	Chosen	Alternative & why not (here)
“Best” ranking	Wilson lower bound on up/(up+down)	Raw score: statistically illiterate at small n; ratio: worse — 1/1 beats 200/210; Bayesian average: defensible, but needs a global prior and is harder to explain to users
Rank computation	Recompute on vote, store rank keys	Sort at query time: 50k-row sorts per page view; scheduled recompute: stale beyond tolerance on fast threads
Vote writes	Synchronous upsert + async event	Fully async queue-first: lowest latency, but integrity (one-vote-per-user) becomes eventual — not acceptable
Tallies	Denormalized, reconciled nightly	Count from `votes` per read: honest but scans the largest table on the hottest path
Thread caching	Version-stamped windows, per-user overlay	Per-user full windows: personalization cost multiplies cache by user count; no cache: origin melts on viral posts
Deep nesting	Depth cap + “continue this thread”	Unbounded inline rendering: pathological subtrees (10^4-deep chains) DoS the renderer and the reader
Comment count on posts	Async maintained counter	Synchronous increment on the post row: every comment write contends on one hot row — the exact anti-pattern Section 5.5 warned about

5.17 Observability & SLOs

Indicator	Definition	Target
Thread read latency	Window fetch, cached / uncached, p95	≤ 200 ms / ≤ 700 ms
Comment write latency	Create reply, p95	≤ 300 ms
Vote latency	PUT vote, p95	≤ 150 ms
Rank freshness	Vote → reflected in stored rank keys, p95	≤ 60 s
Read-your-own-write	Author sees own comment after create, p99	≤ 1 s
Tally accuracy	Comments whose tally ≠ vote-log sum, sampled	≤ 0.1%, self-healing nightly

Indicator	Definition	Target
Removal propagation	Moderation removal → gone from all caches	≤ 60 s
Availability	Thread reads / writes	99.99% / 99.9%

Every one of these is a metric with labels and an alert with a `for` duration — Chapter 4's platform, pointed at this chapter's system. The reconciliation job emits tally-drift as a metric; brigading detection is a velocity alert on votes per thread. The book's chapters compose, and saying so in the interview is a senior signal.

5.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Make it real-time — new comments appear without refresh."** Add a subscription tier: WebSocket/SSE gateways subscribed per post; the event log fans `CommentCreated` events to gateways. Key decision: live comments arrive **appended** (by time), not re-sorted — re-ranking a live view under the reader's eyes is a product bug; re-sort applies on next window load.
2. **"How do you detect vote manipulation?"** Signals: velocity anomalies on single threads, account-age and IP clustering of voters, vote-ring graph analysis (accounts that only vote each other). Enforcement: shadow-exclude flagged votes from **rank keys** while keeping them in the visible score — manipulation loses its feedback signal.
3. **"Users edit comments to bait-and-switch after they rank."** Keep edit history; reset or decay a comment's rank confidence on substantive edits (re-open the Wilson interval); show "edited" markers. Ranking trust is the product — this is an integrity issue, not a UX nicety.
4. **"Sharding: what about a post with 10^6 comments?"** One shard holds it — $10^6 \times 1.5$ KB ≈ 1.5 GB, fine; reads are cached windows; votes partition by comment. The real ceiling is per-shard write QPS, and comment writes even on viral posts stay 10^2/s. If it ever breaks, sub-shard the thread by top-level comment id.
5. **"Design comment search."** Chapter 4's inverted index, fed from the same event log: tokenize bodies, index `(post_id, comment_id, tokens)`, rank results by stored Wilson score. Pipelines compose; nothing new is needed.

5.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> A comment system is a **shape** problem wearing a scale costume. The data is a forest of unbounded trees: store it as an adjacency list with materialized paths, so any subtree is one prefix scan and depth is a stored fact; delete with tombstones so replies are never orphaned. Ranking is statistics, not arithmetic: "best" is the Wilson lower confidence bound (evidence beats luck), "controversial" is balanced volume, "hot" is log score plus 12.5-hour decay — all pure functions of `(up, down, time)`, recomputed on every vote and **stored**, so reads are index scans. Votes are the write storm: idempotent upserts keyed by `(user, comment)`, coalesced rank recomputation, tallies reconciled against the vote log nightly. Reads are windows — ranked pages with shallow children — cached per sort mode and version-stamped. The thread is never serialized; the conversation's shape is what ships.

Further reading.

- The source video: *"15: Reddit Comments — Systems Design Interview Questions With Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=BO2gRisnBcA`

- Randall Munroe / Reddit — *"How Reddit ranking algorithms work"* (redditblog, 2009) — the original public write-up of hot/best/controversial.
- Evan Miller — *"How Not To Sort By Average Rating"* — the Wilson score interval derivation Section 5.8 distills, with the canonical worked numbers.
- Joe Celko — *Trees and Hierarchies in SQL for Smarties* — the encyclopedic treatment of adjacency lists, materialized paths, and nested sets.
- Amir Salihefendić — *"How Reddit ranking algorithms work"* commentary and Hacker News ranking write-ups — production variations on time decay.

5.20 Chapter Glossary

Term	Meaning
Adjacency list	Tree encoding where each row stores its parent's id; cheap writes, recursive reads
Coalescing	Collapsing many updates to the same entity into one recomputation; how vote bursts become trickles of rank writes
Comment count	Denormalized per-post tally of comments, maintained asynchronously for listing pages
Confidence interval	A range estimating a true probability from samples; wider (more pessimistic) when data is scarce
Controversial ranking	$(u+d)^{\min/\max}$ — balanced disagreement weighted by volume
Cursor (pagination)	Opaque token encoding the last seen sort key; stable under insertions, unlike offsets
Denormalization	Storing derived values (tallies, rank keys) to make reads cheap; the derived value must be rebuildable from truth
Depth cap	Maximum inline nesting rendered before collapsing behind "continue this thread"
Forest	The set of all comment trees, one rooted at each post
Hot ranking	$\log_{10}\max(\lvert s\rvert,1)+\text{sign}(s)\cdot(t-t_0)/45000$ — log-score with 12.5-hour decay
Idempotency key	A deduplication token making retried writes safe; for votes it is the natural key (user, comment)
Materialized path	Tree encoding storing each row's full ancestor chain; subtrees become prefix scans
Nested sets	Tree encoding with depth-first lft/rgt bounds; range-scan reads, catastrophic insert renumbering
N+1 query	One query per parent to fetch children; the adjacency-list trap that batched path scans eliminate
Rank key	A stored, indexed, orderable encoding of a comment's rank per sort mode; recomputed on vote
Read-to-write ratio	Reads per write; extreme ratios justify aggressive caching and denormalization

Term	Meaning
Read-your-own-write	Guarantee that a writer immediately sees their own write; routed to primary or merged client-side
Score	`up - down`, the visible number; deliberately **not** the "best" ordering
Single-flight fill	One cache miss rebuilds a value while concurrent misses wait; stampede protection
Soft delete / tombstone	Content erased, structure kept, so replies stay threaded under a "[deleted]" placeholder
Time decay	Ranking term that ages content out, forcing continuous freshness
Version stamp	A per-post counter bumped by rank updates; cached windows keyed by it age out without invalidation fanout
Vote overlay	Per-user vote values merged over a shared cached window at read time
Wilson score interval	Binomial confidence interval on the true upvote probability; its lower bound ranks "best" honestly across sample sizes

— End of Chapter 5 · Next: Chapter 6—

CHAPTER 6

Designing a Top-K Leaderboard

> **NOTE — Problem source**
>
> This chapter solves the problem posed in the talk *"17: Top K Leaderboard"* from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the leaderboard system of a hit online game — millions of players submit scores after every match, and the service answers "who are the top K?" and "what is **my** rank?" in milliseconds, on boards that reset daily, weekly, and run all-time. The challenge is not volume but a very specific primitive — **rank as a first-class query** over an ordered set that mutates 25,000 times a second. All terms are defined before use; all reference code is Rust with deterministic tests.

6.1 The Problem Statement

The interviewer sketches a phone screen — a podium of three avatars, a scrolling list below, and a highlighted row pinned at the bottom: **"You: rank 1 247 003"** — and says:

"Players finish matches and submit scores. Players can view the global top 100, page deeper, and always see their own rank and neighborhood even when they are a million places down. Boards exist per season and all-time. Ties go to whoever got the score first. Design it."

This problem looks small next to Chapters 2–5 — and that is the trap. The arithmetic is gentle (Section 6.5); the difficulty is that the two headline queries, **top-K** and **rank-of-player**, must stay O(log n)-ish over an ordered collection of 10^8 entries while that collection is being updated at storm rates. Choose the wrong structure and either the reads or the writes eat the system. Choose the right one and most of the chapter is careful engineering around it: windows, ties, sharding, and cheating.

> **DEFINITION — Leaderboard / rank / top-K**
>
> A *leaderboard* is an ordering of players by score within some scope (a game, a season, a region, a friend group). A player's *rank* is their 1-based position in that ordering. A *top-K* query returns the first K entries in order. The pair {top-K, rank} is the complete query surface — every UI screen is one or the other.

> **DEFINITION — Best-score vs. every-run boards**
>
> Two admission semantics. *Best-score*: a player occupies exactly one slot, holding their personal best; a worse new score changes nothing. *Every-run* (arcade-style): each match is its own entry and one player may hold many slots. We design best-score — the common default — and note where every-run differs (Section 6.10).

6.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Score semantics?	Personal best per player per board; ties broken by who achieved it first
Which boards?	Global all-time, plus daily and weekly; same game, one platform
Rank granularity?	Exact rank for the querying player; "top X%" displays for everyone are fine approximate
Freshness?	A submitted score should be reflected in rank/top-K within 5 seconds
Live updates?	No push; clients poll or refresh. The list may change between pages — acceptable
Anti-cheat?	Hook it in; don't design it. Suspicious scores can be quarantined
Scale?	20M daily players, 200M matches/day, 200M registered accounts
Friends boards?	Yes — a player vs. their friends ($\leq$ a few hundred)
Score range?	Non-negative integers, 0 to 10^9

NOTE — Agreed scope

1. **Submit score** after each match; best-score semantics per player per board; ties to the earliest achiever.
2. **Top-K** with cursor paging (K $\leq$ 100 per page).
3. **My rank** — exact, fast, at any depth, plus a neighborhood view (the ±r players around me).
4. **Time-windowed boards** — daily/weekly boards rotate; all-time persists.
5. **Friends leaderboard** — my standing among my friend list.
6. **Anti-cheat quarantine** hook on the ingestion path.
7. Out: live push, team boards, tournaments, cross-game boards.

6.3 Functional Requirements

DEFINITION — Neighborhood query

The ±r entries around a given player's rank — the "you are here" view. It is the query that makes the problem famous: it requires **rank** (to locate the player) and **ordered iteration** (to walk r places in both directions), at any depth in the ordering, in milliseconds.

1. **FR-1 — Submit score.** `(player_id, score, achieved_at, match_id)` per board scope; stored if it beats the player's current best; idempotent under retries and duplicate match reports.
2. **FR-2 — Top-K page.** Ordered entries `(rank, player, score)` from a cursor, per board.
3. **FR-3 — Player rank.** Exact rank and score for one player on one board.
4. **FR-4 — Neighborhood.** The ±r window around a player, ranks included.
5. **FR-5 — Windowed boards.** Daily/weekly/all-time boards of the same game, rotating automatically, old windows archived read-only.
6. **FR-6 — Friends board.** Top-K/rank restricted to a player's friend set.

6.4 Non-Functional Requirements

DEFINITION — Order-statistics query

A query about **position** in an ordering — "rank of X", "the k-th element", "count of entries above Y" — as opposed to a lookup by key. Supporting order statistics efficiently under inserts/deletes requires an ordered structure augmented with subtree/span counts (Section 6.7); plain hash indexes cannot answer them at all, and plain sorted scans answer them at O(n).

Requirement	Target & reasoning
Score ingest throughput	25k updates/s peak sustained; a launch event may 5× that briefly
Top-K read latency	p95 ≤ 50 ms (cached pages: ≤ 10 ms); it is a hot UI surface
Rank / neighborhood latency	p95 ≤ 100 ms at 10^8-entry boards — this kills naive designs
Freshness	Score → visible in rank/top-K ≤ 5 s; rank need not be transactional with the match
Durability	A confirmed personal best must survive node loss (replication + journaling)
Availability	Reads ≥ 99.99% (stale board pages acceptable); writes ≥ 99.9%
Integrity	One slot per player per board; ties deterministic; quarantined scores never render

KEY INSIGHT — Reads and writes are both modest — the operation mix is the design

Nothing here rivals Chapter 4's firehose or Chapter 5's vote storm. What is unusual: the system must maintain a **total order** over 10^8 entries, mutate it 25k times a second, and answer positional queries — top-K, rank, neighborhood — in milliseconds. Data structure choice **is** the architecture; everything else is distribution plumbing around one ordered set.

6.5 Back-of-the-Envelope Estimation

Assumptions: 20M daily players; 10 matches per player per day; every match submits one score; each player opens leaderboard surfaces 5×/day; a board entry is 100 B with index overhead.

Quantity	Estimate	Derivation
Score submissions	200M/day ≈ 2.3k/s avg, 25k/s peak	20M players × 10 matches; evening × event multiplier ≈ ×10
Leaderboard reads	100M/day ≈ 1.2k/s avg, 12k/s peak	20M × 5 views; top-K pages + rank lookups
Global all-time board	20 GB in RAM	200M players × 100 B (id, score, ts, structure overhead)
Daily board	2 GB	20M active × 100 B; born and dies daily
Top-100 page	5 KB JSON	100 × (id, name ref, score, rank)
Update cost	27 pointer hops	$\log_2(2\times10^8) \approx 27$ — the ordered set makes writes cheap
Snapshots	20 GB/board, minutes to write	Sequential dump; recovery = snapshot + journal replay

KEY INSIGHT — The whole working set fits in RAM — spend it

20 GB for the all-time board is one machine's RAM, or a few shards' worth. That licenses the chapter's central decision: keep boards **in memory** in a purpose-built ordered structure with O(log n) everything, replicate for reads and failover, journal for durability. A disk-first database would answer the same queries 10–100× slower for zero capacity benefit. Capacity is solved; latency and structure are the game.

The flow of the chapter: Section 6.6 isolates why rank is the crux; 6.7 picks the data structure (with the skip-list mechanics drawn out); 6.8–6.9 design APIs and architecture; 6.10–6.12 go deep on ingestion semantics, sharded top-K, and windowed/friends boards; 6.13 implements it all in Rust; 6.14–6.20 scale, harden, and review.

6.6 The Core Challenge: Rank Is Not a Lookup

Strip the product away and the service is one abstract data type with four operations: `upsert(player, score)`, `top(k, cursor)`, `rank(player)`, `neighborhood(player, r)` — under 25k mutations/s and 12k queries/s, on a collection of 10^8. Walk the candidate structures:

Structure	upsert	top-K	rank(player)	Verdict
Hash map	O(1)	O(n log n)	O(n log n)	Rank does not exist; every query re-sorts 10^8 entries
Sorted array	O(n)	O(K)	O(log n) by score	Reads fine; O(n) write shifts ruin 25k updates/s
B-tree index (DB)	O(log n)	O(log n + K)	O(n) count scan	The classic wrong answer — see below
Heap	O(log n)	O(K log n)	unsupported	No rank, no paging; wrong shape entirely
Skip list + hash map, spans	**O(log n)**	**O(log n + K)**	**O(log n)**	Ordered, rank-augmented, in-memory — Section 6.7

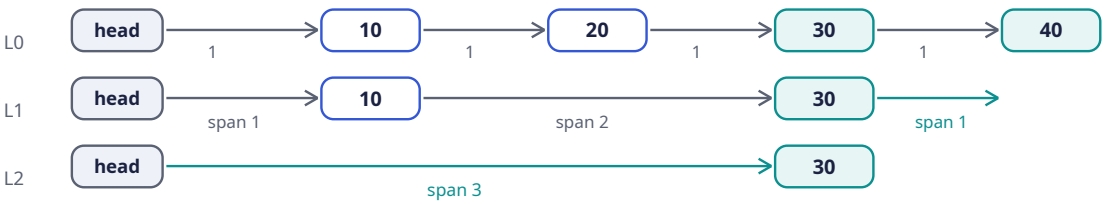
COMMON PITFALL — The `COUNT(*)` rank trap

The most common wrong answer: store scores in a relational table, index the score column, and compute rank as `SELECT COUNT(*) WHERE score > :mine`. The index makes top-K fine, but rank-by-count touches **every entry above the player** — for rank 1.2M of 200M, that is a 198.8M-row index scan per query, thousands of times a second. Rank must be **maintained by the structure**, not computed by counting. This is the sentence the interviewer is listening for.

6.7 Deep Dive: The Data Structure — Skip List with Spans

DEFINITION — Skip list / span

A *skip list* is a probabilistically balanced ordered structure: a sorted linked list (level 0) with express lanes above it — each node is promoted to the next level with probability p (e.g. 1/4), so level i holds p^i of the nodes. Search starts on the top express lane and drops down, skipping 1/p nodes per hop: O(log n) expected search, insert, delete. A *span* is an integer on every forward pointer counting how many level-0 steps it covers; spans turn the skip list into an **order-statistics** structure: summing spans along a search path yields the rank directly, and rank + forward walks yield neighborhoods — still O(log n).



rank(40) = spans along the search path: 3 (L2, head→30) + 1 (L1, 30→40) = 4 — position without counting

Why this over a balanced tree: same asymptotics, but level-promotion makes inserts/deletes local (no rotations), concurrent implementations are far simpler (only level 0 needs careful linking), and span bookkeeping is an 10-line addition. It is exactly the structure at the core of the industry's canonical sorted-set implementations — `hash map player→node` for O(1) lookup, skip list for order and rank.

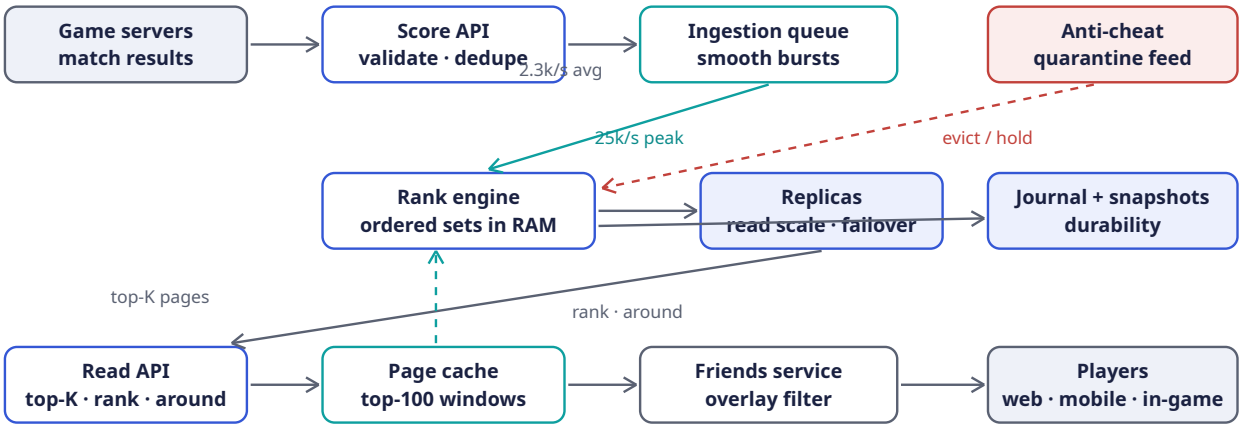
Ties slot in cleanly: the ordering key is not `score` but the triple `(score desc, achieved_at asc, player_id)` — deterministic total order, no equal keys, "first to achieve" falls out of the comparison itself. Section 6.13 implements it on an ordered map (same complexity story; Rust's std ordered map lacks spans, and the listing says so honestly).

6.8 API Design

Method	Path	Purpose
POST	/v1/scores	Submit match score {player_id, board, score, achieved_at, match_id} — best-score + idempotent (FR-1)
GET	/v1/leaderboards/{board}/top?cursor=&limit=	Top-K page (FR-2); cursor = last (score, ts, id) triple
GET	/v1/leaderboards/{board}/players/{id}	Exact rank + score (FR-3)
GET	/v1/leaderboards/{board}/players/{id}/around?r=	Neighborhood window (FR-4)
GET	/v1/leaderboards/{board}/players/{id}/friends	Friends board (FR-6)
POST	/v1/admin/quarantine	Anti-cheat hook: remove/hold entries (scope)

Notes: `board` encodes window and scope (`global:alltime`, `global:2026-W36`, `region:eu:daily:2026-09-05`). Cursors are the rank triple, opaque and base64'd — Chapter 5's pagination lesson, and here the cursor **is** the ordering key, so paging is a pure `range()` from the cursor position. Submission responses return `{accepted, improved, current_best}` so clients can celebrate personal bests without a second call.

6.9 High-Level Architecture



Writes: validate → queue → in-memory ordered sets (replicated, journaled). Reads: cached pages for the top, live rank queries for everything else

Component	Responsibility	Why it is shaped this way
Score API	Authenticate game servers, validate, dedupe by `match_id`	Writes enter through one narrow, auditable door
Ingestion queue	Buffer score events; absorb event-day bursts	Chapter 4's shock absorber; ingestion lag becomes a metric, not an outage
Rank engine	Holds all boards as in-memory ordered sets; applies best-score updates; serves rank/top-K/around	The structure of Section 6.7 **is** the system; 20 GB fits in RAM (Section 6.5)
Replicas	Read scale-out + hot standby	Rank reads ×12k/s spread; promotion covers node loss
Journal + snapshots	Append-only update log; periodic board dumps	Recovery = latest snapshot + replay; the board is rebuildable, never lost
Anti-cheat	Consumes the same stream; quarantines suspects	A consumer like any other — removal is just a delete from the set
Page cache	Top-100 pages per board, few-second TTL	The podium is read 10^3× more often than it changes (top-100 churn is slow; Section 6.14)
Friends service	Intersection of friend list with board entries	Small overlays computed on read; Section 6.12

6.10 Deep Dive: Score Ingestion Semantics

Best-score admission makes the write path almost suspiciously simple:

> **DEFINITION — Monotonic (idempotent) update**
>
> An update whose repeated application changes nothing after the first time. Best-score submission is monotonic: `board[player] = max(old, new)` absorbs retries, duplicates, and out-of-order redeliveries for free — no dedup tokens, no exactly-once machinery on the hot path. (Chapter 5's votes needed

an upsert table; here the **semantics** do the work.) The one non-monotonic case — same player, same score, **earlier** timestamp stealing the tie-break — is excluded by rule: a player's slot keeps its first achieving timestamp.

The ingestion contract, end to end:

1. Game server submits `(player, board, score, achieved_at, match_id)`; the Score API dedupes by `match_id` briefly (same match reported twice) and validates shape.
2. The queue buffers; the rank engine applies `max` updates in stream order. Bursts only stretch the freshness budget (≤5 s) — never correctness.
3. Anti-cheat consumes the same stream independently; a quarantine verdict deletes the player's entries from every board (a delete is just another update) and parks their submissions until cleared. The leaderboard never renders a quarantined score — integrity requirement, Section 6.4.
4. **Every-run variant** (arcade boards): entries keyed `(match_id)` instead of player; dedupe by match id is still free; top-K is unchanged; "my rank" becomes "my best run's rank". One sentence in the interview shows the abstraction holds.

6.11 Deep Dive: Sharding and the Global Top-K

One node holds 20 GB and applies 25k skip-list updates/s — a single beefy rank engine covers our numbers with replicas. But boards grow (new regions, games, seasons), so the sharding answer must be ready:

Shard by `hash(player_id)`. Each shard owns a complete ordered set of its players. Then:

- **Top-K, exact**: fetch the top K from **every** shard and k-way merge. Correctness: any member of the global top-K has at most K−1 players outranking it globally, hence at most K−1 on its own shard — so it must appear in its shard's top-K. Fetching K per shard is sufficient and necessary. Cost: `shards × K` entries per query — 8 shards × 100 = 800 entries per page, trivial.
- **My rank, exact**: `1 + Σ_shards count(better than me)` — a scatter-gather count across all shards, O(log n) per shard with spans, but fan-out per query. Fine at 1.2k rank-reads/s; painful at 10^5/s.
- **"Top X%" displays, approximate**: each shard also maintains a fixed-boundary **histogram** of its scores. Histograms are **mergeable by addition** — sum the bucket counts and interpolate (Section 6.13) — so a global percentile is a cheap aggregate with no global ordering at all. This is how "top 0.4%" renders without ever computing 200M ranks.

> **KEY INSIGHT — Exact where it is read, approximate where it is glanced**
>
> The podium (top-100) must be exact — it is the product. A player's own rank must be exact — they will screenshot it. The "top 2.3%" badge on a profile is **glanced at**, not audited — approximate it with mergeable histograms and save the global scatter-gather entirely. Precision is a budget; spend it only on surfaces where users check the math.

6.12 Deep Dive: Windowed and Friends Boards

Time windows are just keys. A board is identified by `(scope, period)` — `global:alltime`, `global:weekly:2026-W36`, `global:daily:2026-09-05`. Every score submission writes to **all live windows** for its scope (daily + weekly + all-time = three ordered-set updates, still cheap). Rotation is a scheduler flipping the live key: yesterday's daily board stops accepting writes, gets snapshotted to cold storage, and stays readable as a frozen archive. No migration, no reindexing — the empty new window is created lazily by the first write.

Friends boards are overlays, not boards. A friend list is ≤ a few hundred players. On read: fetch each friend's current `(score, ts)` by direct player lookup (the hash-map side of the ordered set, O(1) each), sort in memory, rank locally. Milliseconds, zero storage. The alternative — materializing a per-user friends board at write time — multiplies every score update by the player's appearance in friend lists (a Chapter 2-style fanout write amplification) to answer a rare query. Compute on read wins decisively here.

6.13 Rust Reference Implementations

Three pieces with deterministic tests: the leaderboard itself, the sharded top-K merge, and the mergeable percentile histogram.

6.13.1 The Leaderboard: Ordered Set + Player Index

```rust
use std::cmp::Reverse;
use std::collections::{BTreeMap, HashMap};

/// Ordering key: score desc, then earliest achieved_at, then player id —
/// a total order, so entries never tie and "first to achieve" is free.
type Key = (Reverse<u64>, u64, u64);

/// In-memory board. `entries` is the ordered set (a stand-in for the
/// span-augmented skip list of Section 6.7 — same interface and big-O,
/// except rank, noted below); `by_player` is the O(1) side index.
#[derive(Default)]
pub struct Leaderboard {
    entries: BTreeMap<Key, ()>,
    by_player: HashMap<u64, (u64 /*score*/, u64 /*achieved_at*/)>,
}

impl Leaderboard {
    /// Best-score admission: strictly better scores only. Monotonic, so
    /// retries and duplicate submissions are safe by construction.
    pub fn submit(&mut self, player: u64, score: u64, achieved_at: u64) -> bool {
        if let Some(&(old, _)) = self.by_player.get(&player) {
            if score <= old { return false; }
        }
        if let Some(&(old, old_ts)) = self.by_player.get(&player) {
            self.entries.remove(&(Reverse(old), old_ts, player));
        }
        self.entries.insert((Reverse(score), achieved_at, player), ());
        self.by_player.insert(player, (score, achieved_at));
        true
    }

    pub fn top_k(&self, k: usize) -> Vec<(u64, u64)> {
        self.entries
            .keys()
            .take(k)
            .map(|&(Reverse(s), _, id)| (id, s))
            .collect()
    }

    /// 1-based rank. NOTE: Rust's ordered map keeps no order statistics, so
    /// this counts the better half — O(rank). The span skip list answers the
    /// same call in O(log n); only the implementation changes.
    pub fn rank_of(&self, player: u64) -> Option<u64> {
        let &(s, ts) = self.by_player.get(&player)?;
        let better = self.entries.range(..(Reverse(s), ts, player)).count();
```

```
        Some(better as u64 + 1)
    }

    /// ±r window around a player: (rank, player, score), edges clamped.
    pub fn neighborhood(&self, player: u64, r: usize) -> Vec<(u64, u64, u64)> {
        let rank = match self.rank_of(player) {
            Some(r) => r as usize,
            None => return vec![],
        };
        let lo = rank.saturating_sub(r + 1); // 0-based start
        let hi = (rank + r).min(self.entries.len()); // exclusive end
        self.entries
            .keys()
            .enumerate()
            .skip(lo)
            .take(hi - lo)
            .map(|(i, &(Reverse(s), _, id))| (i as u64 + 1, id, s))
            .collect()
    }
}

#[cfg(test)]
mod board_tests {
    use super::*;

    #[test]
    fn best_score_ties_and_rank() {
        let mut lb = Leaderboard::default();
        assert!(lb.submit(1, 500, 100)); // alice
        assert!(lb.submit(2, 900, 100)); // carol
        assert!(lb.submit(3, 500, 200)); // bob: same score, later ts
        assert!(lb.submit(4, 100, 100)); // dave
        // board order: 2 (900), 1 (500@100), 3 (500@200), 4 (100)
        assert_eq!(lb.top_k(2), vec![(2, 900), (1, 500)]);
        assert_eq!(lb.rank_of(1), Some(2)); // tie broken by earlier ts
        assert_eq!(lb.rank_of(3), Some(3));
        assert_eq!(lb.rank_of(4), Some(4));
        assert_eq!(lb.rank_of(99), None);
        // monotonic admission: worse or equal scores change nothing
        assert!(!lb.submit(1, 400, 300));
        assert!(!lb.submit(1, 500, 50));
        assert_eq!(lb.rank_of(1), Some(2));
        // improvement repositions
        assert!(lb.submit(1, 950, 400));
        assert_eq!(lb.top_k(2), vec![(1, 950), (2, 900)]);
    }

    #[test]
    fn neighborhood_windows_clamp_at_edges() {
        let mut lb = Leaderboard::default();
        for i in 0..10u64 {
            lb.submit(i, 1000 - i * 10, 1); // id i -> rank i+1
        }
        let w = lb.neighborhood(4, 2); // rank 5 -> ranks 3..=7
        let ranks: Vec<u64> = w.iter().map(|e| e.0).collect();
        let ids: Vec<u64> = w.iter().map(|e| e.1).collect();
        assert_eq!(ranks, vec![3, 4, 5, 6, 7]);
        assert_eq!(ids, vec![2, 3, 4, 5, 6]);
        let w0 = lb.neighborhood(0, 2); // top edge
        assert_eq!(w0.iter().map(|e| e.0).collect::<Vec<_>>(), vec![1, 2, 3]);
```

```
        assert!(lb.neighborhood(99, 2).is_empty());
    }
}
```

6.13.2 Sharded Top-K: The K-Way Merge

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

/// One shard's top entries, already in board order (score desc, ts asc).
pub type ShardTop = Vec<(u64 /*player*/, u64 /*score*/, u64 /*ts*/)>;

/// Global top-k by merging each shard's top-k. Sound because any global
/// top-k member has < k entries outranking it globally, hence < k on its own
/// shard — so it is always inside its shard's top-k contribution.
pub fn merge_top_k(shards: &[ShardTop], k: usize) -> Vec<(u64, u64, u64)> {
    // max-heap on (score, Reverse(ts)): highest score, earliest ts pops first
    let mut heap = BinaryHeap::new();
    for (s, shard) in shards.iter().enumerate() {
        if let Some(&(p, sc, ts)) = shard.first() {
            heap.push((sc, Reverse(ts), p, s, 0usize));
        }
    }
    let mut out = Vec::with_capacity(k);
    while let Some((sc, Reverse(ts), p, s, i)) = heap.pop() {
        out.push((p, sc, ts));
        if out.len() == k { break; }
        if let Some(&(p2, sc2, ts2)) = shards[s].get(i + 1) {
            heap.push((sc2, Reverse(ts2), p2, s, i + 1));
        }
    }
    out
}

#[cfg(test)]
mod merge_tests {
    use super::*;

    #[test]
    fn merge_matches_brute_force_global_sort() {
        let shards: Vec<ShardTop> = vec![
            vec![(1, 900, 5), (4, 300, 5)],
            vec![(2, 950, 5), (5, 250, 5)],
            vec![(6, 500, 1), (3, 500, 5)], // tie at 500: earlier ts first
        ];
        let merged = merge_top_k(&shards, 4);
        let ids: Vec<u64> = merged.iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![2, 1, 6, 3]);

        let mut all: Vec<(u64, u64, u64)> = shards.concat();
        all.sort_by(|a, b| b.1.cmp(&a.1).then(a.2.cmp(&b.2)));
        let oracle: Vec<u64> = all.into_iter().take(4).map(|e| e.0).collect();
        assert_eq!(ids, oracle);
    }

    #[test]
    fn k_per_shard_is_sufficient() {
        // all three global leaders live on one shard; asking only k=3
        // per shard still recovers the exact global top-3
```

```
        let shards: Vec<ShardTop> = vec![
            vec![(1, 100, 1), (2, 90, 1), (3, 80, 1), (4, 70, 1)],
            vec![(5, 60, 1)],
        ];
        let per_shard: Vec<ShardTop> =
            shards.iter().map(|s| s.iter().take(3).cloned().collect()).collect();
        let ids: Vec<u64> =
            merge_top_k(&per_shard, 3).iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![1, 2, 3]);
    }
}
```

6.13.3 Mergeable Percentile Histograms

```
/// Fixed-boundary histogram for approximate "top X%" displays.
/// counts[i] = scores in [bounds[i-1], bounds[i]); the last bucket catches
/// everything >= the final bound. Histograms merge by adding counts —
/// that is what makes global percentiles cheap across shards.
pub struct ScoreHistogram {
    bounds: Vec<u64>,
    counts: Vec<u64>,
    total: u64,
}

impl ScoreHistogram {
    pub fn new(bounds: Vec<u64>) -> Self {
        assert!(!bounds.is_empty());
        let m = bounds.len();
        ScoreHistogram { bounds, counts: vec![0; m + 1], total: 0 }
    }

    pub fn add(&mut self, score: u64) {
        let i = self.bounds.partition_point(|&b| score >= b);
        self.counts[i] += 1;
        self.total += 1;
    }

    /// Merge another shard's histogram (identical bounds) by addition.
    pub fn merge(&mut self, other: &ScoreHistogram) {
        assert_eq!(self.bounds, other.bounds);
        for (a, b) in self.counts.iter_mut().zip(&other.counts) {
            *a += b;
        }
        self.total += other.total;
    }

    /// Estimated fraction of players scoring below `score`, with linear
    /// interpolation inside the straddling bucket.
    pub fn percentile_of(&self, score: u64) -> f64 {
        if self.total == 0 { return 0.0; }
        let (mut below, mut lower) = (0.0f64, 0u64);
        for (i, &upper) in self.bounds.iter().enumerate() {
            if score >= upper {
                below += self.counts[i] as f64;
                lower = upper;
            } else {
                if score > lower {
                    let frac = (score - lower) as f64 / (upper - lower) as f64;
                    below += frac * self.counts[i] as f64;
```

```
                }
                return below / self.total as f64;
            }
        }
        // score at/past the final bound: overflow bucket counts only when
        // strictly below `score` (bucket contents are >= the last bound)
        if score > *self.bounds.last().unwrap() {
            below += self.counts[self.bounds.len()] as f64;
        }
        below / self.total as f64
    }
}

#[cfg(test)]
mod hist_tests {
    use super::*;

    #[test]
    fn percentile_tracks_uniform_distribution() {
        let mut h =
            ScoreHistogram::new(vec![100, 1_000, 10_000, 100_000, 1_000_000]);
        for i in 0..1000u64 { h.add(i * 1_000); } // 0, 1000, ..., 999_000
        assert_eq!(h.total, 1000);
        assert_eq!(h.percentile_of(0), 0.0);
        let p50 = h.percentile_of(500_000); // interpolates in [100k, 1M)
        assert!((p50 - 0.5).abs() < 0.05, "p50 = {p50}");
        assert_eq!(h.percentile_of(2_000_000), 1.0);
        // "top X%" badge: the 999_500 score is elite
        assert!((1.0 - h.percentile_of(999_500)) * 100.0 < 1.0);
    }

    #[test]
    fn histograms_merge_by_addition() {
        let bounds = vec![100, 1_000];
        let (mut a, mut b) =
            (ScoreHistogram::new(bounds.clone()), ScoreHistogram::new(bounds));
        for s in [0, 50, 500] { a.add(s); }
        for s in [5_000, 9_999] { b.add(s); }
        a.merge(&b);
        assert_eq!(a.total, 5);
        assert!((a.percentile_of(1_000) - 0.6).abs() < 1e-9); // 3 of 5 below
    }
}
```

6.14 Scaling the Platform

Layer	Scale axis	Mechanism
Score API	Submission rate	Stateless; horizontal
Ingestion queue	25k/s avg→peak bursts	Partitioned by board+shard key; lag is a metric, not an outage (Chapter 4 pattern)
Rank engine	Board entries × update rate	One node per board-group until RAM or write QPS demands sharding by `hash(player)`; O(log n) updates

Layer	Scale axis	Mechanism
		mean a single core does 10^6 ops/s — 40× headroom over peak
Read path	Rank/top-K reads	Replicas + the page cache; the podium page is shared by everyone and changes slowly — a few-second TTL absorbs the viral majority
Storage	200M-entry all-time board	20 GB RAM per replica; snapshots to object storage; journal retention days
Windows	Board count = scopes × live windows	Lazy creation; frozen archives are read-only snapshots — zero live cost

KEY INSIGHT — Top-100 churn is glacial compared to mid-table churn

The millionth-best player changes rank with every match; the podium changes a few times an hour. So cache **top pages** aggressively (they barely move), serve **rank/neighborhood** live from the ordered set (they move constantly and are queried rarely per position), and never cache mid-table pages — they are wrong the moment they are written. The cache policy mirrors the physics of the data.

6.15 Failure Modes & Degradation

Failure	Symptom	Response
Rank engine node loss	Board reads fail over, writes pause seconds	Promote a replica; it is already warm. Recovery to a fresh node = snapshot + journal replay
Journal gap / corruption	Replay inconsistency on recovery	Checksummed journal frames; on gap, rebuild the board from match-history re-ingest (slow but total) — the board is derived state over score facts
Queue backlog on event day	Freshness degrades past 5 s	Degrade **freshness**, never integrity: clients show "updating…" affordance; Chapter 3's limiter sheds anonymous poll traffic first
Replica lag	Stale ranks on some reads	Version the board (update count); reads report the version so clients never go backwards within a session
Hot shard	One shard's players dominate writes	Reshard by splitting the hash range; top-K merge is shard-count-agnostic, so reads need no coordination
Anti-cheat pipeline down	Suspect scores render	Fail toward quarantine for **new** suspicious velocity; previously

Failure	Symptom	Response
		cleared scores unaffected. Integrity beats availability here — Section 6.4
Clock skew in `achieved_at`	Wrong tie-breaks	Ties are rare and low-stakes; still, trust server receipt time over client clocks, accept `achieved_at` only within a tolerance window

6.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Storage of boards	In-memory ordered sets + journal	Disk-first DB: 10–100× slower rank ops for zero capacity gain — the working set fits in RAM
Rank computation	Structure-maintained (spans)	Rank-by-count SQL: O(better-entries) per query — melts (Section 6.6)
Score semantics	Best-score, monotonic	Every-run: right for arcade boards; same machinery, different key (match_id)
Global top-K	Per-shard top-K + k-way merge	One global sorted set across shards: distributed ordering is a consensus problem nobody needs for a leaderboard
Global percentile	Mergeable histograms	Exact scatter-gather counts: fine at 10^3/s, wrong tool for every profile badge
Friends board	Computed on read from the friend list	Materialized per-user boards: write fanout per score — Chapters 2/5's fanout lesson
Freshness	≤5 s via async apply	Synchronous end-to-end: turns every match-end into a distributed transaction
Live push	Out of scope; polling + short TTL	WebSocket fanout of rank changes: a fine Chapter 1-style add-on, but rank updates at 25k/s make push storms the default state

6.17 Observability & SLOs

Indicator	Definition	Target
Ingest freshness	Score accepted → reflected in rank engine, p95	≤ 5 s

Indicator	Definition	Target
Top-K latency	Page read, cached / live, p95	≤ 10 ms / ≤ 50 ms
Rank latency	rank + neighborhood, p95	≤ 100 ms
Board integrity	Sampled players whose stored slot ≠ submitted best	≈ 0 (journal-rebuildable)
Quarantine latency	Verdict → entries removed from all boards	≤ 60 s
Availability	Reads / writes	99.99% / 99.9%

Chapter 4's platform carries all of it: ingestion lag as a consumer-lag metric, freshness as a histogram, an alert with a `for` duration on replica version drift. The reconciliation sampler — comparing stored slots against recomputed bests from the journal — is this chapter's version of Chapter 5's tally audit.

6.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Push live rank changes to clients."** Fan-out problem: 25k updates/s each reorder thousands of ranks — nobody's visible rank actually moved. Push only **material** changes: podium swaps, personal bests, crossing friends. Everything else is noise dressed as real-time.
2. **"Team leaderboards."** Team score = aggregate of members (sum, or ELO-style rating); the ordered set is identical, keyed by team. The interesting part is member-change handling and anti-stack incentives, not the structure.
3. **"Regional boards at 50 regions × 3 windows?"** Boards are cheap keys — 150 extra ordered sets, each small. Writes fan out per applicable board (a player's score lands on their region's set + global); reads unchanged.
4. **"How do you catch cheaters?"** Server-side authority (the game server signs match results, clients never self-report), statistical anomaly detection on score distributions per level (a Chapter 4 alert on percentile outliers), velocity checks (Chapter 3's limiter: matches/hour per player), and quarantine-not-delete so false positives recover.
5. **"10^9 players — what breaks first?"** Single-node RAM (100 GB is still one node, but uncomfortable) and the top-K merge fan-out staying flat — the design already shards; the rank scatter-gather for exact global rank is what you would approximate next (histograms), keeping exact rank only within a player's region shard.

6.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> A leaderboard is one primitive — **rank over a mutating total order** — and the chapter is the engineering that protects it. The structure is an ordered set: hash map for player lookup, skip list with spans for O(log n) rank, top-K, and neighborhoods; ties are a total ordering key `(score, achieved_at, id)`. Writes are monotonic best-score updates — idempotent by construction, journaled for durability, replicated for reads. Reads split by physics: the podium is cached (it barely moves), personal rank is live (it always moves), percentiles are approximated with mergeable histograms (they are glanced at, not audited). Sharding by player hash keeps top-K exact through a k-way merge — any global top-K member is provably inside its shard's top-K. Windows are keys; friends boards are overlays; cheating is a stream consumer with the power to delete. The lesson that transfers: **when the query is positional, the data structure is the architecture.**

Further reading.

- The source video: *"17: Top K Leaderboard — Systems Design Interview Questions With Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=nQpkRONzEQI`
- William Pugh — *"Skip Lists: A Probabilistic Alternative to Balanced Trees"* (1990) — the structure of Section 6.7, including span-based ranks.
- Redis sorted sets documentation (`ZADD` / `ZRANK` / `ZRANGE`) — the canonical production embodiment of this chapter's ordered set.
- Cormen et al., *Introduction to Algorithms* — the order-statistics tree chapter (subtree sizes) for the balanced-tree equivalent of spans.
- *"Elo rating system"* and TrueSkill papers — for the follow-up where the board ranks **skill** rather than scores.

6.20 Chapter Glossary

Term	Meaning
Best-score semantics	One slot per player per board holding their personal best; worse scores are no-ops
Cursor	Opaque paging token; here literally the ordering key triple `(score, achieved_at, id)` of the last entry
Every-run semantics	Arcade-style admission where each match is a separate entry and a player may hold many slots
Exact rank	A player's precise 1-based position; required for self-queries, wasteful for global percentiles
Friends board	A leaderboard restricted to a player's friend set; computed as a read-time overlay, not stored
Histogram (mergeable)	Fixed-bucket score distribution; bucket counts add across shards, giving cheap approximate percentiles
Idempotent / monotonic update	An update whose repetition changes nothing; best-score `max` admission has this property for free
Journal (write-ahead)	Append-only record of updates enabling replay after snapshot restore
K-way merge	Merging k-sorted shard outputs with a heap; sound for top-K because K entries per shard always suffice
Leaderboard	An ordering of players by score within a scope (window, region, friends)
Neighborhood query	The ±r window around a player's rank — needs both rank and ordered iteration
Order-statistics query	A positional query (rank, k-th, count-above); needs an augmented ordered structure, not a plain index
Ordered set	The abstract data type: map from member to score plus rank-ordered iteration; hash map + skip list in practice
Percentile / "top X%"	Approximate positional display computed from merged histograms instead of exact ranks

Term	Meaning
Quarantine	Anti-cheat state: scores held out of all boards pending verdict; deletable, recoverable
Rank	1-based position in a leaderboard's ordering
Shard sufficiency (top-K)	The theorem that global top-K needs only K entries per shard, since a global top-K member is outranked by < K entries anywhere
Skip list	Probabilistically balanced ordered list with express lanes; O(log n) search/insert/delete without rotations
Span	Count on a skip-list forward pointer of level-0 steps covered; summing spans along a search path yields rank
Tie-break	Deterministic total order extension: `(score desc, achieved_at asc, id)` — first to achieve wins
Top-K	The first K entries of a board in order; the podium query
Windowed board	A board scoped to a time period (daily/weekly); rotation is a key change, archives are frozen snapshots

— End of Chapter 6 · Next: Chapter 7 —

CHAPTER 7

Designing a Recommendation Engine

NOTE — Problem source

This chapter solves the problem posed in the talk *"22: Recommendation Engine (YouTube, TikTok)"* from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the personalized home feed of a video platform — hundreds of millions of users, a billion-item catalog, and 300 milliseconds to decide which twenty items a given user sees next. It is the chapter where the book's infrastructure (event pipelines, ranking, caching) meets machine learning as a **systems** concern: feature stores, model freshness, and feedback loops. All terms are defined before use; all reference code is Rust with deterministic tests.

7.1 The Problem Statement

The interviewer opens a video app's home screen — an endless, uncannily well-chosen scroll — and says:

"When a user opens the app, we show them a personalized feed of videos. We know everything they've watched, liked, skipped, and how long they lingered. A million new videos arrive every day. Design the system that decides what goes on this screen."

Every prior chapter had a crisp correct answer to compute; this one computes a **guess**. That changes the engineering: the system's job is to take a billion-item catalog and a user's entire history, and under a hard latency budget produce an ordering that maximizes a business metric nobody can measure directly. The design that emerged across the industry — a **funnel** of candidate generation followed by ranking — is the spine of this chapter, and understanding **why** it has that shape is most of the interview.

DEFINITION — Recommendation / the feed

A *recommendation* is a predicted-relevance ordering of catalog items for a specific user at a specific moment. The *feed* is its product form: the ranked, paginated, infinite scroll. Unlike search (Chapter 4's inverted index answers "what matches this query"), the feed answers "what does this user want, having asked nothing" — the query is the user's history.

DEFINITION — Implicit vs. explicit feedback

Explicit feedback is deliberate: likes, ratings, follows, "not interested". *Implicit* feedback is behavioral exhaust: impressions (the item was shown), plays, watch time, skips, rewatches. Implicit feedback is 1000× more abundant and is what actually trains the system — watch time and skips are votes every user casts on every item, whether they mean to or not (Chapter 5's votes, but continuous and uninvited).

7.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
What surfaces?	Just the home feed. Search, subscriptions page, and ads are separate systems
What do we optimize?	Watch time / session satisfaction — assume a business metric exists; your job is the machinery
Freshness requirements?	A great new video should be discoverable within minutes; user actions should affect their feed within the session
How personalized?	Fully — no two users' feeds need overlap. But logged-out users get a fallback (trending)
Model training?	Not the focus — data scientists own model internals. You own data flow, features, serving, latency
Content safety?	A policy filter gate exists; respect it, don't design it
Scale?	500M daily users, 1B public videos, 2B feed loads/day
Cold start?	Must have an answer for new users and new videos

NOTE — Agreed scope

1. **Feed generation**: personalized, ranked, paginated home feed per user.
2. **Feedback loop**: impressions/plays/watch-time/likes recorded and reflected — in-session effects within seconds, model retraining daily.
3. **Candidate diversity**: multiple retrieval channels (follows, similarity, embeddings, trending), blended.
4. **Freshness & cold start**: new items get an exploration quota; new users get trending + rapid personalization from first signals.
5. **Guardrails**: dedup, policy filter, diversity caps in the final mix.
6. Out: search, ads ranking, model architecture design, content moderation itself.

7.3 Functional Requirements

DEFINITION — Candidate generation (retrieval) vs. ranking (scoring)

The two mandatory stages of any industrial recommender. *Candidate generation* cheaply selects 10^3–10^4 plausibly-relevant items from the billion (per-channel: what your follows posted, items similar to what you loved, embedding neighbors, trending). *Ranking* expensively scores those few thousand with the full feature set and orders them. Retrieval is recall-shaped ("don't miss anything they'd love"); ranking is precision-shaped ("order these twenty slots perfectly"). Neither can do the other's job at scale — Section 7.6 derives this.

1. **FR-1 — Feed.** `GET /feed` returns the next page of ranked items for the user, with an opaque cursor; the first page is the home screen.
2. **FR-2 — Event intake.** Impressions, plays, watch time, likes, skips, "not interested" recorded durably and at scale.
3. **FR-3 — Session adaptation.** Signals from **this** session (last N watches) influence the next page within seconds.
4. **FR-4 — Channel blend.** Every page mixes sources: followed creators, similar-to-history, embedding neighbors, trending, and an exploration quota for new items.

5. **FR-5 — Negative feedback.** "Not interested" removes the item and down-weights its ilk immediately.
6. **FR-6 — Cold start.** New users receive trending + category picks, personalizing from the very first events; new videos receive a small, guaranteed stream of test impressions.

7.4 Non-Functional Requirements

DEFINITION — Latency budget

The total time a request may consume, allocated across stages. Ours: 300 ms p95 end-to-end for a feed page — 50 ms retrieval fan-out, 150 ms ranking 5k candidates, 50 ms re-rank + assemble, 50 ms network and slack. The budget **dictates the architecture**: anything that cannot fit its allocation must be precomputed offline.

Requirement	Target & reasoning
Feed latency	p95 ≤ 300 ms per page; it is the app's front door
Event throughput	580k events/s avg, 3M/s peak — Chapter 4's firehose, with a job to do
Feed freshness	A just-watched video influences the next page ≤ 10 s (session features); new **model** daily
New-item discoverability	Every eligible new video gets its exploration impressions ≤ 1 h from upload
Availability	Feed ≥ 99.95%; graceful degradation = trending + follows (never an empty screen)
Diversity	No category/creator may dominate a page beyond caps; the feed must not collapse to one topic
Model staleness	Serving model ≤ 36 h old; features ≤ 10 s (session) / ≤ 1 h (rest)

KEY INSIGHT — Relevance is a budget problem

Scoring one (user, item) pair well is a solved ML problem. The system's entire shape comes from arithmetic: 2B requests/day × a 1B-item catalog means you may score, at most, a few thousand items per request. Everything — retrieval channels, approximate nearest neighbors, precomputed similarities, the funnel itself — is a device for spending that tiny scoring allowance on the right few thousand items.

7.5 Back-of-the-Envelope Estimation

Assumptions: 500M daily users, 4 feed sessions each, 20 items viewed per first page; 100 events per user per day; catalog 1B videos, growing 20M/day.

Quantity	Estimate	Derivation
Feed requests	2B/day ≈ 23k/s avg, 100k/s peak	500M users × 4 sessions; peaks at regional evenings
Feedback events	50B/day ≈ 580k/s avg, 3M/s peak	100 events/user/day: impressions dominate
Candidates per request	5k retrieved → 20 shown	Retrieval channels × quotas; ranking budget

Quantity	Estimate	Derivation
Scoring rate	115M inferences/s avg	23k req/s × 5k candidates — why ranking runs on inference-optimized hardware
User feature store	500 GB	500M users × 1 KB dense features
Item feature/embedding store	1.25 TB	1B items × (1 KB features + 128-dim fp16 embedding)
Training data	10 TB/day	50B events × 200 B per featurized example
Feed page size	50 KB JSON	20 items × metadata + preview refs

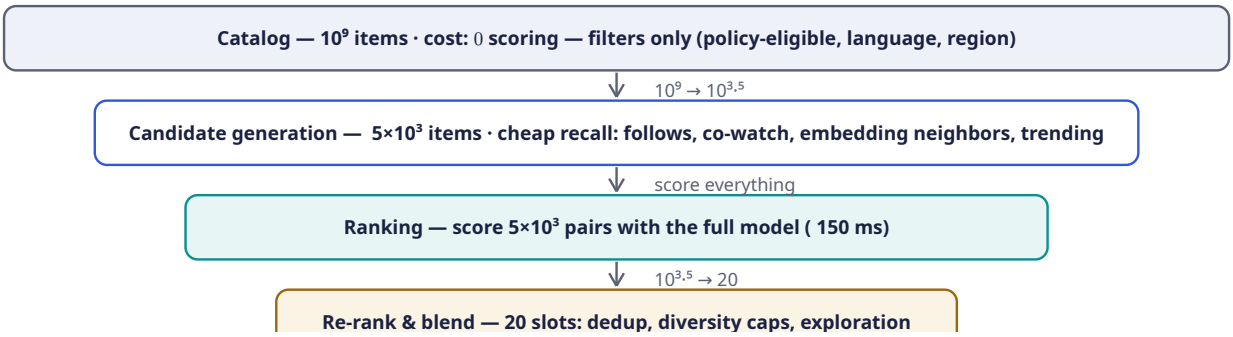
KEY INSIGHT — Three systems wearing one trench coat

The estimation reveals the real architecture: a **telemetry firehose** (580k events/s — Chapter 4 verbatim), a **serving system** (23k requests/s with a 300 ms budget — Chapters 5 and 6 territory), and an **offline factory** (10 TB/day of training data in, one fresh model out daily). The recommendation interview is three familiar interviews stitched by two seams: the feature store and the model store.

The pipeline of the chapter: Section 7.6 derives the funnel; 7.7–7.9 walk its stages (retrieve, rank, re-rank); 7.10–7.12 design APIs, architecture, and the training/serving split; 7.13 implements the core pieces in Rust; 7.14–7.20 scale, harden, and review.

7.6 The Core Challenge: The Funnel

Why must the system be two-staged? Cost arithmetic. A good ranking model costs 10^6–10^7 flops per (user, item) pair. Scoring the whole catalog per request is 10^{15} flops per page view — a data center per user. But **not** scoring is worse: a feed of random items is a dead product. The resolution is to apply cheap relevance filters first and expensive ones last:



COMMON PITFALL — Ranking the catalog

The naive answer — "score every video for every user" — fails by six orders of magnitude of compute (Section 7.5: it would need 10^{11} inferences/s average). Equally wrong in the other direction: **only** candidate channels with no learned ranking, which can't order by the actual objective. The funnel is not an optimization; it is the only shape that satisfies both relevance and the latency budget. State this trade explicitly — it is the pivot of the whole interview.

7.7 Deep Dive: Candidate Generation — Recall at 50 ms

Retrieval runs several **channels** in parallel, each a cheap, independent source of a few hundred to few thousand candidates with a quota in the final mix:

Channel	What it retrieves	How it is served cheaply
Follows	Recent uploads from creators the user follows	Reverse-chronological pull from the subscription index (Chapter 2's pull fanout — feeds are **pulled** here, not pushed)
Co-watch similarity	Items watched by people who watched what you watched	Precomputed item→item similarity table (Section 7.13 computes it); lookup by recent history — no runtime model
Embedding neighbors	Items nearest the user's taste vector	User embedding → approximate nearest-neighbor index; a 10 ms query over 1B vectors
Trending	What's hot globally/regionally right now	Chapter 6's leaderboard: time-decayed watch counts, top-K cached per region
Exploration pool	New/underexposed items earning their first data	A reserved quota (Section 7.9); without it new items starve — the rich-get-richer loop

DEFINITION — Collaborative filtering / co-watch similarity

Collaborative filtering: recommending items via the behavior of **other users** ("users like you liked…"), needing no understanding of content. Its cheapest industrial form is *item-item co-watch similarity*: count how often pairs of items are watched by the same users, normalize into a cosine similarity, and precompute each item's most-similar list offline. Serving a user is then `union(similar-to(x) for x in recent_history)` — table lookups, no math at request time.

DEFINITION — Embedding / ANN index

An *embedding* is a learned dense vector (e.g. 128 floats) representing a user or item such that **distance encodes taste**: items near a user's vector are items they'd probably like (matrix factorization / two-tower models produce them). Retrieval asks for the nearest 500 item vectors to the user vector — over 1B items, done with an *approximate nearest neighbor* (ANN) index (graph- or quantization-based), trading 1% recall for 10 ms latency. Exact search would eat the whole budget.

7.8 Deep Dive: Ranking — Precision at 150 ms

The ranker scores each surviving candidate with the expensive model. As systems engineers we care about **what the model needs at request time**:

- **Item features**: category, duration, language, age since upload, popularity counters, embedding.
- **User features**: historical category affinities, average watch time, creator affinities, embedding.
- **Cross features** (the gold): does this item's category match this user's affinity; similarity between the two embeddings.

- **Context/session features**: time of day, device, and the last N in-session watches — the reason the feed reacts "within the session" (FR-3): these features are updated by the event stream in seconds, **without retraining the model**.

> **DEFINITION — Feature store**
>
> The serving-time database of precomputed ML inputs: user features, item features, and embeddings, keyed for point lookup at ms, refreshed by the event stream (fast path) and batch jobs (slow path). It is the seam between the offline factory and online serving — the ranker never computes features from raw events at request time; it **reads** them.

The model outputs a score per candidate — typically a blend like predicted watch time, or `P(click) × E[watch | click]`. The training label comes from yesterday's implicit feedback: an impression with 80% watch-through is a strong positive; a 2-second skip is a strong negative. **The product's metric choice is the model's soul** — optimize raw clicks and you get clickbait; optimize watch time and you get binge; Section 7.17's guardrail metrics exist because the objective is a policy decision with an engineering delivery mechanism.

7.9 Deep Dive: Re-Ranking — The Last 50 ms

Raw score order is not the feed. The final stage applies product rules to the ranked few hundred:

1. **Dedup** — the same video arriving from three channels appears once.
2. **Diversity caps** — e.g., ≤2 per category/creator per page (Section 7.13 implements this exactly). Uncapped, the ranker collapses the feed to whatever topic the user binged last night — the **filter bubble** failure mode.
3. **Freshness boost** — new uploads from followed creators get a bounded bonus; subscriptions must mean something.
4. **Exploration slot** — reserve 1 of 20 slots for the exploration pool.

> **DEFINITION — Exploration vs. exploitation / multi-armed bandit**
>
> *Exploitation*: show what the model already believes is best. *Exploration*: spend a small quota on items whose value is uncertain, to **learn** — new videos literally cannot be ranked honestly until someone watches them. The *multi-armed bandit* framing formalizes the trade: an ε-greedy policy exploits with probability $1-\varepsilon$ and explores uniformly with probability ε. Without a guaranteed exploration quota, the feedback loop is a closed circle: only shown items get data, only items with data get shown — new creators starve and the catalog fossilizes. Exploration is not charity; it is the system's only source of new information.

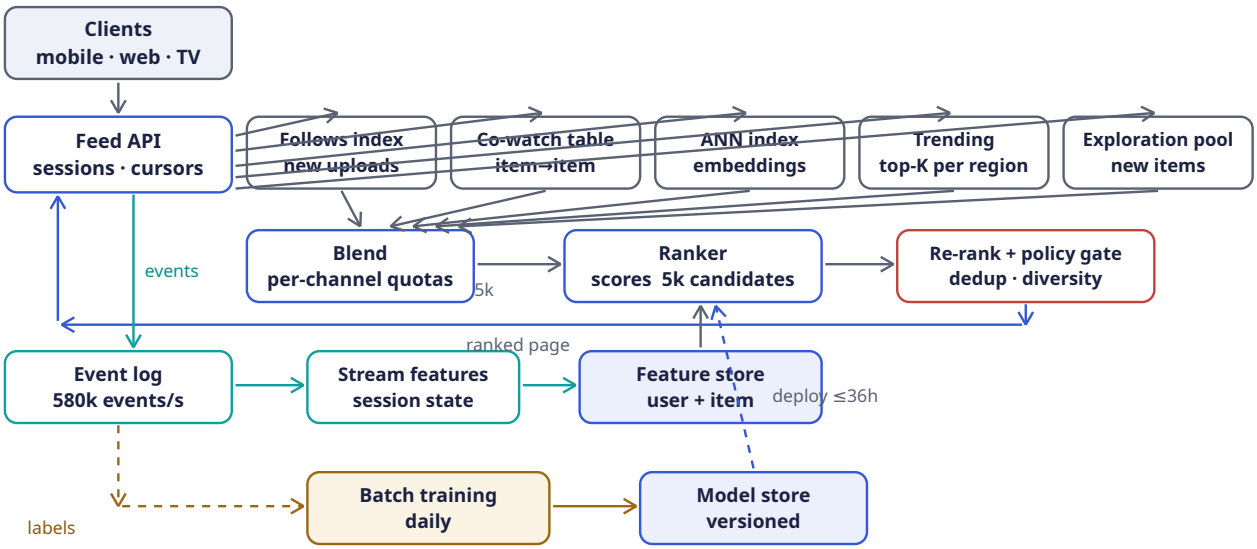
> **KEY INSIGHT — The feed is controlled by whoever sets the caps**
>
> Notice what happened across three sections: the **model** orders candidates, but the **product** owns the final screen — dedup rules, diversity caps, freshness boosts, exploration quota. This is deliberate and healthy: ML optimizes the measurable; humans govern the mixture. In the interview, calling out this separation (and where each lever lives) reads as having operated such a system, not just read about one.

7.10 API Design

Method	Path	Purpose
GET	/v1/feed?cursor=&limit=	Next ranked page (FR-1); cursor encodes session + page state, opaque
POST	/v1/events:batch	Impressions, plays, watch-time, likes, skips — compressed batches (FR-2)
POST	/v1/feedback/not-interested	Negative feedback, effective immediately (FR-5)
GET	/v1/trending?region=	Logged-out / cold-start fallback (FR-6); Chapter 6's board, cached
GET	/v1/feed/explain/{item}	"Why am I seeing this?" — the channel that surfaced it (transparency, cheap to add)

Feed responses carry `{items: [{id, rank, channel}], cursor}`. The `channel` tag is returned on purpose: it powers explainability, per-channel metrics (Section 7.17), and client-side diversity hints. Event ingestion is fire-and-forget for the client (202 Accepted), exactly the Chapter 4 contract — the feed must never block on its own telemetry.

7.11 High-Level Architecture



Component	Responsibility	Why it is shaped this way
Feed API	Session context, cursor state, final assembly	Stateless; the only client-facing door
Candidate channels	Each retrieves 10^2–10^3 plausible items its own way	Independent, parallel, replaceable; quotas blend them (Section 7.7)
Ranker	Scores all candidates with the serving model	The expensive stage: reads the feature store, never computes from raw events
Re-rank + policy	Dedup, diversity caps, freshness boost, exploration slot, safety filter	Product rules the model cannot be trusted with (Section 7.9)

Component	Responsibility	Why it is shaped this way
Event log	Durable buffer for all feedback	Chapter 4's shock absorber, third time in this book
Stream features	Session state and counters updated in seconds	In-session adaptation (FR-3) without model retraining
Feature store	Point-lookup user/item features + embeddings at ms	The seam between offline factory and online serving
Batch training	Daily: labels → features → model → evaluation	Section 7.12; model **internals** are out of scope, its **supply chain** is not
Model store	Versioned models, canary deploys, rollback	A bad model is a deploy incident, treated exactly like bad code

7.12 Training vs. Serving: The Two-Factory Split

The **offline factory** runs daily: join yesterday's 50B events into labeled examples (impression ↔ watch outcome within an attribution window), compute features, train, evaluate against holdout, and publish a versioned model. The **online factory** serves 23k requests/s with that frozen model plus fresh features. Between them sits the discipline this split exists to protect:

> **COMMON PITFALL — Training/serving skew**
>
> The subtlest recommender bug: the model trains on features computed one way (batch SQL over the warehouse) and serves on features computed another (stream jobs with different semantics) — e.g., "watches in last 7 days" counted by calendar week offline and rolling 168 h online. The model silently degrades and no dashboard alarms, because **both** computations are internally correct. Defense: one feature **definition** compiled to both paths (a feature store with shared definitions), plus canary evaluation of every new model on live traffic before promotion. If you name one ML systems pitfall in the interview, name this one.

Freshness policy: the model is ≤36 h old (daily train + evaluation + canary); user/item features ≤1 h; session features ≤10 s. Each layer of staleness is a deliberate price paid for stability, and each is an SLO in Section 7.17.

7.13 Rust Reference Implementations

Four pieces with deterministic tests: item-item collaborative filtering, the diversity re-ranker, ε-greedy exploration, and embedding similarity.

7.13.1 Item-Item Collaborative Filtering

```rust
use std::collections::{HashMap, HashSet};

/// The co-watch relation: user -> items watched, item -> watchers.
/// Implicit positives only — a watch is a vote (Chapter 7.1).
#[derive(Default)]
pub struct WatchMatrix {
    pub by_user: HashMap<u64, HashSet<u64>>,
    pub watchers: HashMap<u64, HashSet<u64>>,
}

impl WatchMatrix {
```

```
    pub fn from_pairs(pairs: &[(u64, u64)]) -> Self {
        let mut m = WatchMatrix::default();
        for &(u, item) in pairs {
            m.by_user.entry(u).or_default().insert(item);
            m.watchers.entry(item).or_default().insert(u);
        }
        m
    }

    /// Cosine similarity over the co-watch relation:
    /// |watchers(a) ∩ watchers(b)| / sqrt(|watchers(a)| * |watchers(b)|).
    /// Precomputed offline per item in production; computed inline here.
    pub fn similarity(&self, a: u64, b: u64) -> f64 {
        let (wa, wb) = match (self.watchers.get(&a), self.watchers.get(&b)) {
            (Some(x), Some(y)) => (x, y),
            _ => return 0.0,
        };
        let (small, big) = if wa.len() <= wb.len() { (wa, wb) } else { (wb, wa) };
        let co = small.iter().filter(|u| big.contains(u)).count() as f64;
        if co == 0.0 { return 0.0; }
        co / (wa.len() as f64 * wb.len() as f64).sqrt()
    }

    /// Recommend unseen items, scored by summed similarity to the user's
    /// whole watch history — the recall channel of Section 7.7.
    pub fn recommend(&self, user: u64, n: usize) -> Vec<(u64, f64)> {
        let seen = self.by_user.get(&user).cloned().unwrap_or_default();
        let mut score: HashMap<u64, f64> = HashMap::new();
        for &item in &seen {
            for &other in self.watchers.keys() {
                if seen.contains(&other) { continue; }
                let s = self.similarity(item, other);
                if s > 0.0 { *score.entry(other).or_default() += s; }
            }
        }
        let mut v: Vec<(u64, f64)> = score.into_iter().collect();
        v.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap().then(a.0.cmp(&b.0)));
        v.truncate(n);
        v
    }
}

#[cfg(test)]
mod cf_tests {
    use super::*;

    fn matrix() -> WatchMatrix {
        // users 1,2,3 ; items 10,20,30,40
        WatchMatrix::from_pairs(&[
            (1, 10), (1, 20),
            (2, 10), (2, 20), (2, 30),
            (3, 30), (3, 40),
        ])
    }

    #[test]
    fn similarity_counts_co_watchers() {
        let m = matrix();
        assert_eq!(m.similarity(10, 20), 1.0);       // identical audiences
        assert_eq!(m.similarity(10, 30), 0.5);       // 1 shared of 2 & 2
```

```
        assert!((m.similarity(30, 40) - 0.7071).abs() < 1e-3); // 1/sqrt(2)
        assert_eq!(m.similarity(10, 40), 0.0);       // no shared watchers
    }

    #[test]
    fn recommendations_exclude_seen_and_rank_by_evidence() {
        let m = matrix();
        let recs = m.recommend(1, 5);
        // user 1 watched {10, 20}; item 30 scores sim(10,30)+sim(20,30) = 1.0,
        // item 40 scores 0 and is not surfaced
        assert_eq!(recs, vec![(30, 1.0)]);
        assert!(!recs.iter().any(|&(id, _)| id == 10 || id == 20));
    }
}
```

7.13.2 The Re-Ranker: Dedup + Diversity Quotas

```
use std::collections::{HashMap, HashSet};

#[derive(Debug, Clone)]
pub struct Candidate {
    pub id: u64,
    pub category: &'static str,
    pub score: f64,
}

/// Same item arriving from several channels: keep its best score, once.
pub fn dedup(ranked: Vec<Candidate>) -> Vec<Candidate> {
    let mut seen = HashSet::new();
    ranked.into_iter().filter(|c| seen.insert(c.id)).collect()
}

/// Take a page of n from score order, allowing at most `per_category`
/// items per category — pass 1 respects the cap, pass 2 fills any
/// remaining slots in score order so the page is never short.
pub fn rerank_with_quota(
    mut ranked: Vec<Candidate>,
    n: usize,
    per_category: usize,
) -> Vec<u64> {
    ranked.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap().then(a.id.cmp(&b.id)));
    let mut out = Vec::with_capacity(n);
    let mut counts: HashMap<&str, usize> = HashMap::new();
    let mut leftover = Vec::new();
    for c in ranked {
        let cnt = counts.entry(c.category).or_insert(0);
        if *cnt < per_category && out.len() < n {
            out.push(c.id);
            *cnt += 1;
        } else {
            leftover.push(c.id);
        }
    }
    for id in leftover {
        if out.len() < n { out.push(id); }
    }
    out
}
```

```rust
#[cfg(test)]
mod rerank_tests {
    use super::*;

    fn c(id: u64, cat: &'static str, score: f64) -> Candidate {
        Candidate { id, category: cat, score }
    }

    #[test]
    fn quota_caps_a_dominant_category_but_fills_the_page() {
        let ranked = vec![
            c(1, "sport", 9.0), c(2, "sport", 8.5), c(3, "sport", 8.0),
            c(4, "music", 7.5), c(5, "news", 7.0),
        ];
        // cap 2 per category, page of 5
        assert_eq!(rerank_with_quota(ranked, 5, 2), vec![1, 2, 4, 5, 3]);
    }

    #[test]
    fn dedup_keeps_first_best_occurrence() {
        let dup = vec![c(1, "sport", 9.0), c(1, "sport", 7.0), c(2, "news", 8.0)];
        let unique = dedup(dup);
        assert_eq!(unique.len(), 2);
        assert_eq!(unique[0].score, 9.0); // first occurrence wins
    }
}
```

7.13.3 ε-Greedy Exploration (Multi-Armed Bandit)

```rust
/// Tiny deterministic RNG (xorshift64) so tests are reproducible.
pub struct XorShift(pub u64);
impl XorShift {
    pub fn next(&mut self) -> u64 {
        let mut x = self.0;
        x ^= x << 13; x ^= x >> 7; x ^= x << 17;
        self.0 = x;
        x
    }
    pub fn f64(&mut self) -> f64 { (self.next() >> 11) as f64 / (1u64 << 53) as f64 }
    pub fn below(&mut self, n: u64) -> u64 { self.next() % n }
}

/// ε-greedy: exploit the best-estimate arm, explore uniformly with
/// probability ε. Arms = channels or new-item pools earning data.
pub struct EpsilonGreedy {
    pub means: Vec<f64>,
    pub counts: Vec<u64>,
    pub epsilon: f64,
    rng: XorShift,
}

impl EpsilonGreedy {
    pub fn new(arms: usize, epsilon: f64, seed: u64) -> Self {
        EpsilonGreedy {
            means: vec![0.0; arms],
            counts: vec![0; arms],
            epsilon,
            rng: XorShift(seed.max(1)),
        }
```

```
    }

    pub fn pull(&mut self) -> usize {
        // initialize: every arm once, in order
        if let Some(i) = self.counts.iter().position(|&c| c == 0) { return i; }
        if self.rng.f64() < self.epsilon {
            self.rng.below(self.means.len() as u64) as usize
        } else {
            self.means
                .iter()
                .enumerate()
                .max_by(|a, b| a.1.partial_cmp(b.1).unwrap())
                .map(|(i, _)| i)
                .unwrap()
        }
    }

    /// Incremental mean update — the bandit learns from implicit feedback.
    pub fn record(&mut self, arm: usize, reward: f64) {
        self.counts[arm] += 1;
        let n = self.counts[arm] as f64;
        self.means[arm] += (reward - self.means[arm]) / n;
    }
}

#[cfg(test)]
mod bandit_tests {
    use super::*;

    #[test]
    fn zero_epsilon_exploits_after_initialization() {
        let true_means = [0.1, 0.5, 0.9];
        let mut b = EpsilonGreedy::new(3, 0.0, 42);
        let mut counts = [0u64; 3];
        for _ in 0..103 {
            let a = b.pull();
            counts[a] += 1;
            b.record(a, true_means[a]);
        }
        assert_eq!(counts, [1, 1, 101]); // 3 init pulls, then all arm 2
    }

    #[test]
    fn pure_exploration_spreads_uniformly() {
        let true_means = [0.1, 0.5, 0.9];
        let mut b = EpsilonGreedy::new(3, 1.0, 88_172_645_463_325_252);
        let mut counts = [0u64; 3];
        for _ in 0..303 {
            let a = b.pull();
            counts[a] += 1;
            b.record(a, true_means[a]);
        }
        assert_eq!(counts.iter().sum::<u64>(), 303);
        for &c in &counts {
            assert!((60..=140).contains(&c), "counts = {counts:?}");
        }
    }

    #[test]
    fn running_mean_converges() {
```

```
        let mut b = EpsilonGreedy::new(1, 0.0, 1);
        b.record(0, 1.0);
        b.record(0, 0.0);
        assert_eq!(b.means[0], 0.5);
    }
}
```

7.13.4 Embedding Similarity: Cosine Top-K

```
/// Cosine similarity between taste vectors; distance encodes preference.
pub fn cosine(a: &[f64], b: &[f64]) -> f64 {
    let dot: f64 = a.iter().zip(b).map(|(x, y)| x * y).sum();
    let na = a.iter().map(|x| x * x).sum::<f64>().sqrt();
    let nb = b.iter().map(|x| x * x).sum::<f64>().sqrt();
    if na == 0.0 || nb == 0.0 { 0.0 } else { dot / (na * nb) }
}

/// Exact nearest neighbors for a user vector. Honest scope note: this is
/// O(n·d) — the teaching core. Production swaps in an ANN index
/// (Section 7.7) with identical interface and ~99% of the recall.
pub fn top_k_similar(
    query: &[f64],
    items: &[(u64, Vec<f64>)],
    k: usize,
) -> Vec<(u64, f64)> {
    let mut scored: Vec<(u64, f64)> =
        items.iter().map(|(id, v)| (*id, cosine(query, v))).collect();
    scored.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap().then(a.0.cmp(&b.0)));
    scored.truncate(k);
    scored
}

#[cfg(test)]
mod ann_tests {
    use super::*;

    #[test]
    fn nearest_neighbors_by_direction_not_magnitude() {
        let q = vec![1.0, 0.0, 0.0];
        let items = vec![
            (1, vec![0.9, 0.1, 0.0]),  // nearly aligned
            (2, vec![0.0, 1.0, 0.0]),  // orthogonal
            (3, vec![5.0, 0.0, 0.0]),  // identical direction, 5x magnitude
        ];
        let top2 = top_k_similar(&q, &items, 2);
        let ids: Vec<u64> = top2.iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![3, 1]);          // direction beats magnitude
        assert_eq!(top2[0].1, 1.0);           // perfectly aligned
        assert!((top2[1].1 - 0.9939).abs() < 1e-3);
        assert_eq!(cosine(&q, &items[1].1), 0.0);
    }
}
```

7.14 Scaling the Platform

Layer	Scale axis	Mechanism
Feed API / ranker	23k req/s avg	Stateless services, horizontal; ranking replicas scale with scoring rate (115M inferences/s avg) on inference-optimized hardware
Candidate channels	Catalog size	Each channel scales independently: follows index (Chapter 2 pull), precomputed co-watch table (offline), ANN shards (embedding space partitioned), trending cache (Chapter 6)
Event pipeline	580k events/s avg, 3M/s peak	Chapter 4 verbatim: partitioned durable log, stream processors for session features, batch for training
Feature store	500 GB user + 1.25 TB item	Sharded key-value with point lookups; read-heavy, so replicas carry the load
Training	10 TB/day examples	Data-parallel GPU/TPU fleet; the daily cadence bounds both cost and staleness
Feed cache	Logged-out / cold-start	Trending pages are shared and cached hard; personalized pages are never shared — cache the **candidates**, not the feed

KEY INSIGHT — Personalization is cache-hostile by definition

Chapter 5 cached windows shared by millions; here **every response is unique**. The caching strategy inverts: cache the shared ingredients (trending boards, co-watch lists, item features, embeddings) and assemble the personalized result fresh. A recommender that caches feeds has accidentally built trending-with-extra-latency.

7.15 Failure Modes & Degradation

Failure	Symptom	Response
Event pipeline lag	Session features stale; feed feels unreactive	Degrade gracefully to non-session features; lag alerts (Chapter 4); catch-up replay
One candidate channel down	Feed still full, less varied	Quotas redistribute to surviving channels; the blend is designed for exactly this — no single point of taste
Ranker overload	Latency budget breached	Score fewer candidates (5k → 1k): relevance drops imper-

Failure	Symptom	Response
		ceptibly before latency does; then trending fallback
Bad model deploy	Feed quality craters at scale	Canary + business-metric guardrails on the deploy pipeline; one-click rollback to previous model version — model incidents are deploy incidents
Feature store stale	Model serves on old affinities	Staleness SLO with alerts; beyond threshold, ranker down-weights stale features
Training pipeline fails	Model ages past 36 h	Yesterday's model keeps serving; freshness alert pages; investigate like any failed batch job
Feedback loop spiral	One topic floods a user's feed	Diversity caps bound the damage structurally; guardrail metrics (Section 7.17) detect drift
Trending fallback poisoned	Manipulated trending board	Chapter 6's integrity tools apply: velocity anomalies, quarantine feed

7.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Architecture	Funnel: retrieve → rank → re-rank	Single-stage scoring of the catalog: 6 orders of magnitude over compute budget (Section 7.6); retrieval-only: no objective-driven order
Retrieval	Multi-channel blend with quotas	One giant embedding index alone: best average relevance, worst failure modes — no guarantee follows or fresh uploads ever surface
Model freshness	Daily retrain + real-time session features	Online learning (update weights per event): fresher, but a feedback-loop amplifier and an operational nightmare to roll back
Objective	Watch time with guardrail metrics	Pure CTR: optimizes clickbait; pure satisfaction surveys: too sparse to train on
Exploration	Guaranteed quota (ε-greedy slot)	Pure exploitation: the rich-get-richer loop fossilizes the catalog; Thompson

Decision	Chosen	Alternative & why not (here)
		sampling: better theory, same quota slot in practice
Feed consistency	Cursor paginates over a snapshot	Re-rank every page live: infinite scroll jumps and repeats; the session cursor pins the candidate set
Cold start	Trending + category picks, then rapid session adaptation	Signup interest quizzes: fine complement, cannot be the only path (friction, stale answers)

7.17 Observability & SLOs

Two metric families — system health (Chapter 4's bread and butter) and **quality** health, which only this kind of system has:

Indicator	Definition	Target
Feed latency	Page generation p95	≤ 300 ms
Event lag	Event → session feature visible	≤ 10 s
Model freshness	Age of serving model	≤ 36 h, alert past
Candidate coverage	Pages served with all channels contributing	≥ 99%
CTR / watch time	Clicks and minutes per impression, per model version	Guardrail: never ship a regression
Diversity index	Distinct categories/creators per page	≥ policy floor
Exploration health	Share of new items reaching N impressions in 1 h	≥ 95%
Negative-feedback rate	"Not interested" per impression	Watch trend, alert on spikes

KEY INSIGHT — Guardrail metrics are the product's conscience

Every optimization target can be gamed by the optimizer itself: CTR invites clickbait, watch time invites doomscrolling. Production recommenders therefore ship **pairs** of metrics — the objective and the guardrails (diversity, negative feedback, long-term retention) — and a model that improves the objective while moving a guardrail does not ship. Encoding this in the deploy pipeline is systems work, and it is the difference between a recommender and a slot machine.

7.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Make it real-time: the feed reacts to every watch instantly."** Already half-built: session features update in seconds. The next step — updating the **user embedding** mid-session — is streaming inference, not training: recompute the user vector from the new history with the frozen item tower. True online weight updates trade the rollback story for freshness; say what you'd be giving up.

2. **"A new creator with zero viewers — how do they ever surface?"** The exploration quota (Section 7.9) plus **content-based** features: with no watch history, the video's own metadata (category, title embeddings, thumbnail features) place it near known items. First impressions go to users whose taste vector is near the **content** vector — cold start is why content features exist at all.
3. **"TikTok vs. YouTube — what differs in the system?"** The graph: YouTube leans on subscriptions (the follows channel is strong); TikTok is graph-free — nearly pure engagement ranking, which makes its exploration quota and per-session adaptation much larger shares of the mix. Same funnel, different quotas. Architecture is the constants.
4. **"Where do ads fit?"** A second ranking system with its own objective (expected revenue × relevance), blended into slots by an auction — and the reason feed slots are strictly partitioned between organic and sponsored before re-ranking.
5. **"How would you detect the model amplifying harmful content?"** Guardrail metrics per content class, policy-gate recalls, and audit sampling of what the exploration pool promotes — plus the honest answer: ranking amplifies whatever the objective correlates with, so this is fought at the objective-and-guardrail layer, not with filters alone.

7.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> A recommender is three systems in a trench coat: a telemetry firehose (Chapter 4), a latency-bound serving funnel, and a daily model factory. The funnel exists because of arithmetic: you may score 5k of 10^9 items per request, so cheap recall channels (follows, co-watch, embedding ANN, trending, exploration) feed one expensive ranker, and a re-ranker applies the product's conscience — dedup, diversity caps, freshness, policy. Implicit feedback is the fuel; the feature store is the seam that keeps training and serving honest (skew is the killer bug); exploration is the quota that keeps the feedback loop open; guardrail metrics decide what ships. The lesson that transfers: **in ML systems, the model is a component — the system around it is the design.**

Further reading.

- The source video: *"22: Recommendation Engine (YouTube, TikTok) — Systems Design Interview Questions With Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=QrZTmiZSRcw`
- Covington, Adams, Sargin — *"Deep Neural Networks for YouTube Recommendations"* (2016) — the canonical two-stage funnel paper this chapter's architecture follows.
- Koren, Bell, Volinsky — *"Matrix Factorization Techniques for Recommender Systems"* (2009) — embeddings before they were called embeddings.
- Sutton & Barto, *Reinforcement Learning* — the multi-armed bandit chapters behind Section 7.9's exploration quota.
- Chapter 4's monitoring reading applies twice here — guardrail metrics are SLOs with a conscience.

7.20 Chapter Glossary

Term	Meaning
A/B test	Controlled experiment over model/parameter versions; how guardrail and objective metrics earn their keep
ANN index	Approximate nearest-neighbor structure over embeddings; 1% recall traded for 10 ms latency over 1B vectors

Term	Meaning
Candidate generation	The recall stage: cheap channels selecting 10^3 plausible items from the catalog
Clickbait / CTR trap	The failure of optimizing clicks alone: the model learns to promise, not to satisfy
Cold start	Serving users or items with no history: trending/category fallbacks for users, content features + exploration for items
Collaborative filtering	Recommendation from other users' behavior, no content understanding required
Co-watch similarity	Item-item cosine over shared watchers; precomputed offline, looked up at serve time
Cross feature	A feature combining user and item (e.g., category × affinity); where ranking accuracy concentrates
Diversity cap	Re-rank rule bounding any category/creator per page; structural filter-bubble defense
Embedding	Learned dense vector for a user or item; distance encodes taste
ε-greedy	Bandit policy exploiting the best arm with probability $1-\varepsilon$, exploring uniformly otherwise
Exploration quota	Reserved feed slots for uncertain items; the only source of new information in the loop
Feature store	Serving-time database of precomputed user/item features with shared offline/online definitions
Feedback loop (rich-get-richer)	Shown items get data, data earns impressions; without exploration the catalog fossilizes
Filter bubble	Feed collapse to one topic under pure score order; fought with diversity caps
Funnel	The retrieve → rank → re-rank shape; a compute-budget necessity, not an optimization
Guardrail metric	A metric that may not regress even when the objective improves; ships alongside the objective
Implicit feedback	Behavioral signals (watch time, skips, impressions) used as training labels
Latency budget	300 ms p95 allocated across retrieval, ranking, re-ranking; dictates what must be precomputed
Matrix factorization / two-tower	Model families producing user and item embeddings from interaction history
Model store	Versioned model registry with canary deploys and rollback; models are deploys
Multi-armed bandit	The exploration/exploitation formalism: arms are channels or items, rewards are engagement

Term	Meaning
Ranking	The precision stage: scoring retrieval's candidates with the full model under 150 ms
Re-ranking	The product-rules stage: dedup, diversity, freshness, policy, exploration slot
Session features	In-session signals (last N watches) updated in seconds; feed reactivity without retraining
Training/serving skew	Feature definitions diverging between offline training and online serving; the silent killer of recommenders
Trending	Time-decayed popularity board per region — Chapter 6's leaderboard reused; the cold-start and outage fallback

— End of Chapter 7 · Next: Chapter 8 —